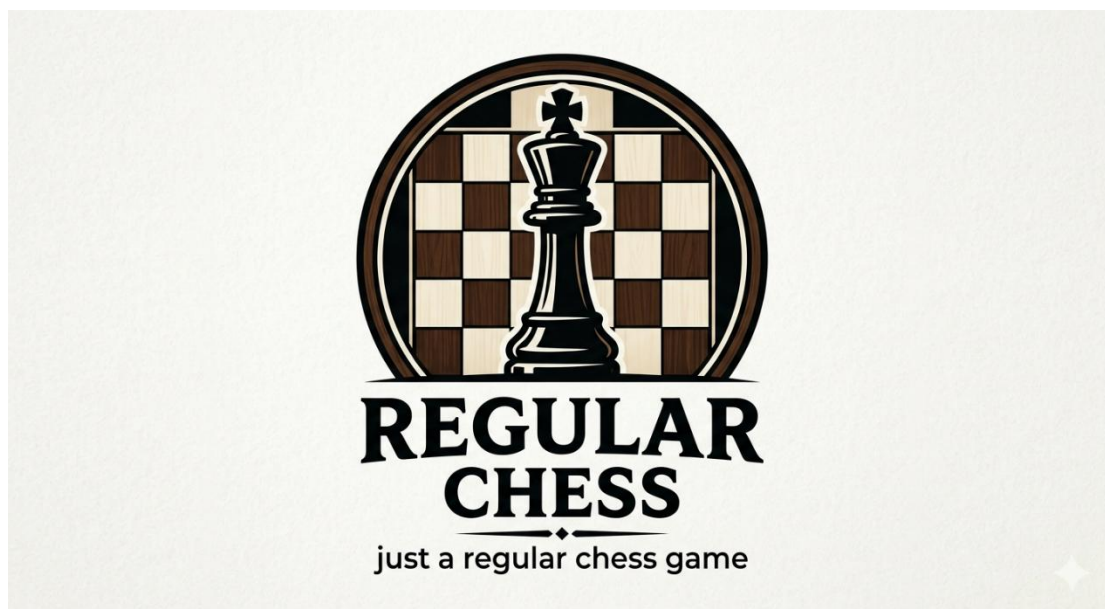


תיק פרויקט



שם: שלמה נהוראי בוזנח

העבודה: **Regular Chess**

ת"ז: 331699215

בית הספר: אורט "אמירים" בית שאן

החלופה: תכנון ותכנות מערכות

תאריך הגשה: מאי 2026

מנחה: מיטל מנוטשרי



Regular Chess

תוכן עניינים

2	תוכן עניינים
3	מבוא
3	הרקע לפרויקט
3	תהליך המחקר
3	אתגרים מרכזיים
4	טבלת דרישות - ע"פ דרישות משרד החינוך
5	תיאור תחום הידע – פרק מילולי
5	אובייקטים נחוצים + סוגי נתונים
8	ייצוג מידע
8	באפליקציה שלי אני מייצג את המידע של הסטטיסטיקות מתוך FireBasen בעזרת מבנים שאני בנית.
8	תיאור פעולות על המידע
10	מבנה / ארכיטקטורה של הפרויקט
10	תכנון ותיעוד מסכי הפרויקט
17	בחירת מחלקות הפרויקט והקשרים ביניהם
18	מימוש הפרויקט
18	מחלקות הפרויקט
81	בסיס נתונים
103	מדריך למשתמש
109	סיכום אישי / רפלקציה
110	ביבליוגרפיה
111	נספחים

Regular Chess

מבוא

הרקע לפרויקט

הפרויקט "Regular Chess". פלטפורמה שנועדה לאפשר לשחק את המשחק הפופולרי "שחמט" דרך אותו טלפון עם חברים. המשחק נועד להתאים לגילאים 6+ ומאפשרת חווית משחק נוחה ופשוטה. בחרתי בשחמט כי הוא המשחק ילדות שלי ושל מלא ילדים ברחבי העולם. בנוסף, בחרתי בשחמט כנושא המשחק שלי כי זה משחק מוכר, מורכב ומגבש.

תהליך המחקר

לפני שקפצתי לאפליקציה שלי חקרתי על עולם השחמט הקיים שהתחלק עבור לכמה נושאים.

חוקים -> אני הכרתי הרבה חוקים בשחמט תנועות של שחקנים היה חלק מרכזי בזה, כגון: צריח (ROOK) שיכול לזוז רק ימינה או שמאלה, פרש (KNIGHT) שיכול לזוז שתי קוביות לאיזה כיוון שהוא מחליט ואז עוד קוביה אחת לעוד כיוון (חוץ מכיוון שהוא בחר בהתחלה ואחורה לכיוון הזה).

היו מלא חוקים מסובכים שהייתי צריך ללמוד מחדש כמו Castle, Pawn Promotion ועוד.

כמובן שיש עוד הרבה נושאים לשחמט סוגי משחק (blitz, bullet ועוד), דירוג לפי נקודות ועוד.

אתרים ואפליקציות -> כמובן שיש את האתר המפורסם ביותר Chess.com שהוא היה המקור להשראה שלי. מאתר זה השגתי את כל ההבנה שלי שאיך האפליקציה אמורה להיות בנויה, הוספת מסך סטטיסטיקות, ועיצוב. בניגוד לאתר הזה אני לא השתמשתי במלא סגנונות משחק או בשיטת ניקוד כמו שלהם

יש עוד מלא אתרים ואפליקציות לשחמט חלקם מוצלחות חלקם לא, לדוגמה: Lichess.org, [ChessKid](http://ChessKid.com), Chess24.com.

אתגרים מרכזיים

במהלך פיתוח אפליקציה השחמט נתקעתי בכמה אתגרים חלק מהאתגרים היו אתגרים שציפיתי להן וחלק פגשתי תוך כדי תנועה.

– התמודדות עם האלגוריתמיקה והחוקים של כל המשחק. חוץ מתזוזות השחקנים יש גם כל מיני מצבים במשחק שחייב להתייחס אליהם תמיד צריך להתייחס למצב שאולי יתקיים שח או שחמט.

– התמודדות בסביבת עבודה חדשה ב android studio שבשבילי היא מאוד איטית ולא נוחה לשימוש ולפעמים יצרה אצלי תסכול

– התמודדות עם Debuging. בקוד כזה ארוך עם לוגיקה טיפה מסובכת תמיד יהיו בעיות שנמצא תוך כדי תנועה אם זה למקם את החיילים הלבנים במקום השחורים או מהלך שלא בדקתי שגרם לקריסה/הפעולה לא חוקית בשחמט

– סנכרון הקוד ברקע של הלוח שאני בנית לעיצוב מה שהיה ממש מאתגר הייתי צריך ללמוד על BitMap ואיך משתמשים בזה בקוד ואיך זה עוזר לי להציג את הנתונים (חיילים) באופן אחיד עם הלוח שבנתי.

הצלחתי לעמוד באתגרים ולסיים את הפרויקט בהצלחה בזכות המחקר המעמיק, סרטונים ביוטיוב, חברים, בינה מלאכותית והמורה שלי במהלך השנה דויד ברעם.

בזכות כל האתגרים האלו יצא לי לחוות על בשרי איך בונים משחק מאפס. מהעיצוב (Frontend) שכל המסכים עד לכתיבת הקוד (Backend) שהפך את כל השלד שבנתי למשחק מתפקד. בנוסף למדתי איך שחמט זה משחק טקטי ומורכב מלא באינספור מהלכים ואיך צריך תכנון מאוד מעמיק בשביל להתחיל לגשת לכתיבת קוד במשחק כזה.

הצורך עליו המשחק עונה הוא על האפשרות לשחק עם כל חבר מטלפון אחד. ככל שאתה משחק יותר ככה הניקוד שלך עולה ובכך מרוויח יותר ניסיון בתחום השחמט. מה שבעיה למי שלא באמת מכיר את המשחק ולא רוצה להתחיל לשחק מול שחקנים טובים.



Regular Chess

טבלת דרישות - ע"פ דרישות משרד החינוך

מספר הדרישה ע"פ טבלת משרד החינוך.	יישום הדרישה בפרויקט
סעיף 6 Alarm Manager + Notification	שליחת התראות חדשות ללקוח להיכנס ולשחק כל פעם שהוא שוהה בלובי
סעיף 7 שימוש ב-Firebase	שמירה ושליפה של נתונים, שימוש בטבלאות של Firestore של משתמש, סטטיסטיקות
סעיף 8	בחירה אחת מסעיף 9 2 בחירות מסעיף 10
סעיף 9 יירושה ממחלקת View	הורשה בשביל מחלקת ChessView שאחראית על כל העיצוב של המשחק עצמו מהצגת הטיימרים עד הלוח והשחקנים הכל בזמן אמת
סעיף 10 Countdown Timer	שעון ספירה לאחור של כל תור שעוצר אחרי ביצוע המהלך וממשיך אחרי ביצוע המהלך של היריב
סעיף 10 SharedPreferences	העברת מידע ללובי המשחק ממסך ההתחברות וממסך המשחק כדי לדעת אם לשלוח את הסטטיסטיקות מהFirestore או לעדכן אותם בFirestore



Regular Chess

תיאור תחום הידע – פרק מילולי

אובייקטים נחוצים + סוגי נתונים

טבלת סטטיסטיקות:

תפקיד הטבלה - באפליקציה ישנה דף סטטיסטיקות שמיועדת לשמור את כל עבר הניסיון של השחקן באפליקציה. הטבלה שומרת את כל הנתונים האלו כגון: שם משתמש, מספר ניצחונות, מספר הפסדים ועוד..

שדות הטבלה:

שם השדה	הסבר
String User Uid	מחרוזת הזהות של המשתמש שיחודית לכל משתמש ונוצרה על ידי Firestore
Long gamesPlayed	מספר כל המשחקים השלמים ששיחק הלקוח לא כולל המשחקים שהוא פרש מהם.
Long losses	מספר כל ההפסדים שהפסיד הלקוח.
Long stalemates	מספר כל מצבי התיקו שהגיע הלקוח אליהם.
Long timespent	הזמן בדקות ששיחק הלקוח אי פעם במשחק באפליקציה.
String username	מחרוזת שם הלקוח שמאותחל כהתחלה של האימייל וזמין לשינוי בכל עת השימוש באפליקציה
Long wins	מספר כל הניצחונות שניצח הלקוח



Regular Chess

תמונה מתוך Firebase:

🏠 > Statistics > IYNcPfwlf2hxq...

(default)

+ Start collection

Statistics >

📄 Statistics

+ Add document

IYNcPfwlf2hxqCLYPFB... >

P3L5oRdKqbvOwbmJNLE6

IYNcPfwlf2hxqCLYPFBcroMawYm2

+ Start collection

+ Add field

Uid: "IYNcPfwlf2hxqCLYPFBcroMawYm2"

gamesPlayed: 1

losses: 1

stalemates: 0

timespent: 0

username: "buzzi"

wins: 0



Regular Chess

טבלת אותנטיקציה:

תפקיד הטבלה - טבלת האותנטיקציה ב-Firebase Authentication (Auth) אחראית על ניהול תהליך ההזדהות והאימות של המשתמשים בתוך האפליקציה. היא מספקת פתרון מאובטח ונוח לניהול הרשאות גישה למשתמשים, ומספקת מספר אפשרויות להזדהות לפי הצורך של המערכת.

תפקידי טבלת האותנטיקציה הם: הרשמה, התחברות.

שדות הטבלה:

שם השדה	הסבר
String User Uid	מחרוזת הזדהות של המשתמש שיחודית לכל משתמש ונוצרה על ידי Firestore
String created	מחרוזת השומרת את התאריך שבו נוצר המשתמש.
String SignedIn	מחרוזת השומרת את תאריך ההתחברות, לאחרונה של המשתמש.
String identifier	מחרוזת השומרת – מייל של משתמש במערכת.

תמונה מתוך Firebase:

Search by email address, phone number, or user UID					Add user	⌂	⋮
Identifier ↓	Providers	Created ↓	Signed In	User UID			
rrr@gmail.com	✉	Jan 30, 2026	May 22, 2026	IYNcPfwf2hxqCLYPFBcroMa...			
dddd@dd.com	✉	Dec 12, 2025	Dec 12, 2025	iDCMwIXRTaZyPuj5S22RIwnF...			
dddd@ddd.com	✉	Dec 12, 2025	Dec 12, 2025	aKMaIQ01C2bl1ohnpzmQY3...			
user@gmail.com	✉	Dec 12, 2025	Dec 12, 2025	wPU1XakK3ATy5jCZsjSD1CBH...			
Rows per page: 50					1 – 4 of 4	<	>



Regular Chess

ייצוג מידע

באפליקציה שלי אני מייצג את המידע של הסטטיסטיקות מתוך Firebaseen בעזרת מבנים שאני בנית. להלן המבנים:

שם המבנה	הסבר
UserStatistics userstats	מבנה זה מייצג את כל הערכים שנמצאים בטבלת הסטטיסטיקות
GlobalStat globalStat	מבנה זה מכיל את משתנה הסטטיסטיקות של הלקוח ומשתנה בוליאני של האם הסטטיסטיקות של הלקוח עלו ל-Firebase המשתנה הזה הוא משתנה גלובלי ונותן את האפשרות לשנות אותו בכל מקום בקוד

תיאור פעולות על המידע

שם הפעולה	מחלקה	הסבר	סוג הפעולה
private void registerUser(String email, String password)	signinPage	פעולה שמוסיפה משתמש חדש בטבלת אותנטיקציה ב-Firebase ואם כן היא שולחת את המשתמש ללובי של התחברות כדי לבצע חיבור	הוספה
private void loginClient(String txtLogemail, String txtLogPw)	loginPage	פעולה שבודקת אם המשתמש קיים בטבלת אותנטיקציה ב-Firebase ואם כן הוא מעדכן את ה-UID ולפי זה שולף את שאר הנתונים ומעביר את הלקוח לחלון של הלובי של המשחק	קריאה
public static void deleteUserAccount(Context context, final DeleteAccountCallback callback)	DatabaseManager	פעולה שמוחקת את המשתמש המחובר לפי ה-UID שלו היא מוחקת גם את טבלת הסטטיסטיקות שלו וגם את טבלת האותנטיקציה שלו	עדכון



Regular Chess

קריאה	פעולה שמעלה את שם הלקוח מטבלת הסטטיסטיקות למשתנה הגלובלי לפי ה UID של הלקוח	DatabaseManager	public static void loadUserProfile(Context context, String uid, final UserProfileCallback callback)
קריאה	פעולה שמעלה את כל הנתונים של הלקוח מטבלת הסטטיסטיקות למשתנה הגלובלי לפי ה UID של הלקוח	DatabaseManager	public static void loadStatsFromFirebase(Context context, final DataStatusCallback callback)
עדכון	פעולה שמעדכנת את הנתונים בטבלת הסטטיסטיקות לפי Firestore במשתנה הגלובלי	DatabaseManager	public static void saveStatsToFirebase(Context context)
עדכון	פעולה שמעדכנת את השם משתמש של הלקוח בטבלת הסטטיסטיקות לפי Firestore במשתנה הגלובלי	DatabaseManager	public static void updateUsername(String uid, String newUsername)

Regular Chess

מבנה / ארכיטקטורה של הפרויקט

תכנון ותיעוד מסכי הפרויקט

מסך SPLASH:

מסך הספלאש מציג מסך טעינה של חלק מכלי השחמט למשך 3 שניות, ואז באופן אוטומטי יעביר את המשתמש למסך לובי ההתחברות.



מסך לובי ההתחברות:

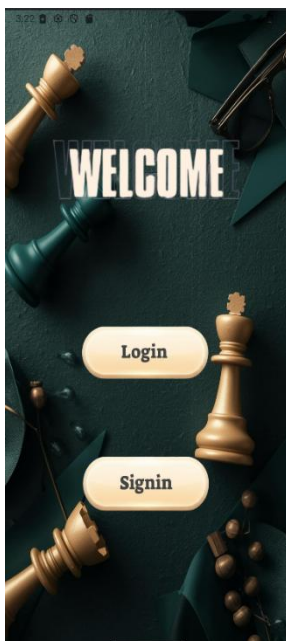
מסך לובי ההתחברות הוא מסך שהייה שבו הלקוח יכול להחליט אם לגשת לדף הירשמות כדי ליצור משתמש חדש, או לגשת לדף ההתחברות כדי להתחבר למשתמש קיים.

במסך ישנם שני כפתורים ותמונה אחת:

תמונה ברוך הבא, "welcome", אלמנט מסוג ImageView

כפתור Signin, כפתור שמעביר לדף ההירשמות, אלמנט מסוג ImageButton.

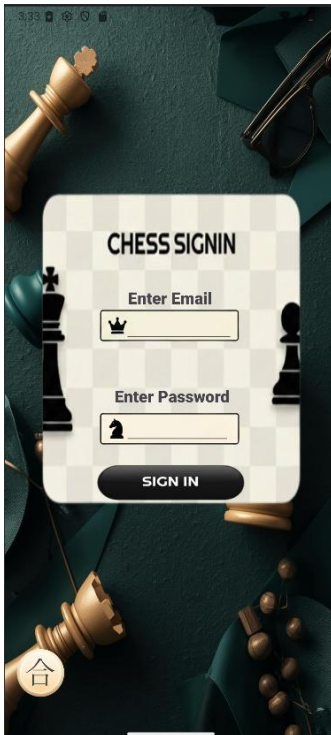
כפתור Login, כפתור שמעביר לדף ההתחברות, אלמנט מסוג ImageButton.



Regular Chess

מסך ההירשמות:

מסך ההירשמות זה מסך שמאפשר לך להקליד את האימייל והסיסמה של החשבון שאתה רוצה לפתוח, ואז ללחוץ על כפתור ההירשמות, וזה יפתח חשבון חדש בטבלת אותנטיקציה ב-Firebase, ואז יעביר אותך שוב ללובי ההירשמות כדי שתוכל להתחבר לחשבון.



ישנם במסך הזה שלוש תמונות שני תיבות טקסט שני הכנסות טקסט ושני כפתורים:

תמונת חלונית ההירשמות, עם טקסט של "CHESS SIGNIN", אלמנט מסוג ImageView

תמונת תיבת האימייל, מלבן עם כתר של מלכה, אלמנט מסוג ImageView. תמונת תיבת סיסמה, מלבן עם פרש, אלמנט מסוג ImageView.

תיבת טקסט, "Enter Email", אלמנט מסוג TextView.

תיבת טקסט, "Enter Password", אלמנט מסוג TextView.

הכנסת טקסט, הכנסת אימייל הלקוח, אלמנט מסוג EditText.

הכנסת טקסט, הכנסת סיסמת הלקוח, אלמנט מסוג EditText.

כפתור SIGN IN, כפתור שרושם אותך בבסיס נתונים ומעביר אותך להתחבר, אלמנט מסוג ImageButton.

כפתור חזרה, כפתור שמחזיר אותך ללובי ההתחברות, אלמנט מסוג ImageButton.

מסך ההתחברות:

מסך ההתחברות זה מסך שמאפשר לך להקליד את האימייל והסיסמה של החשבון הקיים שלך, ואם הסיסמה והאימייל נמצאו בטבלת האותנטיקציה הוא יעביר אותך אל מסך הלובי של המשחק.

ישנם במסך הזה שלוש תמונות שני תיבות טקסט שני הכנסות טקסט ושני כפתורים:

תמונת חלונית ההתחברות, עם טקסט של "CHESS LOGIN", אלמנט מסוג Image

תמונת תיבת האימייל, מלבן עם כתר של מלכה, אלמנט מסוג ImageView.

תמונת תיבת סיסמה, מלבן עם פרש, אלמנט מסוג ImageView.

תיבת טקסט, "Enter Email", אלמנט מסוג TextView.

תיבת טקסט, "Enter Password", אלמנט מסוג TextView.

הכנסת טקסט, הכנסת אימייל הלקוח, אלמנט מסוג EditText.

הכנסת טקסט, הכנסת סיסמת הלקוח, אלמנט מסוג EditText.



Regular Chess

כפתור LOG IN, כפתור שבודק אם החשבון שלך קיים בבסיס נתונים ומעביר אותך אל הלובי של המשחק, אלמנט מסוג ImageButton.

כפתור חזרה, כפתור שמחזיר אותך ללובי ההתחברות, אלמנט מסוג ImageButton





Regular Chess

מסך לובי המשחק:

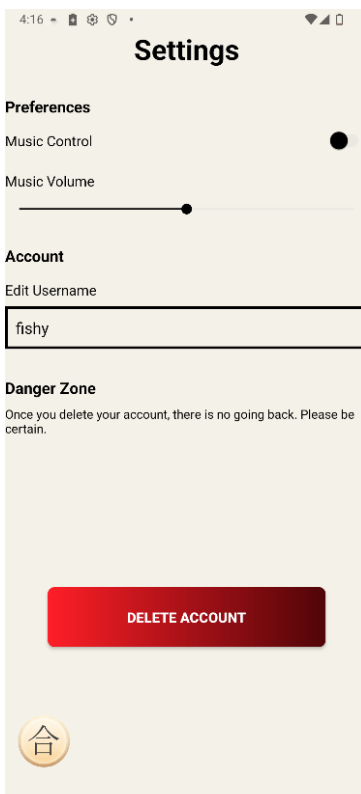
מסך לובי המשחק זה מסך שנותן את האפשרות להתנתק מהחשבון, להגיע לחלון הסטטיסטיקות של השחקן, להגיע לחלון ההוראות משחק, להתחיל משחק ולהגיע לחלון ההגדרות.



ישנם במסך זה חמישה כפתורים ותיבת טקסט אחת תמונה אחת וקלף אחד :
קלף השם, קלף שעליו יוצג שם המשתמש, אלמנט מסוג `CardView`.
תמונת השם, אייקון שמוצג לצד השם משתמש, אלמנט מסוג `ImageView`.
תיבת טקסט, שם המשתמש של הלקוח, אלמנט מסוג `TextView`.
כפתור ההגדרות, כפתור ששולח למסך ההגדרות, אלמנט מסוג `ImageButton`.
כפתור `Play`, כפתור שמתחיל משחק שחמט, אלמנט מסוג `ImageButton`.
כפתור `Instruction`, כפתור ששולח למסך ההוראות, אלמנט מסוג `ImageButton`.
כפתור `Statistics`, כפתור ששולח למסך הסטטיסטיקות, אלמנט מסוג `ImageButton`.
כפתור `Log Out`, כפתור שמנתק את החשבון ומחזיר למסך לובי ההתחברות, אלמנט מסוג `ImageButton`.

מסך ההגדרות:

מסך ההגדרות הוא מסך שנותן ללקוח את האפשרות להפעיל או לכבות את מוזיקת הרקע, ולשלוט בווליום שלה. בנוסף, הוא נותן את האפשרות לשנות את שם המשתמש, ולמחוק את החשבון לצמיתות מהטבלת אותנטיקציה.



ישנם במסך זה שני כפתורים מתג אחד הכנסת טקסט אחד וסרגל חיפוש אחד :
מתג `Music Control`, מכבה או מדליק את המוזיקת רקע, אלמנט מסוג `Switch`.
סרגל חיפוש `Music Volume`, מנמיך או מגביה את ווליום המוזיקה, אלמנט מסוג `SeekBar`.
הכנסת טקסט `Edit Username`, הכנסת שם משתמש חדש, סוג אלמנט `EditText`.
כפתור `DELETE ACCOUNT`, כפתור שמוחק את חשבון המשתמש, אלמנט מסוג `AppCompatButton`.
כפתור חזרה, כפתור שמחזיר למסך לובי המשחק, סוג אלמנט `ImageButton`.



Regular Chess

מסך הסטטיסטיקות:

מסך הסטטיסטיקות הוא מסך המציג את כל הנתונים של הלקוח שנשמרו בבסיס נתונים.

המסך מציג את מספר הניצחונות, מספר ההפסדים, מספר התיקו, מספר המשחקים, ואת זמן המשחק של הלקוח מהרגע שהוא התחבר לראשונה לחשבון שלו. אם הנתונים בה Firestore לא נטענו עדיין הוא יציג שלוש נקודות על כל אחד מהתיבות טקסט.

במסך זה ישנם תמונה אחת חמישה תיבות טקסט וכפתור אחד :

תמונה Game Statistics, קוביות של כל הנתונים, אלמנט מסוג ImageView.

תיבת טקסט wins, מראה את מספר הניצחונות, אלמנט מסוג TextView.

תיבת טקסט losses, מראה את מספר ההפסדים, אלמנט מסוג TextView.

תיבת טקסט stalemates, מראה את מספר התיקו, אלמנט מסוג TextView.

תיבת טקסט time played, מראה את כמות זמן המשחק,

אלמנט מסוג TextView.

תיבת טקסט games played, מראה את מספר המשחקים הכולל,

אלמנט מסוג TextView.

כפתור חזרה, כפתור שמחזיר ללובי של המשחק, אלמנט מסוג ImageButton.



מסך ההוראות:

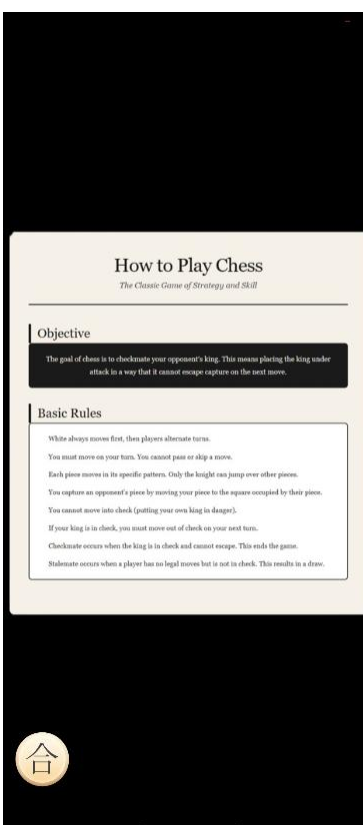
מסך ההוראות הוא מסך המציג את ההוראות המשחק ללקוח

במסך יש תמונה אחת וכפתור אחד :

תמונת הוראות, תמונה בה כתוב את כל ההוראות הבסיסיות של שחמט,

אלמנט מסוג ImageView.

כפתור חזרה, כפתור שמחזיר ללובי של המשחק, אלמנט מסוג ImageButton.



Regular Chess

מסך המשחק:

מסך המשחק הוא מסך המציג את לוח המשחק עם כל השחקים מונחים עליו ושני טיימרים לכל שחקן. כששחקן סיים מהלך הטיימר של היריב מתחיל לפעול והפוך. אפשר להזיז שחקנים בזמן אמת ואפילו לצאת מהמשחק. המשחק יכול לפתוח שני סוגי דיאלוגים. סוג אחד של בחירת חייל (מלכה, צריח, רץ ופרש) וזה נקרא קידום הפיון. ויש אפשרות לפתוח דיאלוג ניצחון/הפסד/תיקו.

במסך זה ישנם כפתור אחד ולוח אחד :

כפתור החזרה, כפתור שמפסיק את המשחק ומחזיר ללובי, אלמנט מסוג ImageButton. לוח המשחק, לוח שאפשר להזיז בו שחקנים בזמן אמת, אלמנט מסוג ChessView.

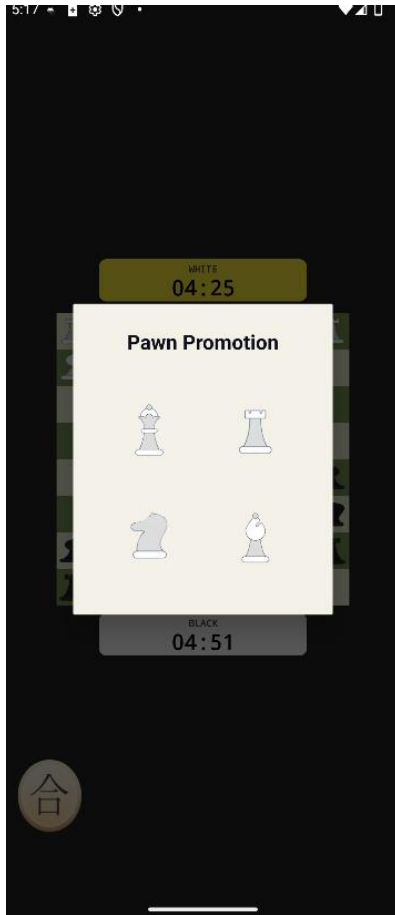




Regular Chess

מסך הקידום פיון:

מסך הקידום פיון הוא מסך המציג ארבעה אפשרויות של חלקים שאפשר לבחור ברגע שהפיון מגיע לקצה השני על הלוח. האפשרויות הן מלכה, רץ, צריח ופרש. ברגע שהוא בוחר חלק הפיון הופך לחלק הזה על הלוח.



למסך זה ישנם ארבעה כפתורים ותיבת טקסט אחת:

תיבת טקסט, Pawn Promotion, אלמנט מסוג TextView.

כפתור מלכה, כפתור שמקדם מפיון למלכה, אלמנט מסוג ImageButton.

כפתור פרש, כפתור שמקדם מפיון לפרש, אלמנט מסוג ImageButton.

כפתור רץ, כפתור שמקדם מפיון לרץ, אלמנט מסוג ImageButton.

כפתור צריח, כפתור שמקדם מפיון לצריח, אלמנט מסוג ImageButton.

מסך סיום משחק:

מסך הסיום משחק זהו מסך שמוצג רק שנגמר מסך בהפסד/ניצחון/תיקו. המסך מציג את הסיטואציה הנתונה אם השחקן הלבן ניצח הפסיד או הגיע לתיקו. המסך גם נותן את האופציה לחזור חזרה ללובי המשחק. למסך ישנם שתי תיבות טקסט תמונה אחת וכפתור אחד:

תיבת טקסט, "CHECKMATE!", אלמנט מסוג TextView.

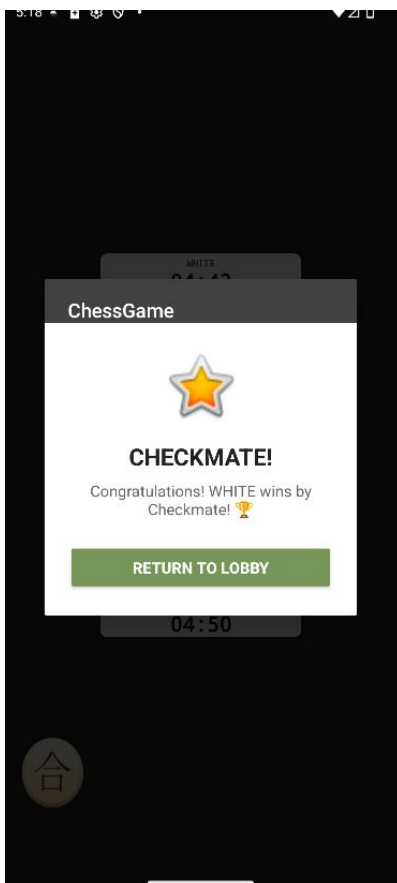
תיבת טקסט, "Congratulations! COLOR wins by Checkmate!",

אלמנט מסוג TextView.

תמונה, תמונת כוכב, אלמנט מסוג ImageView.

כפתור RETURN TO LOBBY, מחזיר את הלקוח ללובי של המשחק,

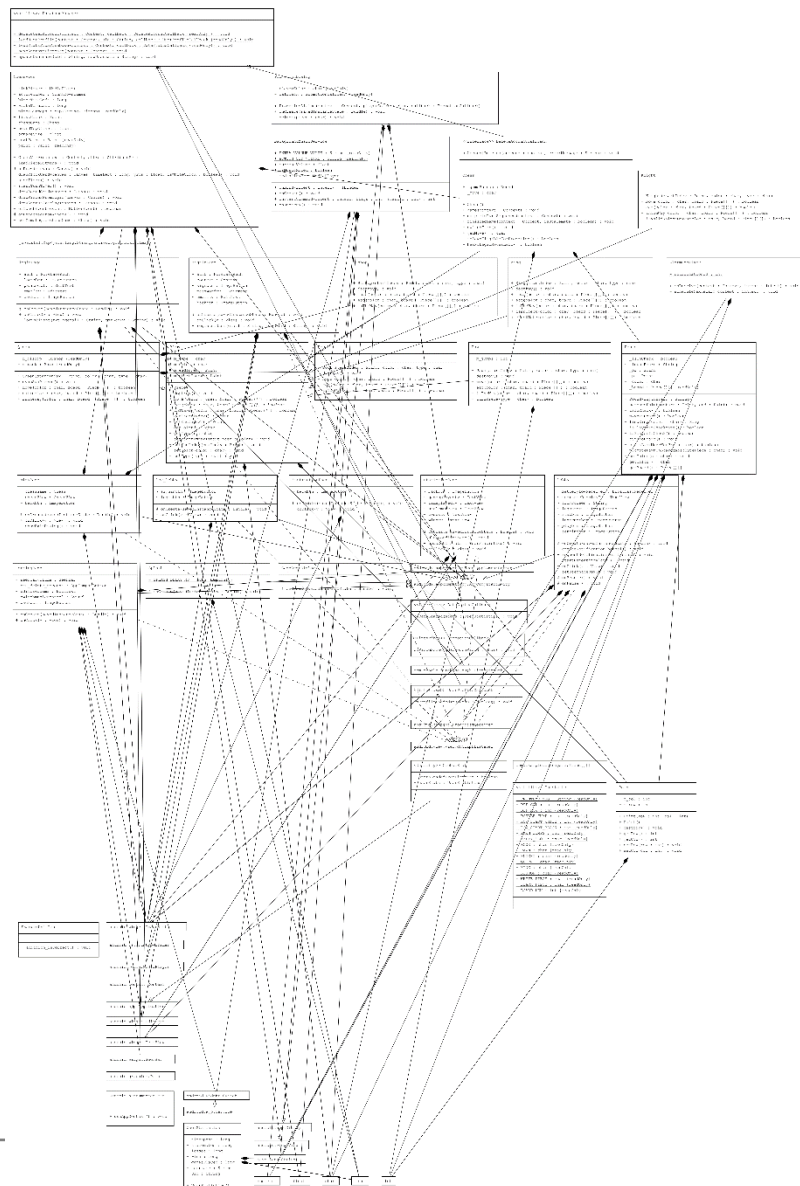
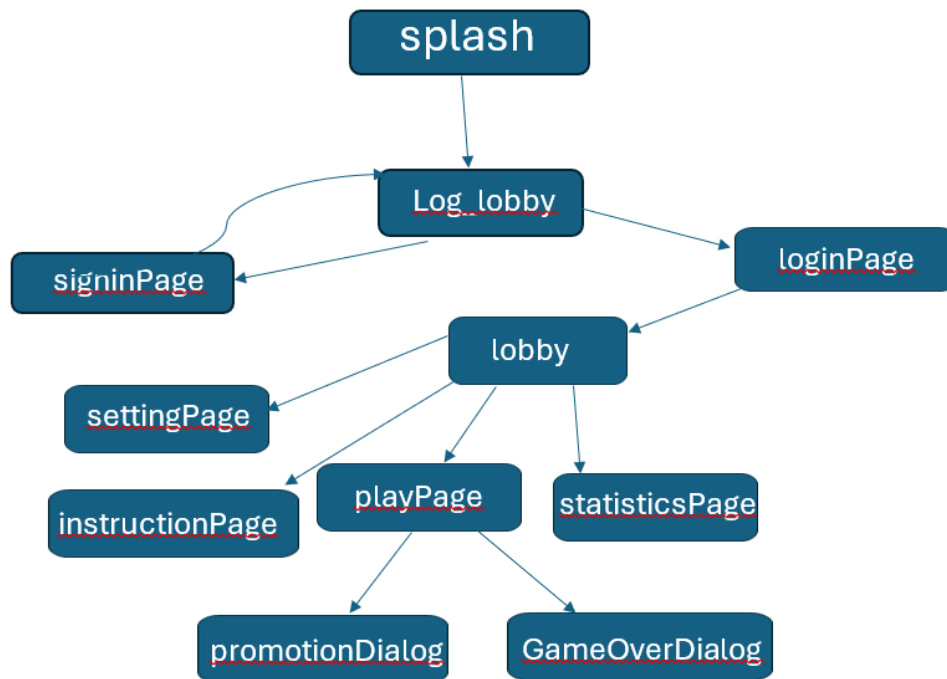
אלמנט מסוג Button.





Regular Chess

בחירת מחלקות הפרויקט והקשרים ביניהם
תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם :Screen flow diagram



תרשים UML:



Regular Chess

מימוש הפרויקט

מחלקות הפרויקט

SPLASH מחלקת

תפקיד המחלקה:

תפקיד מחלקת ספלאש היא לשמש כתחנת שהייה של 3 שניות עד מעבר לדף לובי ההתחברות היא אחראית על להציג תמונה שמשרה את אווירת השחמט ולהעביר אוטומטית את הקוץ לדף הבא.

תכונות המחלקה:

שם התכונה	הסבר
private static final long <i>SPLASH_DELAY_MS</i>	משתנה שמכיל את מספר השניות שהתהליכון ירוץ

שם הפעולה	הסבר
onCreate()	פעולה בונה שמריצה את התהליכון של הספלאש

```
private static final long SPLASH_DELAY_MS = 3000;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.splash);

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
        insets) -> {
            Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
                systemBars.bottom);
            return insets;
        });

    //run the splash intent for three sec
    new Handler(Looper.getMainLooper()).postDelayed(new Runnable() {
        @Override
        public void run() {

            Intent intent = new Intent(Splash.this, log_lobby.class);
            startActivity(intent);

            finish();
        }
    }, SPLASH_DELAY_MS);
}
```



Regular Chess

מחלקת UserStatistics

תפקיד המחלקה:

תפקיד המחלקה היא לאחסן את כל המידע שמגיע מטבלת הסטטיסטיקות מהFirestore

תכונות המחלקה:

שם התכונה	הסבר
public String Uid	מחרוזת הזהות של המשתמש שיחודית לכל משתמש ונוצרה על ידי Firestore
public String username	מחרוזת שם הלקוח שמאותחל כהתחלה של האימייל וזמין לשינוי בכל עת השימוש באפליקציה
public long gamesPlayed	מספר כל המשחקים השלמים ששיחק הלקוח לא כולל המשחקים שהוא פרש מהם.
public long wins	מספר כל הניצחונות שניצח הלקוח
public long losses	מספר כל ההפסדים שהפסיד הלקוח.
public long stalemates	מספר כל מצבי התיקו שהגיע הלקוח אליהם.
public long timespent	הזמן בדקות ששיחק הלקוח אי פעם במשחק באפליקציה.

שם הפעולה	הסבר
UserStatistics ()	פעולה בונה

```
//stats fields
public String Uid = "";
public String username = "";
public long gamesPlayed = 0;
public long wins = 0;
public long losses = 0;
public long stalemates = 0;
public long timespent = 0;
```

```
//constructor
public UserStatistics() {}
```

מחלקת DatabaseManager

תפקיד המחלקה:

מחלקה סטטית שאחראית על כל התקשורת עם Firebase Auth ו-Firebase Firestore. היא מספקת פעולות לטעינה ושמירה של סטטיסטיקות משתמש, טעינת פרופיל, עדכון שם משתמש ומחיקת חשבון



Regular Chess

תכונות המחלקה:

שם הפעולה	הסבר
deleteUserAccount(context, callback)	מוחקת את המסמך של המשתמש מ-Firebase ואת מוחקת את חשבון ה-Auth שלו
loadUserProfile(context, uid, callback)	טוענת את שם המשתמש מ-Firebase ומעדכנת אתGlobalStat
loadStatsFromFirebase(context, callback)	טוענת את כל הסטטיסטיקות של המשתמש מ-Firebase לזיכרון הגלובלי
saveStatsToFirebase(context)	שומרת את כל הסטטיסטיקות מהזיכרון הגלובלי חזרה ל-Firebase
updateUsername(uid, newUsername)	מעדכנת רק את שדה ה-username במסמך של המשתמש

```
/*
The func delete the user account from the firestore thats include the stats
table
input: context -> which class he came from
output: none ( delete the account )
*/
public static void deleteUserAccount(Context context, final
DeleteAccountCallback callback) {
    com.google.firebase.auth.FirebaseAuth user =
    com.google.firebase.auth.FirebaseAuth.getInstance().getCurrentUser();

    if (user == null) {
        if (callback != null) callback.onAccountDeleted(false, "No user is
        currently signed in.");
        return;
    }

    String uid = user.getId();

    //wipe out their row/document from the Firestore collection completely
    com.google.firebase.firestore.FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .delete()
        .addOnSuccessListener(aVoid -> {
            android.util.Log.d("DB_MANAGER", "Firestore statistics
            document deleted successfully.");
        });

    // delete the identity credentials from Auth
    user.delete()
        .addOnSuccessListener(aVoidAuth -> {
            android.util.Log.d("DB_MANAGER", "User Auth
            credentials purged successfully.");
        });

    // Clear local memory space cache
    representation
    GlobalStat globalStat = (GlobalStat)
    context.getApplicationContext();
    globalStat.userStats = new UserStatistics();
}
```



Regular Chess

```
        if (callback != null)
            callback.onAccountDeleted(true, null);
    })
    .addOnFailureListener(e -> {
        android.util.Log.e("DB_MANAGER", "Auth account
            purge failed: " + e.getMessage());
        // This happens if the user logged in a long
            time ago and needs to re-authenticate
            if (callback != null)
                callback.onAccountDeleted(false, "Authentication timeout. Please log out
                    and back in to delete your account.");
    });
    })
    .addOnFailureListener(e -> {
        android.util.Log.e("DB_MANAGER", "Failed to clear Firestore
            statistics: " + e.getMessage());
        if (callback != null) callback.onAccountDeleted(false,
            e.getMessage());
    });
}
```

```
/*
The func load the username from the stats table by using the uid of the
client.
input: uid -> the user id that the firestore create for each client
output: none ( update the username in the globalstat )
*/
public static void loadUserProfile(Context context, String uid, final
    UserProfileCallback callback) {
    if (uid == null || uid.trim().isEmpty()) {
        if (callback != null) callback.onProfileLoaded(false);
        return;
    }

    GlobalStat globalStat = (GlobalStat) context.getApplicationContext();

    FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                // Extract just the username field explicitly
                String cloudUsername =
                    documentSnapshot.getString("username");
                if (cloudUsername != null &&
                    !cloudUsername.trim().isEmpty()) {
                    globalStat.userStats.username = cloudUsername;
                }
            }
            // Signal the UI thread that processing has finished safely
            if (callback != null) callback.onProfileLoaded(true);
        })
        .addOnFailureListener(e -> {
            android.util.Log.e("DB_MANAGER", "Failed to load user
                profile: " + e.getMessage());
            // Fire fallback signal anyway so login screen flow isn't
                left hanging frozen
            if (callback != null) callback.onProfileLoaded(false);
        });
}
```



Regular Chess

```
});  
}
```

```
The func load all the stats from the stats table by using the uid of the client.  
input: uid -> the user id that the firestore create for each client  
output: none ( update the username in the globalstat )  
*/  
public static void loadStatsFromFirebase(Context context, final  
DataStatusCallback callback) {  
GlobalStat globalStat = (GlobalStat) context.getApplicationContext();  
  
if (globalStat.userStats == null || globalStat.userStats.Uid == null  
|| globalStat.userStats.Uid.trim().isEmpty()) {  
    android.util.Log.e("FIREBASE_READ", "Cannot load stats: UID is  
null.");  
    return;  
}  
  
String uid = globalStat.userStats.Uid;  
  
    FirebaseFirestore.getInstance()  
        .collection("Statistics")  
        .document(uid)  
        .get()  
        .addOnSuccessListener(snapshot -> {  
            if (snapshot.exists()) {  
                UserStatistics stats =  
                    snapshot.toObject(UserStatistics.class);  
                if (stats != null) {  
                    // Keep the UID and username that we already have  
                    // in memory  
                    stats.Uid = globalStat.userStats.Uid;  
                    stats.username = globalStat.userStats.username;  
  
                    // Write everything back into global memory  
                    globalStat.userStats = stats;  
                    globalStat.statsLoadedFromFirebase = true;  
                }  
            } else {  
                // New user - no document yet, that's fine  
                globalStat.statsLoadedFromFirebase = true;  
            }  
  
            if (callback != null)  
                callback.onDataLoaded(globalStat.userStats);  
        })  
        .addOnFailureListener(e -> {  
            android.util.Log.e("FIREBASE_READ", "Read failed: " +  
                e.getMessage());  
            // Still fire callback so UI doesn't stay stuck on "..."  
            if (callback != null)  
                callback.onDataLoaded(globalStat.userStats);  
        });  
}
```

```
/*  
The func saves the stats from the globalstat in the stats table in the  
fireStore
```



Regular Chess

```
input: uid -> the user id that the firestore create for each client
output: none ( update the stats in firestore )
*/
public static void saveStatsToFirebase(Context context) {
    GlobalStat globalStat = (GlobalStat) context.getApplicationContext();

    if (globalStat.userStats == null || globalStat.userStats.Uid == null
        || globalStat.userStats.Uid.trim().isEmpty()) {
        android.util.Log.e("FIREBASE_WRITE", "Cannot save stats: UID is
            null.");
        return;
    }

    String uid = globalStat.userStats.Uid;

    FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .set(globalStat.userStats)
        .addOnSuccessListener(aVoid ->
            android.util.Log.d("FIREBASE_WRITE", "Stats saved
                successfully for UID: " + uid))
        .addOnFailureListener(e ->
            android.util.Log.e("FIREBASE_WRITE", "Save failed: " +
                e.getMessage()));
}
```

```
/*
    The func update just the username in the stats table
input: uid -> the user id that the firestore create for each client
output: none ( update the username in firestore )
*/
public static void updateUsername(String uid, String newUsername) {
    if (uid == null || uid.trim().isEmpty()) return;

    FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .update("username", newUsername)
        .addOnSuccessListener(aVoid -> android.util.Log.d("DB_MANAGER",
            "Username synced"))
        .addOnFailureListener(e -> android.util.Log.e("DB_MANAGER",
            "Sync failed: " + e.getMessage()));
}
```

מחלקת gameOverDialog

תפקיד המחלקה:

Activity שמוצגת בסיום המשחק. היא מציגה הודעה מתאימה (נצחון, תיקו, או ניצחון על זמן) ומאפשרת לשחקן לחזור ללובי תוך שמירת הסטטיסטיקות ל-Firebase.

תכונות המחלקה:

שם התכונה	הסבר
dialogMessage (TextView)	תצוגת הטקסט שמציגה את הודעת סיום המשחק



Regular Chess

כפתור שמחזיר את השחקן ללובי	btnGoToLobby (Button)
-----------------------------	-----------------------

שם הפעולה	הסבר
onCreate(savedInstanceState)	שמאתחלת את הפרמטרים וקוראת מה Intent ומציגה את הודעת הסיום המתאימה

```

/*
The func is a constructor the find out which win or lose or stalemate
happened and show it on the activity by getIntent
*/
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.dialog_game_over);
    setFinishOnTouchOutside(false);

    TextView dialogMessage = findViewById(R.id.dialogMessage);
    Button btnGoToLobby = findViewById(R.id.btnGoToLobby);

    boolean isStalemate = getIntent().getBooleanExtra("IS_STALEMATE",
                                                    false);
    boolean whiteWon = getIntent().getBooleanExtra("WHITE_WON", true);

    // Extract the custom finish style parameter context
    String winReason = getIntent().getStringExtra("WIN_REASON");

    if (isStalemate) {
        dialogMessage.setText("Stalemate! The game ends in a Draw. ♠️");
    } else if ("TIME".equals(winReason)) {
        //Time win
        dialogMessage.setText(whiteWon
            ? "Time's up! WHITE wins on time! ⏰♠️"
            : "Time's up! BLACK wins on time! ⏰♟️");
    } else {
        // checkmate win
        dialogMessage.setText(whiteWon
            ? "Congratulations! WHITE wins by Checkmate! ♠️"
            : "Game Over! BLACK wins by Checkmate! ♟️");
    }

    btnGoToLobby.setOnClickListener(v -> {
        DatabaseManager.saveStatsToFirebase(this);

        Intent intent = new Intent(GameOverDialog.this, lobby.class);
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP |
            Intent.FLAG_ACTIVITY_NEW_TASK);
        intent.putExtra("FROM_SCREEN", "GAME");
        startActivity(intent);
        finish();
    });
}

```

מחלקת GlobalStat



Regular Chess

תפקיד המחלקה:

מחלקת Application גלובלית שמשמשת כמחסן זיכרון משותף לכל ה-Activities באפליקציה. היא שומרת את הסטטיסטיקות הנוכחיות של המשתמש ודגל שמציין האם הנתונים נטענו מ-Firebase.

תכונות המחלקה:

שם התכונה	הסבר
public UserStatistics userStats	אובייקט שמכיל את כל הסטטיסטיקות של המשתמש הנוכחי
public boolean statsLoadedFromFirebase	דגל בוליאני שמציין האם הסטטיסטיקות כבר נטענו מ-Firebase

```
public UserStatistics userStats = new UserStatistics(); //the store of the userStats
public boolean statsLoadedFromFirebase = false; //bool var to see if the stats are already loaded
```

מחלקת AlarmReceiver

תפקיד המחלקה:

מחלקת BroadcastReceiver שאחראית על קבלת אירועי alarm, יצירת התראה (Notification) למשתמש ותזמון ה-alarm הבא. כל עוד המשתמש נמצא בלובי, תישלח התראה כל 10 שניות.

תכונות המחלקה:

שם התכונה	הסבר
private static int notificationId	מונה גלובלי שמבטיח שכל התראה תקבל מזהה ייחודי ולא תדרוס התראה קודמת

שם הפעולה	הסבר
onReceive(context, intent)	בונה ומציגה התראה ומתזמנת את ההתראה הבאה
scheduleNextAlarm(context)	מתזמנת התראה שיופעל 10 שניות קדימה תוך בדיקת הרשאות

```
private static int notificationId = 0; // Increment so each notification is unique
```

```
/*
The func create the notification message and notify it ( display ) through
the screen.
input: context -> which screen the message came from
output: none ( but you get notify )
*/
```



Regular Chess

```

                                                                    @Override
public void onReceive(Context context, Intent intent) {

    NotificationCompat.Builder builder = new
    NotificationCompat.Builder(context, "alarm_channel")
        .setSmallIcon(R.drawable.kingb) //the king icon
        .setContentTitle("Chess Game")
        .setContentText("Hi! Lets Play Some Chess.")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setAutoCancel(true); // Dismisses notification when tapped

    NotificationManager manager =
        (NotificationManager)
        context.getSystemService(Context.NOTIFICATION_SERVICE);

    if (manager != null) {
        manager.notify(notificationId++, builder.build());
    }

    // Reschedule next alarm
    scheduleNextAlarm(context);
}
```

```

/*
    The func create the next alarm to start working
    if the client stays in the lobbyPage for more then 10 sec
    input: context -> which screen does the alarm happens
    output: none
*/
private void scheduleNextAlarm(Context context) {
    AlarmManager alarmManager = (AlarmManager)
    context.getSystemService(Context.ALARM_SERVICE);
    if (alarmManager == null) return;

    Intent intent = new Intent(context, AlarmReceiver.class); //creating an
    intent for the next alarm
    PendingIntent pendingIntent = PendingIntent.getBroadcast(
        context, 0, intent,
        PendingIntent.FLAG_UPDATE_CURRENT |
        PendingIntent.FLAG_IMMUTABLE
    );

    long triggerAtMillis = SystemClock.elapsedRealtime() + 10_000; // start
    alarm after 10 seconds

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        if (alarmManager.canScheduleExactAlarms()) {
            alarmManager.setExactAndAllowWhileIdle(
                AlarmManager.ELAPSED_REALTIME_WAKEUP, triggerAtMillis,
                pendingIntent);
        } else {
            alarmManager.setExactAndAllowWhileIdle(
                AlarmManager.ELAPSED_REALTIME_WAKEUP, triggerAtMillis,
                pendingIntent);
        }
    }
}
```

BackgroundMusicService מחלקת



Regular Chess

תפקיד המחלקה:

Service שרץ ברקע ומנגן מוזיקה לאורך כל זמן השימוש באפליקציה. תומך בהשתקה, שינוי עצמת השמע בזמן אמת, ועצירה מסודרת של המוזיקה

תכונות המחלקה:

שם התכונה	הסבר
private MediaPlayer mediaPlayer	מנגן האודיו שמנהל את קובץ המוזיקה ברקע
public static boolean isMusicMuted	דגל גלובלי שמציין האם המוזיקה מושתקת
public static float currentVolume	(ערכים 0.0 עד 1.0) עצמת השמע הנוכחית
public static final String ACTION_SET_VOLUME	מפתח Action שמעדכן את עוצמת השמע
public static final String EXTRA_VOLUME_VALUE	מפתח שמכיל את ערך עצמת השמע

שם הפעולה	הסבר
onBind(intent)	מקשרת בין המוזיקה לIntent
onCreate()	MediaPlayer- יוצרת ומאתחלת את עצמת השמע
onStartCommand(intent, flags, startId)	מפעילה את המוזיקה או מעדכנת את העצמה שהתקבל בהתאם ל Action
onDestroy()	עוצרת ומשחררת את ה MediaPlayer בצורה מסודרת

```
//fields
private MediaPlayer mediaPlayer;

public static boolean isMusicMuted = false;

// Holds current track volume level safely (0.0f to 1.0f)
public static float currentVolume = 0.5f;

// Action constant key for processing slider event signals
public static final String ACTION_SET_VOLUME =
    "com.example.chessgame.SET_VOLUME";
public static final String EXTRA_VOLUME_VALUE = "volume_value";
```

```
//bind the music to an intent
@Override
public IBinder onBind(Intent intent) {
    return null;
}
```

```
/*
The func set up the background music
input: none
output: none
*/
@Override
public void onCreate() {
    super.onCreate();

    mediaPlayer = MediaPlayer.create(this, R.raw.backgroundmusic); //create
```

Regular Chess

```

the basic music

        if (mediaPlayer != null) {
            mediaPlayer.setLooping(true);
// Initialize volume from our shared global state flag
            mediaPlayer.setVolume(currentVolume, currentVolume);
        }
    }
}

```

```

@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    // Intercept incoming volume updates broadcasted from UI sliders
    if (intent != null && ACTION_SET_VOLUME.equals(intent.getAction())) {
        float incomingVol = intent.getFloatExtra(EXTRA_VOLUME_VALUE, 0.5f);
        currentVolume = incomingVol; // Cache runtime memory state

        if (mediaPlayer != null && mediaPlayer.isPlaying()) {
            // Instantly update the native audio playback frame mix wrapper
            // properties
            mediaPlayer.setVolume(currentVolume, currentVolume);
        }
        return START_STICKY; // Process action payload early and terminate
                               // control stream mapping
    }

    if (mediaPlayer != null && !mediaPlayer.isPlaying() && !isMusicMuted) {
        mediaPlayer.setVolume(currentVolume, currentVolume); // Sync before
                                                                // playing
        mediaPlayer.start();
        Log.d("MUSIC_SERVICE", "Background music started.");
    }
    return START_STICKY;
}

```

```

@Override
public void onDestroy() {
    if (mediaPlayer != null) {
        mediaPlayer.stop();
        mediaPlayer.release();
        mediaPlayer = null;
        Log.d("MUSIC_SERVICE", "Background music stopped and resources
        freed.");
    }
    super.onDestroy();
}

```

מחלקת Constants

תפקיד המחלקה:

מחלקה שמרכזת את כל הקבועים של משחק השחמט – גדלי לוח, סוגי כלים, מיקומי התחלה וייצוג מחרוזות של מצב ההתחלה. מאפשרת גישה קלה וממוקמת לכל הקבועים בפרויקט.



Regular Chess

תכונות המחלקה:

שם התכונה	הסבר
public static final int BOARD_LEN	8 גודל לוח השחמט
public static final char BLACK_PIECE / WHITE_PIECE	תווים המייצגים צבע הכלי (B / W)
public static final char BISHOP/KING/QUEEN/KNIGHT/PAWN/ROOK	תווים המייצגים את סוגי כלי השחמט השונים
public static final char EMPTY_SLOT	(#) תו המייצג משבצת ריקה בלוח
public static final int PAWN_JUMPS / DOUBLE_JUMP / DIF_ROW / DIF_COL	קבועי תנועה של כלים
public static final int ROW_START_BLACK / ROW_START_WHITE	שורות ההתחלה של חיילי שחור ולבן
public static final String FRONTEND_MSG	מחרוזת המייצגת את מצב הלוח ההתחלתי

```
//consts
public static final int BOARD_LEN = 8;
public static final char BLACK_PIECE = 'B';
public static final char WHITE_PIECE = 'W';
public static final char BISHOP = 'b';
public static final char KING = 'k';
public static final char QUEEN = 'q';
public static final char KNIGHT = 'n';
public static final char PAWN = 'p';
public static final char ROOK = 'r';
public static final char EMPTY_SLOT = '#';
public static final int PAWN_JUMPS = 1;
public static final int ROW_START_BLACK = 6;
public static final int ROW_START_WHITE = 1;
public static final int DOUBLE_JUMP = 2;
public static final int DIF_ROW = 1;
public static final int DIF_COL = 1;
public static final String FRONTEND_MSG =
"rnbkqbnrpppppppp00000000000000000000000000000000PPPPPPPRNBKQBNR";
```

מחלקת instructionPage

תפקיד המחלקה:

Activity שמציגה למשתמש את הוראות המשחק. מאפשרת חזרה ללובי באמצעות כפתור חזרה.

תכונות המחלקה:

שם התכונה	הסבר
ImageButton backbtn	כפתור שמחזיר את המשתמש לדף הלובי

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את Activity ומחברת את כפתור החזרה
onClick(v)	מטפלת בלחיצה על כפתור החזרה ומנווטת ללובי



Regular Chess

ImageButton backbtn;

```
@SuppressWarnings("MissingInflatedId")
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.instructionpage);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
            systemBars.bottom);
        return insets;
    });

    //find the button
    backbtn = findViewById(R.id.backbtn);
    backbtn.setOnClickListener(this);
}
```

```
//The func make the back button work by set the intent to the lobby
@Override
public void onClick(View v) {
    if(backbtn == v)
    {
        Intent intent = new Intent(instructionPage.this, lobby.class);
        startActivity(intent);
    }
}
```

מחלקת lobby

תפקיד המחלקה:

המסך הראשי של האפליקציה לאחר ההתחברות. מפעיל מוזיקת רקע, מנהל התראות Alarm, מאזין לסוללה חלשה, ומנווט לכל מסכי האפליקציה (משחק, הגדרות, סטטיסטיקות, הוראות, התנתקות).

תכונות המחלקה:

שם התכונה	הסבר
ImageButton settingbtn / playbtn / instructbtn / statbtn / logoutbtn	כפתורי ניווט לכל מסכי האפליקציה
TextView usernameDisplayTv	מציג את שם המשתמש הנוכחי
String fromScreen	מזהה מאיזה מסך הגיעו ללובי (LOGIN / GAME)
BroadcastReceiver batteryLowReceiver	מאזין לאירוע סוללה חלשה ומציג אזהרה

שם הפעולה	הסבר
onCreate(savedInstanceState)	מפעילה מוזיקה, יוצרת ערוץ, מאתחלת את הלובי ומפעילה התראות



Regular Chess

יוצרת ערוץ Notification נדרש לאנדרואיד +8	createNotificationChannel()
מבקשת הרשאת הצגת התראות באנדרואיד +13	requestNotificationPermission()
מתזמנת Alarm מדויק שיופעל כל 10 שניות	startRepeatingAlarm()
מנווטת לדף המתאים בהתאם לכפתור שנלחץ	onClick(v)
קוראת את שם המשתמש GlobalStatm ומציגה אותו	setUsernameBox()
מרשמת את ה BroadcastReceiver לאירוע סוללה חלשה	onResume()
מסירה את ה BroadcastReceiver כדי למנוע memory leak	onPause()

```
//fields
ImageButton settingbtn;
ImageButton playbtn;
ImageButton instructbtn;
ImageButton statbtn;
ImageButton logoutbtn;
String fromScreen;
TextView usernameDisplayTv;
private BroadcastReceiver batteryLowReceiver;
```

```
/*
The func construct the page start the background music start all the alarms
*/
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.lobby);
    fromScreen = getIntent().getStringExtra("FROM_SCREEN");

    Intent musicIntent = new Intent(this, BackgroundMusicService.class);
    startService(musicIntent);

    batteryLowReceiver = new BroadcastReceiver() {
        @Override
        public void onReceive(Context context, Intent intent) {
            if (Intent.ACTION_BATTERY_LOW.equals(intent.getAction())) {

                // Display a warning alert to the player
                Toast.makeText(context,
                    "⚠ Battery is low! Plug in your charger before
                    starting a match.",
                    Toast.LENGTH_LONG).show();
            }
        }
    };

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
        insets) -> {
        Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
            systemBars.bottom);
    });
}
```

Regular Chess

```

return insets;
});
//start all the notification process
createNotificationChannel();
requestNotificationPermission();
startRepeatingAlarm();

//the update or load data logic
if (fromScreen != null) {
    switch (fromScreen) {
        case "LOGIN":
            Log.d("LOBBY", "Arrived from login");
            // DO NOTHING HERE - statisticsPage handles its own loading
            break;

        case "GAME":
            Log.d("LOBBY", "UPDATE IN FIREBASE");
            DatabaseManager.saveStatsToFirebase(this); // use this
            // instead of the old method
            break;

        default:
            break;
    }
}

//finds all the buttons amd text view
usernameDisplayTv = findViewById(R.id.usernameDisplayTv);
setUsernameBox(); // set the user name box

settingbtn = findViewById(R.id.settingbtn);
settingbtn.setOnClickListener(this);

playbtn = findViewById(R.id.playbtn);
playbtn.setOnClickListener(this);

instructbtn = findViewById(R.id.instructbtn);
instructbtn.setOnClickListener(this);

statbtn = findViewById(R.id.statbtn);
statbtn.setOnClickListener(this);

logoutbtn = findViewById(R.id.logoutbtn);
logoutbtn.setOnClickListener(this);

```

```

//The func creates a channel for the notification process
private void createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationChannel channel = new NotificationChannel(
            "alarm_channel", // Channel ID
            "Alarm Notifications", // Channel name (shown in
            settings)
            NotificationManager.IMPORTANCE_HIGH
        );
        channel.setDescription("Notifications fired by the alarm");

        NotificationManager manager =
            getSystemService(NotificationManager.class);
        if (manager != null) {

```




Regular Chess

```
manager.createNotificationChannel(channel);  
}  
}  
}
```

```
//This func request a notification permission from the phone  
private void requestNotificationPermission() {  
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) { // Android  
13+  
if (ActivityCompat.checkSelfPermission(this,  
android.Manifest.permission.POST_NOTIFICATIONS)  
!= android.content.pm.PackageManager.PERMISSION_GRANTED) {  
ActivityCompat.requestPermissions(  
this,  
new  
String[]{android.Manifest.permission.POST_NOTIFICATIONS},  
1001  
)  
}  
}  
}
```

```
//The func check if permission is granted and if so  
// he start the all alarm process of alert the client to play some games  
private void startRepeatingAlarm() {  
AlarmManager alarmManager = (AlarmManager)  
getSystemService(Context.ALARM_SERVICE);  
if (alarmManager == null) return;  
  
Intent intent = new Intent(this, AlarmReceiver.class);  
PendingIntent pendingIntent = PendingIntent.getBroadcast(  
this,  
0,  
intent,  
PendingIntent.FLAG_UPDATE_CURRENT |  
PendingIntent.FLAG_IMMUTABLE  
);  
  
// check permission before scheduling exact alarm  
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {  
if (!alarmManager.canScheduleExactAlarms()) {  
// Open settings so user can grant permission  
Intent permIntent = new  
Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM);  
startActivity(permIntent);  
return;  
}  
}  
  
// setExactAndAllowWhileIdle + reschedule in receiver  
long triggerAtMillis = SystemClock.elapsedRealtime() + 10_000;  
alarmManager.setExactAndAllowWhileIdle(  
AlarmManager.ELAPSED_REALTIME_WAKEUP,  
triggerAtMillis,  
pendingIntent  
);  
}
```



Regular Chess

```
//move to every screen by the button
@Override
public void onClick(View v) {

    if(settingbtn == v)
    {
        Intent intent = new Intent(lobby.this,settingPage.class);
        startActivity(intent);
    }

    if(playbtn == v)
    {
        Intent intent = new Intent(lobby.this,playPage.class);
        startActivity(intent);
    }

    if(instructbtn == v)
    {
        Intent intent = new Intent(lobby.this,instructionPage.class);
        startActivity(intent);
    }

    if(statbtn == v)
    {
        Intent intent = new Intent(lobby.this,statisticsPage.class);
        startActivity(intent);
    }

    //log out process starting
    //stop the music than wipe the cache from the global stat and in last
    //move to the log lobby
    if(logoutbtn == v) {
        Intent musicIntent = new Intent(lobby.this,
            BackgroundMusicService.class);
        stopService(musicIntent);

        BackgroundMusicService.isMusicMuted = false;

        GlobalStat globalStat = (GlobalStat) getApplicationContext();
        if (globalStat.userStats != null) {
            globalStat.userStats = new UserStatistics(); // Wipes current
            cache
        }

        Intent intent = new Intent(lobby.this, log_lobby.class);
        startActivity(intent);
        finish();
    }
}
```

```
//set the username on the textView
private void setUsernameBox()
{
    GlobalStat globalStat = (GlobalStat) getApplicationContext();
    usernameDisplayTv.setText(globalStat.userStats.username);
}
```

Regular Chess

```

@Override
protected void onResume() {
    super.onResume();
    // listening for the low-battery broadcast when the user enters the lobby
    IntentFilter filter = new IntentFilter(Intent.ACTION_BATTERY_LOW);
    registerReceiver(batteryLowReceiver, filter);
}

@Override
protected void onPause() {
    super.onPause();
    // Stop listening when the user navigates away from the lobby
    // prevent memory leaks!
    if (batteryLowReceiver != null) {
        unregisterReceiver(batteryLowReceiver);
    }
}

```

מחלקת log_lobby

תפקיד המחלקה:

מסך הכניסה הראשוני של האפליקציה שמאפשר למשתמש לבחור בין התחברות לחשבון קיים (Login) לבין יצירת חשבון חדש (Sign In).

תכונות המחלקה:

שם התכונה	הסבר
ImageButton loginbtn	כפתור שמנווט למסך הLogin
ImageButton signinbtn	כפתור שמנווט למסך הSign In

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את ה Activity ומחברת את הכפתורים
onClick(v)	מנווטת ל loginPage או ל signinPage בהתאם ללחיצה

```

//fields
ImageButton loginbtn;
ImageButton signinbtn;

```

```

//constructor
@SuppressLint("MissingInflatedId")
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.log_lobby);
}

```



Regular Chess

```
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
    Insets systemBars =
        insets.getInsets(WindowInsetsCompat.Type.systemBars());
    v.setPadding(systemBars.left, systemBars.top, systemBars.right,
        systemBars.bottom);
    return insets;
});

//find buttons
loginbtn = findViewById(R.id.loginbtn);
loginbtn.setOnClickListener(this);

signinbtn = findViewById(R.id.signbtn);
signinbtn.setOnClickListener(this);
```

```
//move to screen by the press of a button
@Override
public void onClick(View v) {

    if(loginbtn == v)
    {
        Intent intent = new Intent(log_lobby.this,loginPage.class);
        startActivity(intent);
    }

    if(signinbtn == v)
    {
        Intent intent = new Intent(log_lobby.this,signinPage.class);
        startActivity(intent);
    }
}
```

מחלקת loginPage

תפקיד המחלקה:

מסך ההתחברות לאפליקציה. מאפשר למשתמש להזין אימייל וסיסמה, מבצע אימות מול Firebase Auth, טוען את פרטי הפרופיל מ-Firebase ומנווט ללובי בהצלחה.

תכונות המחלקה:

שם התכונה	הסבר
ImageButton backbtn	כפתור חזרה לlog_lobby
EditText emailtv / passwordtv	שדות קלט לאימייל ולסיסמה
ImageButton loginbtn	כפתור ביצוע ההתחברות
FirebaseAuth auth	אובייקט Firebase לאימות משתמשים

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את הActivity ומחברת את הWidgets
onClick(v)	מטפלת בלחיצות חזרה אחורה או ביצוע Login



Regular Chess

מבצעת התחברות ל-Firebase Auth
טוענת את הפרופיל ומנווטת ללובי

loginClient(email, password)

```
//fields
ImageButton backbtn;
EditText emailtv, passwordtv;
ImageButton loginbtn;

FirebaseAuth auth;
```

```
//constructor
@SuppressLint("MissingInflatedId")
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.login);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    //find buttons and textviews
    backbtn = findViewById(R.id.backbtn);
    backbtn.setOnClickListener(this);

    emailtv = findViewById(R.id.emailtv);
    passwordtv = findViewById(R.id.passwordtv);
    loginbtn = findViewById(R.id.loginbtn);

    auth = FirebaseAuth.getInstance();

    loginbtn.setOnClickListener(this);
```

```
/*
The func see which button pressed and move to the right activity
*/
@Override
public void onClick(View v) {

    if(backbtn == v)
    {
        //letting the lobby know where does the client come from
        Intent intent = new Intent(loginPage.this, log_lobby.class);
        intent.putExtra("FROM_SCREEN", "LOGIN");
        startActivity(intent);
        finish();
    }

    if(loginbtn == v)
    {
        //try to login
        String txt_logemail = emailtv.getText().toString();
        String txt_logPw = passwordtv.getText().toString();
        loginClient(txt_logemail, txt_logPw);
    }
}
```

Regular Chess

```

    }
}

/*
The func check if the email and password are already register in the auth
table
if yes he will move to the lobby activity and let the lobby know the client
came from
the loginPage
input: txtLogemail -> the email the client gave
txtLogPw -> the password the client gave
output: none
*/
private void loginClient(String txtLogemail, String txtLogPw) {
    auth.signInWithEmailAndPassword(txtLogemail,
txtLogPw).addOnSuccessListener(new OnSuccessListener<AuthResult>() {
        @Override
        public void onSuccess(AuthResult authResult) {
            String uid = auth.getUid();
            if (uid == null) return;

GlobalStat globalStat = (GlobalStat) getApplicationContext();

            if (globalStat.userStats == null) {
                globalStat.userStats = new UserStatistics();
            }

            // Bind the essential local variables first
            globalStat.userStats.Uid = uid;

            // Set a fallback username from the email prefix in case
            // Firestore is empty
            globalStat.userStats.username = txtLogemail.substring(0,
txtLogemail.indexOf('@'));

            DatabaseManager.loadUserProfile(loginPage.this, uid, new
DatabaseManager.UserProfileCallback() {
                @Override
                public void onProfileLoaded(boolean success) {
                    // This block executes dynamically after your network
                    // request drops
                    Toast.makeText(loginPage.this, "Login successful",
Toast.LENGTH_SHORT).show();

                    Intent intent = new Intent(loginPage.this,
lobby.class);
                    intent.putExtra("FROM_SCREEN", "LOGIN");
                    startActivity(intent);
                    finish();
                }
            });
        }
    });
}

```



Regular Chess

מחלקת playPage

תפקיד המחלקה:

מסך המשחק הראשי. מאתחל את לוגיקת השחמט ואת תצוגת הלוח, מטפל באירועי פג-זמן, ומאפשר יציאה מהמשחק עם דיאלוג אישור.

תכונות המחלקה:

שם התכונה	הסבר
ImageButton backbtn	כפתור יציאה מהמשחק
ChessView chessView	תצוגת לוח השחמט הגרפית
Chess chessGame	אובייקט לוגיקת המשחק

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את לוגיקת המשחק, הלוח, ומגדירה מאזין לאירוע פג-זמן
onClick(v)	פותחת דיאלוג יציאה – לחיצה על כפתור חזרה
showExitDialog()	עוצרת את השעון ומציגה דיאלוג אישור יציאה מהמשחק

```
//fields
private ImageButton backbtn;
private ChessView chessView;
private Chess chessGame;
```

```
/*
The func construct and start the game logic
*/
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.playpage);

    // Initialize Chess Logic first
    chessGame = new Chess();

    //updating the games amount
    GlobalStat globalStat = (GlobalStat)getApplicationContext();
    globalStat.userStats.gamesPlayed++;

    // Initialize Views
    chessView = findViewById(R.id.chessBoard);
    backbtn = findViewById(R.id.backbtn);

    //Safety Check
    if (chessView == null) {
        android.util.Log.e("playPage", "chessView not found! Check your XML ID.");
    }

    // Link logic to view
    if (chessView != null) {
        chessView.setGameLogic(chessGame);
    }
}
```



Regular Chess

```
chessView.setOnTimeoutListener(new ChessView.OnTimeoutListener() {
    //if the game stoped on time this handle it to the
    //GameOverDialog
    @Override
    public void onTimeout(boolean isWhiteTurn) {
        // If White ran out of time, Black wins (whiteWon = false)
        // If Black ran out of time, White wins (whiteWon = true)
        boolean whiteWon = !isWhiteTurn;

        // Route straight to the GameOverDialog package lane
        Intent intent = new Intent(playPage.this,
            GameOverDialog.class);
        intent.putExtra("IS_STALEMATE", false);
        intent.putExtra("WHITE_WON", whiteWon);

        // Pass the dynamic layout polish key we set up
        intent.putExtra("WIN_REASON", "TIME");

        // Wipe backstack
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP |
            Intent.FLAG_ACTIVITY_NEW_TASK);
        startActivity(intent);
        finish();
    }
});

if (backbtn != null) {
    backbtn.setOnClickListener(this);
}
```

```
//handles the exit button
@Override
public void onClick(View v) {
    if (v.getId() == R.id.backbtn) {
        showExitDialog();
    }
}
```

```
/*
The func open a dialog for the player and handle if the player wants to
stop the game
and it handle if the player wants to continue the game
input: none
output: none (continue timer/move to lobby)
*/
private void showExitDialog() {
    // Stop the clock
    if (chessView != null) {
        chessView.stopTimer();
    }

    new AlertDialog.Builder(this)
        .setTitle("Exit Game")
        .setMessage("Are you sure you want to quit?")
        .setCancelable(false)
```




Regular Chess

```
.setPositiveButton("Yes, Exit", (dialog, which) -> {
    Intent intent = new Intent(playPage.this, lobby.class);
    intent.putExtra("FROM_SCREEN", "GAME");
    finish();
})
.setNegativeButton("No, Resume", (dialog, which) -> {
    // Resume the timer
    if (chessView != null) {
        chessView.startTurnTimer();
    }
})
.show();
}
```

מחלקת settingPage

תפקיד המחלקה:

מסך ההגדרות של האפליקציה. מאפשר למשתמש לשנות שם משתמש, להשתיק מוזיקה, לכוון עצמת שמע באמצעות Slider ולמחוק את חשבון לצמיתות.

תכונות המחלקה:

שם התכונה	הסבר
ImageButton backbtn	כפתור חזרה ללובי תוך שמירת שם המשתמש
Switch switchmusiccontrol	מתג להפעלה / השתקה של המוזיקה
EditText editusername	שדה עריכה לשינוי שם המשתמש
AppCompatActivity btn_delete_account	כפתור למחיקת החשבון לצמיתות
SeekBar seekbarvolume	פס גלילה לכיוון עצמת המוזיקה

שם הפעולה	הסבר
onCreate(savedInstanceState)	מסנכרנת מצב, מאתחלת את כל הרכיבים מוזיקה ועצמת שמע מהזיכרון הגלובלי
onClick(v)	מטפלת בחזרה ללובי (שמירת שם משתמש) ובמחיקת חשבון

```
//fields
ImageButton backbtn;
Switch switchmusiccontrol;
EditText editusername;
AppCompatActivity btn_delete_account;
SeekBar seekbarvolume;
```



Regular Chess

```
//this func construct and check if the seekbar is changing
// if it does he will increase or decrease the volume of the background
music

@SuppressLint({"MissingInflatedId", "ClickableViewAccessibility"})
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.settingpage);

    seekbarvolume = findViewById(R.id.seekbarvolume);

    int initialProgress = (int) (BackgroundMusicService.currentVolume *
                                100);
    seekbarvolume.setProgress(initialProgress);

    seekbarvolume.setOnSeekBarChangeListener(new
        SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar seekBar, int progress,
                boolean fromUser) {
                // Map the integer progression scale back down to fractional
                // volume ranges
                float scalarVolume = progress / 100f;

                // Ship the updated calculation directly down to your
                // background Service pipeline
                Intent volumeIntent = new Intent(settingPage.this,
                    BackgroundMusicService.class);

                volumeIntent.setAction(BackgroundMusicService.ACTION_SET_VOLUME);

                volumeIntent.putExtra(BackgroundMusicService.EXTRA_VOLUME_VALUE,
                    scalarVolume);
                startService(volumeIntent);
            }

            @Override
            public void onStartTrackingTouch(SeekBar seekBar) {}

            @Override
            public void onStopTrackingTouch(SeekBar seekBar) {}
        });

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
        insets) -> {
        Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
            systemBars.bottom);
        return insets;
    });

    // Initialize widgets
    switchmusiccontrol = findViewById(R.id.switchmusiccontrol);
    editusername = findViewById(R.id.editusername);

    // Initialize Button and Bind Listeners
    btn_delete_account = findViewById(R.id.btn_delete_account);
    btn_delete_account.setOnClickListener(this);
}
```



Regular Chess

```
// Click Compression Effect (scale: 0.92)
btn_delete_account.setOnTouchListener(new View.OnTouchListener() {
    @Override
    public boolean onTouch(View v, MotionEvent event) {
        if (event.getAction() == MotionEvent.ACTION_DOWN) {
            v.animate().scaleX(0.92f).scaleY(0.92f).setDuration(150).start();
        } else if (event.getAction() == MotionEvent.ACTION_UP ||
            event.getAction() == MotionEvent.ACTION_CANCEL) {
            v.animate().scaleX(1.0f).scaleY(1.0f).setDuration(600).start();
            if (event.getAction() == MotionEvent.ACTION_UP) {
                v.performClick();
            }
        }
        return false;
    }
});

backbtn = findViewById(R.id.backbtn);
backbtn.setOnClickListener(this);

// Populate local cache inputs
GlobalStat globalStat = (GlobalStat) getApplicationContext();
if (globalStat.userStats != null && globalStat.userStats.username !=
    null) {
    editusername.setText(globalStat.userStats.username);
}

// Music toggle engine system state sync
switchmusiccontrol.setChecked(!BackgroundMusicService.isMusicMuted);
switchmusiccontrol.setOnCheckedChangeListener(new
    CompoundButton.OnCheckedChangeListener() {
        @Override
        public void onCheckedChanged(CompoundButton buttonView, boolean
            isChecked) {
            Intent musicIntent = new Intent(settingPage.this,
                BackgroundMusicService.class);
            if (isChecked) {
                BackgroundMusicService.isMusicMuted = false;
                startService(musicIntent);
            } else {
                BackgroundMusicService.isMusicMuted = true;
                stopService(musicIntent);
            }
        }
    });
}
```

```
//this func move and act by the logic of the button that got pressed
@Override
public void onClick(View v) {
    if (backbtn == v) {
        String newUsername = editusername.getText().toString().trim();
        if (!newUsername.isEmpty()) {
            GlobalStat globalStat = (GlobalStat) getApplicationContext();
            globalStat.userStats.username = newUsername;
            DatabaseManager.updateUsername(globalStat.userStats.Uid,
                newUsername);
        }
    }
}
```

Regular Chess

```

    } else {
        Toast.makeText(this, "Username cannot be empty!",
            Toast.LENGTH_SHORT).show();
        return;
    }

    Intent intent = new Intent(settingPage.this, lobby.class);
    startActivity(intent);
    finish();
}

        if (btn_delete_account == v) {
// Stop background audio before destroying session states
        Intent musicIntent = new Intent(settingPage.this,
            BackgroundMusicService.class);
        stopService(musicIntent);
        BackgroundMusicService.isMusicMuted = false;

// deletes data from Firestore & Auth table inside the
        DatabaseManager
        DatabaseManager.deleteUserAccount(this, new
            DatabaseManager.DeleteAccountCallback() {
                @Override
            public void onAccountDeleted(boolean success, String
                errorMessage) {
                    if (success) {
                        Toast.makeText(settingPage.this, "Account deleted
                            permanently.", Toast.LENGTH_LONG).show();

// Route to log_lobby initialization lane
                        Intent intent = new Intent(settingPage.this,
                            log_lobby.class);
// Wipe backstack completely so hardware arrows cannot
                            re-enter back into active states
                        intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
                            Intent.FLAG_ACTIVITY_CLEAR_TASK);
                        startActivity(intent);
                        finish();
                    } else {
                        Toast.makeText(settingPage.this, "Error: " +
                            errorMessage, Toast.LENGTH_LONG).show();
                    }
                }
            });
        }
    }
}

```

מחלקת signInPage

תפקיד המחלקה:

מסך הרישום של משתמשים חדשים. מאפשר הזנת אימייל וסיסמה, מבצע ולידציה בסיסית, ויוצר חשבון חדש ב-Firebase Authentication.



Regular Chess

תכונות המחלקה:

שם התכונה	הסבר
ImageButton backbtn	כפתור חזרה לlog_lobby
EditText emailtv / passwordtv	שדות קלט לאימייל ולסיסמה
ImageButton signbtn	כפתור ביצוע הרישום
FirebaseAuth auth	Firebase אוברייקט לניהול הרישום
Context context	הקשר Activity
	לשימוש בהודעות Toast

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את Activity ומחברת את הכפתורים
onClick(v)	מטפלת בחזרה אחורה ובלחיצת רישום תוך ולידציה של הקלט
registerUser(email, password)	יוצרת חשבון חדש ב-Firebase Auth ומנווטת לlog_lobby בהצלחה

```
//fields
ImageButton backbtn;

//firebase variables
EditText emailtv, passwordtv;
ImageButton signbtn;
Context context;

FirebaseAuth auth;
```

```
//constructor
@SuppressLint("MissingInflatedId")
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.signin);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.signin), (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    //bind buttons and textviews
    backbtn = findViewById(R.id.backbtn);
    backbtn.setOnClickListener(this);

    emailtv = findViewById(R.id.emailtv);
    passwordtv = findViewById(R.id.passwordtv);
    signbtn = findViewById(R.id.signbtn);

    context = signinPage.this;
    auth = FirebaseAuth.getInstance();
```



Regular Chess

```
signbtn.setOnClickListener(this);
```

```
//The func move to the desire activity and act by the button that got
pressed
@Override
public void onClick(View v) {
    if(backbtn == v)
    {
        Intent intent = new Intent(signinPage.this, log_lobby.class);
        startActivity(intent);
        finish();
    }
    //make sure the strings are not empty
    if(signbtn == v)
    {
        String txt_email = emailtv.getText().toString();
        String txt_password = passwordtv.getText().toString();
        if(TextUtils.isEmpty(txt_email) || TextUtils.isEmpty(txt_password))
        {
            Toast.makeText(context, "fields must not be
empty", Toast.LENGTH_SHORT).show();
        }
        else if(txt_password.length() < 6){
            Toast.makeText(context, "password is to
short", Toast.LENGTH_SHORT).show();
        }
        else {
            //try to register
            registerUser(txt_email, txt_password);
        }
    }
}
```

```
/*
The func try to register in the auth table
input: email -> client desire email
password -> client desire password
output: or a new row in the auth table or toast failed
*/
private void registerUser(String email, String password)
{
    auth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener(signinPage.this, new
            OnCompleteListener<AuthResult>() {
                @Override
                public void onComplete(@NonNull Task<AuthResult> task) {
                    if(task.isSuccessful())
                    {
                        Toast.makeText(context, "success",
                            Toast.LENGTH_SHORT).show();
                        startActivity(new Intent(signinPage.this,
                            log_lobby.class));
                    }
                    else {
                        Toast.makeText(context, "failed",
```

Regular Chess

```
Toast.LENGTH_SHORT).show();
}
}
});
}
```

מחלקת statisticsPage

תפקיד המחלקה:

מסך הסטטיסטיקות שמציג למשתמש את נתוני הביצועים שלו: ניצחונות, הפסדים, תיקוים, משחקים שהתנגנו וזמן משחק. טוען נתונים מ-Firebase ומציג אותם בזמן אמת.

תכונות המחלקה:

שם התכונה	הסבר
TextView winstv / losestv / stalematestv / timeplayedtv / gamesplayedtv	תצוגות טקסט לכל אחד מנתוני הסטטיסטיקה
ImageButton backbtn	כפתור חזרה ללובי

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את Activity, cache, אם קיימים, וטוענת נתונים עדכניים מ-Firebase
showLoadingState()	מציגה '...' בכל שדה בזמן טעינת הנתונים
updateUI(stats)	מעדכנת את כל תצוגות הטקסט עם הנתונים שהתקבלו
onClick(v)	כפתור חזרה – מנווטת ללובי

```
//fields
TextView winstv, losestv, stalematestv, timeplayedtv, gamesplayedtv;
ImageButton backbtn;
```

```
//constructor
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.statistics);

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
            systemBars.bottom);
        return insets;
    });

    //bind all the text views and button

    winstv = findViewById(R.id.winstv);
    losestv = findViewById(R.id.losestv);
```



Regular Chess

```
stalematestv = findViewById(R.id.stalematestv);
timeplayedtv = findViewById(R.id.timeplayedtv);
gamesplayedtv = findViewById(R.id.gamesplayedtv);
backbtn      = findViewById(R.id.backbtn);
backbtn.setOnClickListener(this);

GlobalStat globalStat = (GlobalStat) getApplicationContext();

// If we already loaded from Firebase before
// show the cached values immediately – no "..." flicker
if (globalStat.statsLoadedFromFirebase && globalStat.userStats != null)
{
    updateUI(globalStat.userStats);
} else {
    showLoadingState();
}

// Always fetch fresh data from Firebase to stay up to date
DatabaseManager.loadStatsFromFirebase(this, new
    DatabaseManager.DataStatusCallback() {
        @Override
        public void onDataLoaded(UserStatistics stats) {
            updateUI(stats);
        }
    });
}
```

```
/*
The func shows lodaing stats if it hasent got anything from the firebase
input: none
output: none
*/
private void showLoadingState() {
    winstv.setText("...");
    losestv.setText("...");
    stalematestv.setText("...");
    gamesplayedtv.setText("...");
    timeplayedtv.setText("...");
}
```

```
/*
The func update all the text views by the stats from the globalstat
input: stats -> all of the user stats
output: none
*/
@SuppressWarnings("SetTextI18n")
private void updateUI(UserStatistics stats) {
    if (stats == null) return;
    runOnUiThread() -> {
```


Regular Chess

```
winstv.setText(String.valueOf(stats.wins));
losestv.setText(String.valueOf(stats.losses));
stalematestv.setText(String.valueOf(stats.stalemates));
gamesplayedtv.setText(String.valueOf(stats.gamesPlayed));
timeplayedtv.setText(stats.timespent + " min");
});
}
```

```
//the func goes back to the lobby page if the backbtn was pressed
@Override
public void onClick(View v) {
    if (backbtn == v) {
        Intent intent = new Intent(statisticsPage.this, lobby.class);
        startActivity(intent);
        finish();
    }
}
```

מחלקת Piece

תפקיד המחלקה:

מחלקה אבסטרקטית שמהווה את הבסיס לכל כלי השחמט. מגדירה את השדות המשותפים (מיקום, צבע, סוג) ואת המתודות האבסטרקטיות שכל כלי חייב לממש.

תכונות המחלקה:

שם התכונה	הסבר
protected Point m_startIndex	המיקום שממנו הכלי מתחיל את תנועתו
protected Point m_endIndex	המיקום אליו הכלי מנסה להגיע
protected char m_color	(שחור B / לבן W) צבע הכלי
protected char m_type	(צריח r, מלכה q, מלך k ועוד) סוג הכלי

שם הפעולה	הסבר
Piece()	קונסטרקטור – מאתחל נקודות ריקות וסימני EMPTY_SLOT
destroy()	משחרר את הנקודות ומאפס את שדות הצבע והסוג
move(color, board) [abstract]	בודקת אם תנועה (ללא אכילה) חוקית לפי חוקי הכלי
eat(color, board) [abstract]	בודקת אם אכילת כלי יריב חוקית לפי חוקי הכלי
inTheWay(color, board) [abstract]	בודקת אם הנהיג בין startIndex ל endIndex חסום
getStartIndex() / getEndIndex()	מחזירות את נקודות ההתחלה והסיום
getColor() / getType()	מחזירות את צבע הכלי וסוגו
setStartIndex() / setEndIndex() / setColor() / setType()	מגדירות את שדות הכלי

```
//fields
protected Point m_startIndex;
protected Point m_endIndex;
protected char m_color;
protected char m_type;
```



Regular Chess

```
//constructor reset fields
    Piece() {
this.m_startIndex = new Point();
    this.m_endIndex = new Point();
    setColor(Constants.EMPTY_SLOT);
    setType(Constants.EMPTY_SLOT);
    }
```

```
//destructor
    public void destroy()
    {
this.m_startIndex.destroy();
    this.m_endIndex.destroy();
    this.m_startIndex = null;
    this.m_endIndex = null;
    this.m_color = '\0';
    this.m_type = '\0';
    }
```

```
//abstract function for every piece
    public abstract boolean move(char color, Piece[][] board);
    public abstract boolean eat(char color, Piece[][] board);
    public abstract boolean inTheWay(char color, Piece[][] board);
```

```
//get functions
    public Point getStartIndex()
    {
        return this.m_startIndex;
    }
    public Point getEndIndex()
    {
        return this.m_endIndex;
    }
    public char getColor()
    {
        return this.m_color;
    }
    public char getType()
    {
        return this.m_type;
    }
```

```
//set functions
    public void setStartIndex(Point startIndex)
    {
        this.m_startIndex = startIndex;
    }
    public void setEndIndex(Point endIndex)
    {
        this.m_endIndex = endIndex;
    }
```



Regular Chess

```
public void setColor(char color)
{
    this.m_color = color;
}
public void setType(char type)
{
    this.m_type = type;
}
```

מחלקת Point

תפקיד המחלקה:

מחלקה המייצגת נקודה (תא) על לוח השחמט באמצעות שורה ועמודה. משמשת לאחסון מיקום כלים ותנועות.

תכונות המחלקה:

שם התכונה	הסבר
private int m_row	מספר השורה של הנקודה בלוח (0-7)
private int m_col	מספר העמודה של הנקודה בלוח (0-7)

שם הפעולה	הסבר
Point(int row, int col)	קונסטרקטור עם ערכי שורה ועמודה
Point()	קונסטרקטור ברירת מחדל – מאתחל ל-(0,0)
destroy()	מאפס את שורה ועמודה לאפס
getRow() / getCol()	מחזירות את ערכי השורה והעמודה
setRow(int) / setCol(int)	מגדירות ערכי שורה ועמודה חדשים

```
//fields
private int m_row;
private int m_col;
```

```
//constructor with row and col
public Point(int row, int col)
{
    this.m_row = row;
    this.m_col = col;
}
```

```
//constructor without row and col
public Point()
{
    this.m_row = 0;
    this.m_col = 0;
}
```

```
//destructor
public void destroy()
{
}
```



Regular Chess

```
this.m_row = 0;
```

```
//The func return the row of the point
public int getRow()
{
    return this.m_row;
}

//The func return the col of the point
public int getCol()
{
    return this.m_col;
}
```

```
//the func get a new row and replace with the old row
public void setRow(int row)
{
    this.m_row = row;
}

//the func get a new col and replace with the old col
public void setCol(int col)
{
    this.m_col = col;
}
```

Bishop מחלקת

תפקיד המחלקה:

מממשת את כלי הרץ בשחמט. הרץ נע בלבד באלכסון ובודק שאין כלים חוסמים בנתיב.

תכונות המחלקה:

שם הפעולה	הסבר
Bishop(startIndex, color, type)	קונסטרוקטור – מאתחל את שדות מחלקת האב
destroy()	מפעיל את destroy של מחלקת האב
move(color, board)	מאמת תנועה אלכסונית



Regular Chess

מאמת אכילה אלכסונית	eat(color, board)
בודק שהמעבר אלכסוני חופשי ושהתא הסופי פנוי או שייך ליריב	inTheWay(color, board)

```
//constructor
public Bishop(Point startIndex, char color, char type) {
    super.setStartIndex(startIndex);
    super.setEndIndex(startIndex);
    super.setColor(color);
    super.setType(type);
}
```

```
//destructor
@Override
public void destroy() {
    super.destroy();
}
```

```
//for bishop it just in the way
@Override
public boolean move(char color, Piece[][] board) {
    return inTheWay(color, board);
}

//for bishop it just in the way
@Override
public boolean eat(char color, Piece[][] board) {
    return inTheWay(color, board);
}
```

```
/*
The func cheks if the move is valid and if any piece in the way of the
bishop
input: color of the piece, that board it self
output: true if move is valid else false
*/
@Override
public boolean inTheWay(char color, Piece[][] board) {
    // 1. Strictly enforce diagonal geometry validation
    if (Math.abs(getEndIndex().getRow() - getStartIndex().getRow()) ==
        Math.abs(getEndIndex().getCol() - getStartIndex().getCol())) {

        int rowStep = Integer.compare(getEndIndex().getRow(),
            getStartIndex().getRow());
        int colStep = Integer.compare(getEndIndex().getCol(),
            getStartIndex().getCol());

        int row = getStartIndex().getRow() + rowStep;
        int col = getStartIndex().getCol() + colStep;

        // Check intermediate squares
        while (row != getEndIndex().getRow() || col !=
            getEndIndex().getCol()) {
            if (board[row][col] != null) {
                return false; // Blocked
            }
            row += rowStep;
        }
    }
}
```



Regular Chess

```
col += colStep;
}

// Check landing target validity
Piece target =
board[getEndIndex().getRow()][getEndIndex().getCol()];
if (target == null) return true;
return target.getColor() != color;
}

return false; // Not a diagonal move!
}
```

מחלקת Rook

תפקיד המחלקה:

מממשת את כלי הצריח בשחמט. הצריח נע בקווים ישרים (אופקית או אנכית) ובודק שאין כלים חוסמים בנתיב.

תכונות המחלקה:

שם הפעולה	הסבר
Rook(startIndex, color, type)	קונסטרוקטור – מאתחל שדות מחלקת האב
destroy()	מפעיל את Destroy של מחלקת האב
move(color, board)	מאמת תנועה ישרה
eat(color, board)	מאמת אכילה ישרה
inTheWay(color, board)	בודק שהנתיב האנכי/אופקי נקי ושהתא הסופי חוקי

```
//constructor
public Rook(Point startIndex, char color, char type) {
    super.setStartIndex(startIndex);
    super.setEndIndex(startIndex);
    super.setColor(color);
    super.setType(type);
}

//destructor
@Override
public void destroy() {
    super.destroy();
}
```

```
//for rook just return the intheway
@Override
public boolean move(char color, Piece[][] board) {
    // Must be a valid straight line and have a clear path
    return inTheWay(color, board);
}
```

Regular Chess

```

//for rook just return the intheway
@Override
public boolean eat(char color, Piece[][] board) {
// To attack/eat, it MUST follow the exact same structural path rules
return inTheWay(color, board);
}

```

```

/*
The func cheks if the move is valid and if any piece in the way of the rook
input: color of the piece, that board it self
output: true if move is valid else false
*/
@Override
public boolean inTheWay(char color, Piece[][] board) {
// Strictly enforce positional validation (Same row OR same column)
if (getStartIndex().getRow() == getEndIndex().getRow() ||
    getStartIndex().getCol() == getEndIndex().getCol()) {

        // Vertical movement logic
        if (getStartIndex().getCol() == getEndIndex().getCol()) {
            int upOrDown = (getStartIndex().getRow() >
                getEndIndex().getRow()) ? -1 : 1;

            for (int row = getStartIndex().getRow() + upOrDown; row !=
                getEndIndex().getRow(); row += upOrDown) {
                if (board[row][getStartIndex().getCol()] != null) {
                    return false; // Blocked
                }
            }

            // Horizontal movement logic
        } else if (getStartIndex().getRow() == getEndIndex().getRow()) {
            int leftOrRight = (getStartIndex().getCol() >
                getEndIndex().getCol()) ? -1 : 1;

            for (int col = getStartIndex().getCol() + leftOrRight; col !=
                getEndIndex().getCol(); col += leftOrRight) {
                if (board[getStartIndex().getRow()][col] != null) {
                    return false; // Blocked
                }
            }

            // 2. Check the landing square targeting rule
            Piece target =
                board[getEndIndex().getRow()][getEndIndex().getCol()];
            if (target == null) return true; // Empty square is fine
            return target.getColor() != color; // Enemy square is fine,
                // teammate is blocked
        }

        return false; // Not a straight line!
    }
}

```

מחלקת Knight

תפקיד המחלקה:



Regular Chess

מממשת את כלי הסוס בשחמט. הסוס נע בצורת L ומסוגל לקפוץ מעל כלים אחרים.

תכונות המחלקה:

שם הפעולה	הסבר
Knight(startIndex, color, type)	קונסטרוקטור – מאתחל שדות מחלקת האב
move(color, board)	מאמת תנועת L
eat(color, board)	מאמת אכילת L
inTheWay(color, board)	תמיד מחזירה True הסוס קופץ מעל כלים
isValidKnightMove(color, board)	בודק גאומטריית L ותקינות תא הנחיתה (2+1 / 1+2)

```
//constructor
public Knight(Point startIndex, char color, char type) {
    super.setStartIndex(startIndex);
    super.setEndIndex(startIndex);
    super.setColor(color);
    super.setType(type);
}
```

```
//check just if the endpoint is valid
@Override
public boolean move(char color, Piece[][] board) {
    return isValidKnightMove(color, board);
}

//check just if the endpoint is valid
@Override
public boolean eat(char color, Piece[][] board) {
    return isValidKnightMove(color, board);
}
```

```
@Override
public boolean inTheWay(char color, Piece[][] board) {
    // Knights jump over everything, so the path is never "blocked"
    return true;
}
```

```
/*
The func if the knight move is valid by geometry
input: color of the piece and the board
output: true if valid else false
*/
private boolean isValidKnightMove(char color, Piece[][] board) {
    int rowDiff = Math.abs(getEndIndex().getRow() -
        getStartIndex().getRow());
    int colDiff = Math.abs(getEndIndex().getCol() -
        getStartIndex().getCol());
}
```


Regular Chess

```
// Strict L-shape calculation
if ((rowDiff == 2 && colDiff == 1) || (rowDiff == 1 && colDiff == 2)) {

    // Validate the final landing square
    Piece target =
        board[getEndIndex().getRow()][getEndIndex().getCol()];
    if (target == null) return true; // Empty square is valid
    return target.getColor() != color; // Enemy square is valid,
    // teammate blocks it
}

return false; // Not a valid L-shape
}
```

מחלקת Queen

תפקיד המחלקה:

מממשת את כלי המלכה בשחמט. המלכה משלבת את תנועות הצריח והרץ על-ידי הכלת אובייקטים פנימיים של שתי המחלקות האלו.

תכונות המחלקה:

שם התכונה	הסבר
private final Rook m_rook	אובייקט צריח פנימי לטיפול בתנועות ישירות
private final Bishop m_bishop	אובייקט רץ פנימי לטיפול בתנועות אלכסוניות
שם הפעולה	הסבר
Queen(startIndex, color, type)	קונסטרקטור – יוצר אובייקטים פנימיים Rook ו-Bishop
syncSubPieces()	מסנכרנת את נקודות ה Rook ו-Bishop עם נקודות המלכה הנוכחיות
move(color, board)	זזה ומסנכרנת לפי כיוון התנועה
eat(color, board)	אוכלת ומסנכרנת לפי כיוון התנועה
inTheWay(color, board)	מסנכרנת ובודקת חסימות דרך

```
//fields
private final Rook m_rook;
private final Bishop m_bishop;
```

```
//constructor
public Queen(Point startIndex, char color, char type) {
    this.m_rook = new Rook(new Point(startIndex.getRow(),
        startIndex.getCol()), color, Constants.ROOK);
    this.m_bishop = new Bishop(new Point(startIndex.getRow(),
        startIndex.getCol()), color, Constants.BISHOP);
    super.setStartIndex(startIndex);
    super.setEndIndex(startIndex);
    super.setColor(color);
    super.setType(type);
}
```

```
//This func sync the pieces points
private void syncSubPieces() {
```



Regular Chess

```
// Pass brand new Point objects so mutations don't alter shared data
structures
Point currentStart = new Point(getStartIndex().getRow(),
                                getStartIndex().getCol());
Point currentEnd = new Point(getEndIndex().getRow(),
                              getEndIndex().getCol());

m_rook.setStartIndex(currentStart);
m_rook.setEndIndex(currentEnd);

m_bishop.setStartIndex(currentStart);
m_bishop.setEndIndex(currentEnd);
}
```

```
//this func checks if the queen can move diagonal or in straight line
@Override
public boolean move(char color, Piece[][] board) {
    syncSubPieces();

    // Diagonal check
    if (Math.abs(getEndIndex().getRow() - getStartIndex().getRow()) ==
        Math.abs(getEndIndex().getCol() - getStartIndex().getCol())) {
        return m_bishop.move(color, board);
    }

    // Straight line check
    else if (getStartIndex().getRow() == getEndIndex().getRow() ||
             getStartIndex().getCol() == getEndIndex().getCol()) {
        return m_rook.move(color, board);
    }

    return false;
}
```

```
//this func checks if the queen can eat diagonal or in straight line
@Override
public boolean eat(char color, Piece[][] board) {
    syncSubPieces();

    if (Math.abs(getEndIndex().getRow() - getStartIndex().getRow()) ==
        Math.abs(getEndIndex().getCol() - getStartIndex().getCol())) {
        return m_bishop.eat(color, board);
    } else if (getStartIndex().getRow() == getEndIndex().getRow() ||
               getStartIndex().getCol() == getEndIndex().getCol()) {
        return m_rook.eat(color, board);
    }

    return false;
}
```

```
//this func checks if the queen have any pieces int the way diagonal or in
straight line
@Override
public boolean inTheWay(char color, Piece[][] board) {
    syncSubPieces();

    if (Math.abs(getEndIndex().getRow() - getStartIndex().getRow()) ==
```



Regular Chess

```
Math.abs(getEndIndex().getCol() - getStartIndex().getCol())) {  
    return m_bishop.inTheWay(color, board);  
} else if (getStartIndex().getRow() == getEndIndex().getRow() ||  
    getStartIndex().getCol() == getEndIndex().getCol()) {  
    return m_rook.inTheWay(color, board);  
}  
return false;  
}
```

מחלקת King

תפקיד המחלקה:

מממשת את כלי המלך בשחמט. אחראית על תנועת המלך (צעד אחד בכל כיוון), בדיקת מצב שח, ובדיקת שח-מט על-ידי סימולציה של כל המהלכים האפשריים.

תכונות המחלקה:

הסבר	שם הפעולה
קונסטרוקטור – מאתחל שדות מחלקת האב	King(startIndex, color, type)
מאמת שהמלך זז לכל היותר תא אחד ומפעיל inTheWay	move(color, board)
כמו – move מאמת תנועה של תא אחד	eat(color, board)
בודק שהתא הסופי פנוי או שייך ליריב	inTheWay(color, board)
מאתר את המלך ובודק אם כלי יריב יכול לתקוף אותו	isInCheck(color, board)
בודק שח-מט: אם כל המהלכים האפשריים משאירים את המלך בשח	checkMate(color, board)

```
//constructor  
public King(Point startIndex, char color, char type) {  
    super.setStartIndex(startIndex);  
    super.setEndIndex(startIndex);  
    super.setColor(color);  
    super.setType(type);  
}  
  
//destructor  
@Override  
public void destroy() {  
    super.destroy();  
}
```

```
//the func checks if the king make a valid move  
// if yes it return true else false  
@Override  
public boolean move(char color, Piece[][] board) {  
    if (abs(getStartIndex().getRow() - getEndIndex().getRow()) <= Constants.DIF_ROW &&  
        abs(getStartIndex().getCol() - getEndIndex().getCol()) <= Constants.DIF_COL) {  
        return inTheWay(color, board);  
    }  
    return false;  
}
```

Regular Chess

```

//the func checks if the king can eat
// if yes it return true else false
@Override
public boolean eat(char color, Piece[][] board) {
    if (abs(getStartIndex().getRow() - getEndIndex().getRow()) <= Constants.DIF_ROW &&
        abs(getStartIndex().getCol() - getEndIndex().getCol()) <= Constants.DIF_COL) {
        return inTheWay(color, board);
    }
    return false;
}

```

```

//the func checks if theres any piece in the way
// if yes it return true else false
@Override
public boolean inTheWay(char color, Piece[][] board) {
    Piece target = board[getEndIndex().getRow()][getEndIndex().getCol()];
    if (target == null) return true;
    return target.getColor() != color;
}

```

```

/*
    The func checks if the king is in check
    input: color of the king and the game board
    output: if in check true else false
*/
public boolean isInCheck(char color, Piece[][] board) {
    int kRow = -1, kCol = -1;

    // Locates the King on the active board
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            if (board[i][j] instanceof King && board[i][j].getColor() == color) {
                kRow = i;
                kCol = j;
                break;
            }
        }
    }

    if (kRow == -1) return false;

    char opponentColor = (color == Constants.WHITE_PIECE) ? Constants.BLACK_PIECE :
        Constants.WHITE_PIECE;

    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            Piece p = board[i][j];
            if (p == null || p.getColor() != opponentColor) continue;

            // Create deep copies to isolate side-effects
            Point savedStart = new Point(p.getStartIndex().getRow(), p.getStartIndex().getCol());
            Point savedEnd = new Point(p.getEndIndex().getRow(), p.getEndIndex().getCol());

            p.setStartIndex(new Point(i, j));
            p.setEndIndex(new Point(kRow, kCol));

```



Regular Chess

```
// Attack evaluation using enemy context color
boolean canAttack = p.eat(opponentColor, board);

p.setStartIndex(savedStart);
p.setEndIndex(savedEnd);

if (canAttack) {
    return true;
}

return false;
}
```

```
/*
    The func checks if the king is in checkmate
    input: color of the king and the game board
    output: if in checkmate true else false
*/
public boolean checkMate(char color, Piece[][] board) {
    // If the king is not even in check, it cannot be checkmate
    if (!isInCheck(color, board)) return false;

    // Loop through all squares to find current player's pieces
    for (int i = 0; i < Constants.BOARD_LEN; i++) {
        for (int j = 0; j < Constants.BOARD_LEN; j++) {
            Piece p = board[i][j];
            if (p == null || p.getColor() != color) continue;

            // Loop through all potential target destination squares
            for (int x = 0; x < Constants.BOARD_LEN; x++) {
                for (int y = 0; y < Constants.BOARD_LEN; y++) {
                    if (i == x && j == y) continue;

                    // Check if the piece rules allow moving from (i,j) to (x,y)
                    p.setStartIndex(new Point(i, j));
                    p.setEndIndex(new Point(x, y));

                    boolean legal = p.move(color, board) || p.eat(color, board);

                    if (!legal) {
                        continue; // Not a valid chess move, skip testing it
                    }

                    // Deep copy state attributes before running simulation
                    Piece captured = board[x][y];

                    // Perform Simulation
                    board[x][y] = p;
                    board[i][j] = null;
                    p.setStartIndex(new Point(x, y));
                    p.setEndIndex(new Point(x, y));

                    // Evaluate if this simulated move successfully broke the check line
                }
            }
        }
    }
}
```

Regular Chess

```

boolean stillInCheck = isInCheck(color, board);

// Undo Simulation perfectly to original states
board[i][j] = p;
board[x][y] = captured;
p.setStartIndex(new Point(i, j));
p.setEndIndex(new Point(i, j));

if (!stillInCheck) {
    Log.d("CHECKMATE_DEBUG", "Escape route found! Piece " + p.getClass().getSimpleName() +
        " can move from (" + i + ", " + j + ") to (" + x + ", " + y + ")");
    return false;
}
}
}
}
}

// If we exhausted every single friendly piece's move combinations
// and every single one leaves the king in check, it is Checkmate!
return true;
}

```

מחלקת Pawn

תפקיד המחלקה:

מממשת את כלי החייל בשחמט. החייל נע קדימה (בכיוון תלוי צבע), יכול לנוע שתי משבצות ממיקום ההתחלה, אוכל באלכסון, ויכול להתקדם לקצה הלוח ולעבור קידום.

תכונות המחלקה:

שם התכונה	הסבר
private int m_jumps	מונה קפיצות – נשמר לשימוש עתידי ומאופס בdestroy
שם הפעולה	הסבר
Pawn(startIndex, color, type)	קונסטרוקטור – מאתחל m_jumps ושדות מחלקת האב
destroy()	מאפס m_jumps ומפעיל destroy של האב
move(color, board)	מאמת תנועה ישרה קדימה (1 או 2 משבצות מהשורה ההתחלתית)
eat(color, board)	מאמת אכילה אלכסונית של כלי יריב
inTheWay(color, board)	בודק שהתא האמצעי פנוי בתנועה של שתי משבצות
turnPiece(char type)	מחליף את סוג הכלי לאחד מארבעת הכלים לקידום

```

//field
private int m_jumps;

```



Regular Chess

```
//constructor
public Pawn(Point startIndex, char color, char type) {
    this.m_jumps = Constants.PAWN_JUMPS;
    super.setStartIndex(startIndex);
    super.setEndIndex(startIndex);
    super.setColor(color);
    super.setType(type);
}

//destructor
public void destroy() {
    this.m_jumps = 0;
    super.destroy();
}
```

```
//check if the pawn can be moved one or two up or down
@Override
public boolean move(char color, Piece[][] board) {
    int startRow = getStartIndex().getRow();
    int startCol = getStartIndex().getCol();
    int endRow = getEndIndex().getRow();
    int endCol = getEndIndex().getCol();

    // Straight movement must land on an empty square
    if (board[endRow][endCol] != null) {
        return false;
    }

    // Straight movement cannot change columns
    if (startCol != endCol) {
        return false;
    }

    // Check the path using the corrected inTheWay logic
    if (!inTheWay(color, board)) {
        return false; // Something is blocking the path forward
    }

    // Validate the exact step length allowed based on color direction
    if (color == Constants.WHITE_PIECE) {
        // White moves down (increasing rows): either 1 step, or 2 steps from starting row (row 1)
        if (endRow == startRow + 1) {
            return true;
        }
        if (startRow == 1 && endRow == 3) {
            return true;
        }
    } else {
        // Black moves up (decreasing rows): either 1 step, or 2 steps from starting row (row 6)
        if (endRow == startRow - 1) {
            return true;
        }
        if (startRow == 6 && endRow == 4) {
            return true;
        }
    }
}
```



Regular Chess

```
return true;
}
}

return false;
}
```

```
//the func checks if the pawn can eat diagonal
@Override
public boolean eat(char color, Piece[][] board) {
    int startRow = getStartIndex().getRow();
    int startCol = getStartIndex().getCol();
    int endRow = getEndIndex().getRow();
    int endCol = getEndIndex().getCol();

    Piece target = board[endRow][endCol];

    // To capture, there must be an enemy piece sitting on the target square
    if (target == null || target.getColor() == color) {
        return false;
    }

    int rowDiff = endRow - startRow;
    int colDiff = abs(endCol - startCol);

    // Capturing must be exactly 1 square diagonally
    if (colDiff != 1) {
        return false;
    }

    if (color == Constants.WHITE_PIECE) {
        return rowDiff == 1; // White captures moving down
    } else {
        return rowDiff == -1; // Black captures moving up
    }
}
```

```
//the func checks if there is a player in the way of the pawn moves
@Override
public boolean inTheWay(char color, Piece[][] board) {
    int startRow = getStartIndex().getRow();
    int startCol = getStartIndex().getCol();
    int endRow = getEndIndex().getRow();

    // If moving 2 squares, check the intermediate square right in front of the pawn
    if (abs(endRow - startRow) == 2) {
        int direction = (color == Constants.WHITE_PIECE) ? 1 : -1;
        int intermediateRow = startRow + direction;

        // If the intermediate square is occupied, path is blocked
        if (board[intermediateRow][startCol] != null) {
            return false;
        }
    }
}
```




Regular Chess

```
// Path is completely clear  
return true;  
}
```

```
//the func turn the piece type to a queen rook bishop or knight  
public boolean turnPiece(char type) {  
    if (type == Constants.QUEEN) super.setType(Constants.QUEEN);  
    else if (type == Constants.ROOK) super.setType(Constants.ROOK);  
    else if (type == Constants.BISHOP) super.setType(Constants.BISHOP);  
    else if (type == Constants.KNIGHT) super.setType(Constants.KNIGHT);  
    else return false;  
    return true;  
}
```

מחלקת Round

תפקיד המחלקה:

מנהל את מצב לוח השחמט: בניית הלוח ממחרוזות, ביצוע וסימולציה של מהלכים, בדיקת שח ושח-מט, וטיפול בקידום חיילים.

תכונות המחלקה:

שם התכונה	הסבר
private final Piece[][] _board	מטריצת 8x8 המייצגת את כלי הלוח
private char _color	צבע השחקן שתורו לשחק כעת
private Point _p, _p1	נקודות ההתחלה והסיום של המהלך הנוכחי
String _inputBoard	מחרוזת 64 תווים המייצגת את מצב הלוח ההתחלתי
private boolean _isInCheck	דגל שמציין האם המהלך האחרון נכשל בגלל שח

שם הפעולה	הסבר
Round(inputBoard)	קונסטרוקטור – מאתחל לוח ממחרוזת תווים
setMovePoints(start, end)	מגדירה את נקודות ההתחלה והסיום של המהלך הבא
moveInBoard()	מחזיר True אם המבצע מהלך תוך סימולציה ובדיקת חוקיות עובר בהצלחה
findKing(color)	מאתר ומחזיר את אובייקט המלך של הצבע הנתון
isPlayerInCheck()	מחזיר True אם המלך הנוכחי נמצא בשח
isPlayerInCheckMate()	מחזיר True



Regular Chess

אם המלך הנוכחי נמצא בשח-מט	
ממיר את מחרוזת _inputBoard למטריצת אובייקטי Piece	turnToBoard()
מחזיר True אם חייל הגיע לשורת הקידום	checkLastMoveForPromotion()
מחליף את החייל בכלי הנבחר על הלוח	promotePawnOnBoard(absolutePiece)
מגדירה ומחזירה את צבע התור הנוכחי	setColor(c) / getColor()
מחזיר את דגל _isInCheck	isInCheck()
מחזיר את מטריצת הלוח הנוכחית	getBoard()

```
//fields
private final Piece[][] _board;
private char _color;
private Point _p, _p1;
String _inputBoard;
private boolean _isInCheck = false;
```

```
//constructor
public Round(String inputBoard) {
    if (inputBoard == null || inputBoard.length() < 64) {
        Log.e("Round", "Invalid board string length!");
        this._inputBoard = "0".repeat(64);
    } else {
        this._inputBoard = inputBoard;
    }

    this._board = new Piece[Constants.BOARD_LEN][Constants.BOARD_LEN];
    this._p = new Point();
    this._p1 = new Point();
    this._color = Constants.WHITE_PIECE;

    turnToBoard();
}
```

```
/*
The func get to points and reset the p and p1 points
*/
public void setMovePoints(Point start, Point end) {
```



Regular Chess

```
this._p = start;  
this._p1 = end;  
}
```

```
//the function returns a bool var of the islnCheck  
public boolean islnCheck() {  
    return _islnCheck;  
}
```

```
/*  
The func move the piece that locates in p to p1 but  
first it simulate to see if that cause any check  
*/  
public boolean moveInBoard() {  
    if (_p == null || _p1 == null) return false;  
  
    Piece pieceToMove = _board[_p.getRow()][_p.getCol()];  
  
    if (pieceToMove == null) {  
        Log.d("CHESS_DEBUG", "No piece at " + _p.getRow() + "," + _p.getCol());  
        return false;  
    }  
  
    if (pieceToMove.getColor() != this._color) {  
        Log.d("CHESS_DEBUG", "Wrong color. Piece=" + pieceToMove.getColor() + " Turn=" + this._color);  
        return false;  
    }  
  
    pieceToMove.setStartIndex(new Point(_p.getRow(), _p.getCol()));  
    pieceToMove.setEndIndex(new Point(_p1.getRow(), _p1.getCol()));  
  
    boolean canMove = pieceToMove.move(this._color, _board);  
    boolean canEat = pieceToMove.eat(this._color, _board);  
  
    Log.d("CHESS_DEBUG", "Move from (" + _p.getRow() + "," + _p.getCol() + ") to (" +  
        + _p1.getRow() + "," + _p1.getCol() + ")");  
    Log.d("CHESS_DEBUG", "canMove=" + canMove + " canEat=" + canEat);  
  
    if (!canMove && !canEat) {  
        Log.d("CHESS_DEBUG", "Piece says move is illegal by its own rules");  
        return false;  
    }  
  
    // Simulate  
    Piece captured = _board[_p1.getRow()][_p1.getCol()];  
    _board[_p1.getRow()][_p1.getCol()] = pieceToMove;  
    _board[_p.getRow()][_p.getCol()] = null;  
  
    pieceToMove.setStartIndex(new Point(_p1.getRow(), _p1.getCol()));  
    pieceToMove.setEndIndex(new Point(_p1.getRow(), _p1.getCol()));  
  
    King king = findKing(this._color);  
    Log.d("CHESS_DEBUG", "King found: " + (king != null));  
    if (king != null) {  
        Log.d("CHESS_DEBUG", "King position: " + king.getStartIndex().getRow() + "," +
```

Regular Chess

```

king.getStartIndex().getCol());
    }

    boolean stillInCheck = (king != null) && king.isInCheck(this._color, _board);
    Log.d("CHESS_DEBUG", "stillInCheck after simulation=" + stillInCheck);

    // Undo
    _board[_p.getRow()][_p.getCol()] = pieceToMove;
    _board[_p1.getRow()][_p1.getCol()] = captured;
    pieceToMove.setStartIndex(new Point(_p.getRow(), _p.getCol()));
    pieceToMove.setEndIndex(new Point(_p1.getRow(), _p1.getCol()));

    if (stillInCheck) {
        _isInCheck = true;
        Log.d("CHESS_DEBUG", "REJECTED: king still in check");
        return false;
    }

    // Apply
    _board[_p1.getRow()][_p1.getCol()] = pieceToMove;
    _board[_p.getRow()][_p.getCol()] = null;
    pieceToMove.setStartIndex(new Point(_p1.getRow(), _p1.getCol()));
    pieceToMove.setEndIndex(new Point(_p1.getRow(), _p1.getCol()));

    _isInCheck = false;
    return true;
}

```

```

//The func try to find the king by the color its given
public King findKing(char color) {
    for (int i = 0; i < 8; i++)
        for (int j = 0; j < 8; j++)
            if (_board[i][j] instanceof King && _board[i][j].getColor() == color)
                return (King) _board[i][j];
    return null;
}

```

```

/*
The func return true if the player king is in checkmate else false
*/
public boolean isPlayerInCheckMate() {
    King activeKing = findKing(this._color);
    if (activeKing != null)
        return activeKing.checkMate(this._color, this._board);
    return false;
}

```

```

/*
The func return true if the player king is in check else false
*/
public boolean isPlayerInCheck() {
    King activeKing = findKing(this._color);
    if (activeKing != null)

```

Regular Chess

```
return activeKing.isInCheck(this._color, this._board);
return false;
}
```

```
/*
The func takes a list of chars and turn it to a full board of pieces
*/
public void turnToBoard() {
if (this._inputBoard == null || this._inputBoard.length() < 64) {
Log.e("CHESS_ERROR", "Board string is too short! Length: " +
(this._inputBoard == null ? 0 : this._inputBoard.length()));
return;
}

//the turn board logic
int txtIndex = 0;
for (int row = 0; row < 8; row++) {
for (int col = 0; col < 8; col++) {
char type = this._inputBoard.charAt(txtIndex);
Point p = new Point(row, col);

if (type == ' ' || type == '0') {
this._board[row][col] = null;
} else {
char color = Character.isLowerCase(type) ? Constants.WHITE_PIECE : Constants.BLACK_PIECE;
char lowerType = Character.toLowerCase(type);

if (lowerType == Constants.KING) this._board[row][col] = new King(p, color, type);
else if (lowerType == Constants.KNIGHT) this._board[row][col] = new Knight(p, color, type);
else if (lowerType == Constants.PAWN) this._board[row][col] = new Pawn(p, color, type);
else if (lowerType == Constants.QUEEN) this._board[row][col] = new Queen(p, color, type);
else if (lowerType == Constants.ROOK) this._board[row][col] = new Rook(p, color, type);
else if (lowerType == Constants.BISHOP) this._board[row][col] = new Bishop(p, color, type);
}

txtIndex++;
}
}
}
```

```
// checks if the piece that just successfully moved is a pawn landing on its promotion row
public boolean checkLastMoveForPromotion() {
if (_p1 == null) return false;

int targetRow = _p1.getRow();
Piece lastMovedPiece = _board[targetRow][_p1.getCol()];

if (lastMovedPiece instanceof Pawn) {

// White Pawns move DOWN to row index 7
if (targetRow == 7 && lastMovedPiece.getColor() == Constants.WHITE_PIECE) {
Log.d("CHESS_PROMOTION", "White pawn reached row 7! Opening Dialog...");
return true;
}
}
```



Regular Chess

```
// Black Pawns move UP to row index 0
if (targetRow == 0 && lastMovedPiece.getColor() == Constants.BLACK_PIECE) {
    Log.d("CHESS_PROMOTION", "Black pawn reached row 0! Opening Dialog...");
    return true;
}
return false;
}
```

```
/*
    The func promote the pawn on the board by
    turning the pawn to queen, bishop, knight or a rook
*/
public void promotePawnOnBoard(char absolutePiece) {
    if (_p1 == null) return; // Exit early safely if there's no point context

    int row = _p1.getRow();
    int col = _p1.getCol();
    Point p = new Point(row, col);

    // Determine color context by checking the case format style of the character parameter
    char pieceColor = Character.isLowerCase(absolutePiece) ? Constants.WHITE_PIECE : Constants.BLACK_PIECE;
    char lowerType = Character.toLowerCase(absolutePiece);

    if (lowerType == Constants.QUEEN) this._board[row][col] = new Queen(p, pieceColor, absolutePiece);
    else if (lowerType == Constants.ROOK) this._board[row][col] = new Rook(p, pieceColor, absolutePiece);
    else if (lowerType == Constants.KNIGHT) this._board[row][col] = new Knight(p, pieceColor, absolutePiece);
    else if (lowerType == Constants.BISHOP) this._board[row][col] = new Bishop(p, pieceColor, absolutePiece);

    Log.d("CHESS_PROMOTION", "Pawn at (" + row + ", " + col + ") promoted to char target: " + absolutePiece);
}
```

```
//set the color
public void setColor(char c) { this._color = c; }
//return the turn by the color
public char getColor() { return _color; }
//return the game board
public Piece[][] getBoard() { return _board; }
```

מחלקת Chess

תפקיד המחלקה:

מחלקת הבקרה הראשית של משחק השחמט. מנהלת את תורות השחקנים, טיפול בשח ושח-מט, קידום חיילים וסיום המשחק.

תכונות המחלקה:

שם התכונה	הסבר
private char _move	(לסיום W / B / 0) הצבע של השחקן שתורו כעת
public Round _gameRounds	אובייקט Round שמנהל את מצב הלוח



Regular Chess

שם הפעולה	הסבר
Chess()	Round קונסטרוקטור – מאתחל תור לבן ו עם מיקום ברירת מחדל
game(context)	מבצע את המהלך הנוכחי, בודק קידום, מחליף תור ומעריך שח/שח-מט
completeTurnSequence(context)	מחליף תור, בודק שח ומעריך שח-מט או תיקו
finishedGame(context, isStalemate)	מעדכן סטטיסטיקות ופותח את GameOverDialog
switchTurn()	_move מחליפה את W ל B - בין Round ומעדכנת את
isPawnEligibleForPromotion()	בודק אם החייל האחרון שנע זכאי לקידום
hasNoLegalMovesLeft()	בודק אם אין מהלכים חוקיים (לצורך זיהוי תיקו)
getMove() / setMove(char)	מחזירה/מגדירה את הצבע הפעיל

```
//fields
private char _move;
public Round _gameRounds;
```

```
//constructor
public Chess() {
    this._move = Constants.WHITE_PIECE;
    this._gameRounds = new Round(Constants.FRONTEND_MSG);
}
```

```
/*
The func is responsible on the game logic first selection of the piece to move
and then after all of the checks it checks for pawn promotion and if there is
a win lose or tie it open the game over dialog and finish the game
*/
public void game(Context context) {
    this._gameRounds.setColor(_move);
    boolean moveSuccessful = this._gameRounds.moveInBoard();

    if (moveSuccessful) {
        // Check for Pawn Promotion FIRST (While it is still the moving player's turn)
        if (isPawnEligibleForPromotion()) {
            // Open the custom dialog, passing the context, current player color ('W' or 'B'), and the choice callback
            PromotionDialog dialog = new PromotionDialog(context, _move, new
                PromotionDialog.PromotionCallback() {
                    @Override
                    public void onPieceSelected(char chosenPiece) {
                        // Normalize the character case depending on piece color matching your matrix framework rules
                        char absolutePiece = (_move == Constants.WHITE_PIECE)
                            ? Character.toLowerCase(chosenPiece)
                            : Character.toUpperCase(chosenPiece);

                        // Update board array layout cell with the new upgraded piece character type
                        _gameRounds.promotePawnOnBoard(absolutePiece);
                    }
                });
        }
    }
}
```



Regular Chess

```
completeTurnSequence(context);
    }
    });
    dialog.show();

    } else {
// No promotion happened! Run normal turn finalization workflows instantly
        completeTurnSequence(context);
    }

    } else {
// Move was rejected — tell the player why
        if (this._gameRounds.isInCheck()) {
            Toast.makeText(context,
                "Illegal move — your King is still in check!",
                Toast.LENGTH_SHORT).show();
        }
    }
}
```

```
/*The func complete all the turn event from switching turns
to check if the kings in check or checkmate or stalemate
*/
private void completeTurnSequence(Context context) {
// Switch turn here so we can evaluate the NEW active player's state
    switchTurn();

    boolean inCheck = this._gameRounds.isPlayerInCheck();

// Warn the new active player if they are now caught in check
    if (inCheck) {
        String colorName = (_move == Constants.WHITE_PIECE) ? "White" : "Black";
        android.util.Log.d("CHESS_GAME", colorName + " is in CHECK!");
        Toast.makeText(context,
            colorName + " is in CHECK!",
            Toast.LENGTH_SHORT).show();
    }

// Evaluate structural end-game check patterns safely
    if (this._gameRounds.isPlayerInCheckMate()) {
// No legal moves left AND King is in check -> Checkmate!
        finishedGame(context, false);
    } else if (!inCheck && hasNoLegalMovesLeft()) {
// No legal moves left AND King is NOT in check -> Stalemate Draw!
        finishedGame(context, true);
    }
}
```

```
// End-Game handling pipeline supporting both Checkmates and Stalemates
public void finishedGame(Context context, boolean isStalemate) {
    GlobalStat globalState = (GlobalStat) context.getApplicationContext();
    final boolean whiteWon = (this._move == Constants.BLACK_PIECE);

    if (isStalemate) {
// A stalemate is a draw
```




Regular Chess

```
globalState.userStats.stalemates++; // Increment your draws accumulator tracker
android.util.Log.d("CHESS_GAME", "Game ended in a Stalemate (Draw)!");
    } else {
        if (whiteWon) {
            globalState.userStats.wins++;
            android.util.Log.d("CHESS_GAME", "White wins by Checkmate!");
        } else {
            globalState.userStats.losses++;
            android.util.Log.d("CHESS_GAME", "Black wins by Checkmate!");
        }
    }

    this._move = '0';

    //move to the game over dialog
    Intent dialogIntent = new Intent(context, GameOverDialog.class);
    dialogIntent.putExtra("IS_STALEMATE", isStalemate);
    dialogIntent.putExtra("WHITE_WON", whiteWon);
    dialogIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
    context.startActivity(dialogIntent);
}
```

```
//this func switch to white or black or zero ( game ended )
private void switchTurn() {
    _move = (_move == Constants.WHITE_PIECE) ? Constants.BLACK_PIECE : Constants.WHITE_PIECE;
    _gameRounds.setColor(_move);
}
```

```
//set a new move
public void setMove(char move) {
    this._move = move;
}

//return the turn
public char getMove() {
    return _move;
}
```

Regular Chess

PromotionDialog מחלקת

תפקיד המחלקה:

דיאלוג שמוצג כאשר חייל מגיע לשורת הקידום. מציג ארבעה כפתורים (מלכה, צריח, רץ, סוס) עם תמונות מתאימות לצבע השחקן ומחזיר את הבחירה דרך callback.

תכונות המחלקה:

שם התכונה	הסבר
private final PromotionCallback callback	ממשק callback שמועבר עם בחירת הכלי
private final char playerColor	צבע השחקן – קובע את תמונות הכפתורים

שם הפעולה	הסבר
PromotionDialog(context, playerColor, callback)	קונסטרקטור – שומר צבע וcallback
onCreate(savedInstanceState)	בונה את ה Dialog, מגדיר תמונות לפי צבע ומחבר לחצנים
select(char piece)	שולח את הבחירה ל callback וסוגר את הדיאלוג

```
private final PromotionCallback callback;
private final char playerColor;
```

```
// Constructor that pass the color of the current turn player to load the correct assets
public PromotionDialog(Context context, char playerColor, PromotionCallback callback) {
    super(context);
    this.playerColor = playerColor;
    this.callback = callback;
}
```

```
/*
    The func bind the image buttons and set their images by the turn color
    and open the dialog so the player have to choose which piece he wants to promote
*/
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    requestWindowFeature(Window.FEATURE_NO_TITLE);
    setContentView(R.layout.dialog_promotion);
    setCancelable(false); // Make sure they can't dismiss without choosing

    ImageButton btnQueen = findViewById(R.id.img_btn_queen);
    ImageButton btnRook = findViewById(R.id.img_btn_rook);
    ImageButton btnKnight = findViewById(R.id.img_btn_knight);
    ImageButton btnBishop = findViewById(R.id.img_btn_bishop);
}
```

Regular Chess

```
// Set the images programmatically depending on whose turn it is
if (playerColor == Constants.WHITE_PIECE) {
    btnQueen.setImageResource(R.drawable.queenw);
    btnRook.setImageResource(R.drawable.rookw);
    btnKnight.setImageResource(R.drawable.knightw);
    btnBishop.setImageResource(R.drawable.bishopw);
} else {
    btnQueen.setImageResource(R.drawable.queenb);
    btnRook.setImageResource(R.drawable.rookb);
    btnKnight.setImageResource(R.drawable.knightb);
    btnBishop.setImageResource(R.drawable.bishopb);
}

// Set up the click listeners to trigger selection
btnQueen.setOnClickListener(v -> select('Q'));
btnRook.setOnClickListener(v -> select('R'));
btnKnight.setOnClickListener(v -> select('N'));
btnBishop.setOnClickListener(v -> select('B'));
```

```
//The func send the desire piece to the callback and close the dialog
private void select(char piece) {
    callback.onPieceSelected(piece);
    dismiss();
}
```

מחלקת ChessView

תפקיד המחלקה:

View מותאם אישית שמצייר את לוח השחמט, מציג שעוני ספירה לאחור לשני השחקנים, מטפל בדרישות ובחירות שחקנים של המשתמש ומנגן צלילי אזהרה בזמן אמת.

תכונות המחלקה:

שם התכונה	הסבר
private final Paint paint / textPaint	אובייקט ציור לרקעים ולטקסטים
private float squareSize / boardTopOffset	גודל משבצת וסטיית הלוח מלמעלה – מחושבים ב-onDraw
private Chess chessGame	אובייקט לוגיקת המשחק המחובר לתצוגה
private Point firstClick	הנקודה הראשונה שנבחרה על ידי המשתמש
private final Map<String, Bitmap> pieceBitmaps	מפה (ועוד) מפה מפתח-תמונה לכל כלי (wp, bk)
private long whiteTimeLeft / blackTimeLeft	זמן שנותר לכל שחקן במילישניות
private CountDownTimer activeTimer	הטיימר הפעיל לשחקן שתורו
private MediaPlayer tickPlayer	מנגן צליל אזהרה ב-15 השניות האחרונות
private OnTimeOutListener timeOutListener	שמועבר לאחר שהזמן נגמר callback

Regular Chess

שם הפעולה	הסבר
ChessView(context, attrs)	קונסטרוקטור – טוען תמונות, מאתחל צלילים ומפעיל טיימר
loadPieceBitmaps()	טוען את כל 12 תמונות הכלים למפה
onDraw(canvas)	לוח, כלים ושכבת סטטוס, מצייר שעוני ספירה
drawClockCard(canvas, timeLeft, yPos, isWhite)	מצייר כרטיס שעון לשחקן – צבע משתנה לפי זמן ותור
startTurnTimer()	מפעיל טיימר לשחקן הנוכחי ומעדכן את התצוגה בכל שניה
stopTimer()	מבטל את הטיימר הפעיל
drawBoardGrid(canvas)	מצייר את רשת הלוח עם הדגשת תא נבחר
drawPiecesFromLogic(canvas)	מצייר את כלי הלוח לפי מצב הRound
drawStatusOverlay(canvas)	מציג טקסט CHECKMATE כאשר המשחק נגמר
onTouchEvent(event)	מטפל במגע: בחירת כלי ואז ביצוע מהלך

```
//fields
private final Paint paint = new Paint();
private final Paint textPaint = new Paint();
private float squareSize = 0f;
private float boardTopOffset = 0f;

private Chess chessGame;
private Point firstClick = null;
private final Map<String, Bitmap> pieceBitmaps = new HashMap<>();

private long whiteTimeLeft = 300000;
private long blackTimeLeft = 300000;
private CountDownTimer activeTimer;
private MediaPlayer tickPlayer;
```

```
//constructor that start the timers reset the tick sound and load the images to the board
public ChessView(Context context, AttributeSet attrs) {
    super(context, attrs);
    this.chessGame = new Chess();
    paint.setAntiAlias(true);
    textPaint.setAntiAlias(true);
    textPaint.setTypeface(Typeface.create(Typeface.MONOSPACE, Typeface.BOLD));

    loadPieceBitmaps();
    try {
        tickPlayer = MediaPlayer.create(context, R.raw.tick);
    } catch (Exception e) {
        android.util.Log.e("ChessView", "Sound file missing");
    }
    startTurnTimer();
}
```



Regular Chess

```
//This func load all the images if the pieces on to the board
private void loadPieceBitmaps() {
    int[] resIds = {
        R.drawable.pawnw, R.drawable.rookw, R.drawable.knightw, R.drawable.bishopw, R.drawable.queenw,
        R.drawable.kingw,
        R.drawable.pawnb, R.drawable.rookb, R.drawable.knightb, R.drawable.bishopb, R.drawable.queenb,
        R.drawable.kingb
    };
    String[] keys = {"wp", "wr", "wn", "wb", "wq", "wk", "bp", "br", "bn", "bb", "bq", "bk"};
    for (int i = 0; i < resIds.length; i++) {
        pieceBitmaps.put(keys[i], BitmapFactory.decodeResource(getResources(), resIds[i]));
    }
}
```

```
//This func draw the board and clock on to the screen
@Override
protected void onDraw(Canvas canvas) {
    super.onDraw(canvas);

    // Reserve at least 160 pixels at top and bottom for clocks
    float verticalPadding = 160f;
    float availableHeightForBoard = getHeight() - (verticalPadding * 2);
    float availableWidth = getWidth();

    // The square size is the smaller of the two to ensure it's a perfect square that fits
    squareSize = Math.min(availableWidth / 8f, availableHeightForBoard / 8f);

    float totalBoardHeight = squareSize * 8;
    float totalBoardWidth = squareSize * 8;

    // Center the board vertically and horizontally
    boardTopOffset = (getHeight() - totalBoardHeight) / 2f;
    float boardLeftOffset = (getWidth() - totalBoardWidth) / 2f;

    // Draw Clocks (positioned relative to board, not screen edges)
    drawClockCard(canvas, whiteTimeLeft, boardTopOffset - 140, true);
    drawClockCard(canvas, blackTimeLeft, boardTopOffset + totalBoardHeight + 20, false);

    // Draw the Board
    canvas.save();
    canvas.translate(boardLeftOffset, boardTopOffset);
    drawBoardGrid(canvas);
    drawPiecesFromLogic(canvas);
    canvas.restore();

    drawStatusOverlay(canvas);
}
```

Regular Chess

```
//This func draw on to the screen the clock card and when the timer gets to 15 sec
//the clock card turn from yellow to red and if the turn isnt his the card color turn to white
private void drawClockCard(Canvas canvas, long timeLeft, float yPos, boolean isWhiteClock) {
    String timeStr = String.format(Locale.getDefault(), "%02d:%02d",
        (timeLeft / 1000) / 60, (timeLeft / 1000) % 60);

    float cardWidth = getWidth() * 0.5f;
    float cardHeight = 110f;
    float xPos = (getWidth() - cardWidth) / 2;

    RectF card = new RectF(xPos, yPos, xPos + cardWidth, yPos + cardHeight);

    boolean isCurrentTurn = (isWhiteClock && chessGame.getMove() == Constants.WHITE_PIECE) ||
        (!isWhiteClock && chessGame.getMove() == Constants.BLACK_PIECE);

    if (timeLeft <= 15000) {
        // Intense warning red when 15 seconds or less remain
        paint.setColor(Color.parseColor("#EF5350"));
    } else if (isCurrentTurn) {
        // Bright yellow to denote active normal thinking turn state
        paint.setColor(Color.parseColor("#FFEB3B"));
    } else {
        // Plain white for the resting opponent card lane
        paint.setColor(Color.parseColor("#FFFFFF"));
    }

    // Render card background element
    paint.setShadowLayer(8, 0, 4, Color.parseColor("#22000000"));
    canvas.drawRoundRect(card, 20, 20, paint);
    paint.clearShadowLayer();

    textPaint.setTextSize(25f);
    textPaint.setColor(timeLeft <= 15000 ? Color.WHITE : Color.DKGRAY);
    textPaint.setTextAlign(Paint.Align.CENTER);
    canvas.drawText(isWhiteClock ? "WHITE" : "BLACK", xPos + (cardWidth/2), yPos + 35, textPaint);

    textPaint.setTextSize(55f);
    textPaint.setColor(timeLeft <= 15000 ? Color.WHITE : Color.BLACK);
    canvas.drawText(timeStr, xPos + (cardWidth/2), yPos + 95, textPaint);
}
```

```
//This function stop the timer from countdown
public void stopTimer() {
    if (activeTimer != null) {
        activeTimer.cancel();
    }
}
```

```
/**
 * Starts or resumes the countdown for the player whose turn it currently is.
 * This is public so it can be called from the Activity when a dialog is closed.
 */
public void startTurnTimer() {
    // cancel the previous timer before starting a new one
}
```

Regular Chess

```

        if (activeTimer != null) {
            activeTimer.cancel();
        }

        // Determine whose turn it is from the chess engine
        final boolean isWhiteTurn = (chessGame.getMove() == Constants.WHITE_PIECE);
        long currentTime = isWhiteTurn ? whiteTimeLeft : blackTimeLeft;

        // Create the new countdown
        // millisInFuture: the time remaining for the active player
        // countDownInterval: 1000ms (1 second) per tick
        activeTimer = new CountDownTimer(currentTime, 1000) {
            @Override
            public void onTick(long millisUntilFinished) {
                // Update the correct time variable
                if (isWhiteTurn) {
                    whiteTimeLeft = millisUntilFinished;
                }
                GlobalStat globalStat = (GlobalStat) getContext().getApplicationContext();
                globalStat.userStats.timespent += 300000 - whiteTimeLeft;
                globalStat.userStats.timespent /= 60000; //get mins
            } else {
                blackTimeLeft = millisUntilFinished;
            }
        };

        // Play a warning sound if 15 seconds or less remain
        if (millisUntilFinished <= 15000 && tickPlayer != null) {
            if (!tickPlayer.isPlaying()) {
                tickPlayer.start();
            }
        }

        // Redraw the view to update the clock UI
        invalidate();
    }

    @Override
    public void onFinish() {
        // Logic for when time runs out
        final boolean whiteLost = isWhiteTurn;

        if (isWhiteTurn) whiteTimeLeft = 0;
        else blackTimeLeft = 0;

        // Stop the game interaction by changing the engine state to '0'
        chessGame.setMove('0');
        invalidate();

        android.util.Log.d("ChessTimer", "Time is up for " + (whiteLost ? "White" : "Black"));

        //open the game over dialog
        if (timeOutListener != null) {
            timeOutListener.onTimeOut(whiteLost);
        }
    }
}

```



Regular Chess

```
}.start(); // Start the timer immediately  
}
```

```
//this func draws the board grid on to the screen  
private void drawBoardGrid(Canvas canvas) {  
    for (int row = 0; row < 8; row++) {  
        for (int col = 0; col < 8; col++) {  
            paint.setColor((row + col) % 2 == 0 ? Color.parseColor("#EBECD0") : Color.parseColor("#779556"));  
            canvas.drawRect(col * squareSize, row * squareSize, (col + 1) * squareSize, (row + 1) * squareSize,  
                             paint);  
  
            if (firstClick != null && firstClick.getRow() == row && firstClick.getCol() == col) {  
                paint.setColor(Color.argb(160, 186, 202, 68));  
                canvas.drawRect(col * squareSize, row * squareSize, (col + 1) * squareSize, (row + 1) * squareSize,  
                                paint);  
            }  
        }  
    }  
}
```

```
//this func draw the pieces from the game board in the chess var on to the screen  
//so the screen can be update every turn  
private void drawPiecesFromLogic(Canvas canvas) {  
    Piece[][] board = chessGame._gameRounds.getBoard();  
    if (board == null) return;  
    for (int row = 0; row < 8; row++) {  
        for (int col = 0; col < 8; col++) {  
            Piece piece = board[row][col];  
            if (piece != null && piece.getColor() != Constants.EMPTY_SLOT) {  
                String colorPrefix = (piece.getColor() == Constants.WHITE_PIECE) ? "w" : "b";  
                String typeKey = String.valueOf(piece.getType()).toLowerCase();  
                String key = colorPrefix + typeKey;  
  
                Bitmap bitmap = pieceBitmaps.get(key);  
                if (bitmap != null) {  
                    RectF dst = new RectF(col * squareSize, row * squareSize, (col + 1) * squareSize, (row + 1) * squareSize);  
                    canvas.drawBitmap(bitmap, null, dst, paint);  
                }  
            }  
        }  
    }  
}
```

```
//This func draw the status of the game  
private void drawStatusOverlay(Canvas canvas) {  
    if (chessGame.getMove() == '0') {  
        textPaint.setColor(Color.RED);  
        textPaint.setTextSize(squareSize);  
        textPaint.setTextAlign(Paint.Align.CENTER);  
    }  
}
```




Regular Chess

```
canvas.drawText("CHECKMATE", getWidth() / 2f, getHeight() / 2f, textPaint);
    }
}
```

```
//this func handle the touch when a player moves one piece in a square into another
```

```
@Override
public boolean onTouchEvent(MotionEvent event) {
    if (event.getAction() != MotionEvent.ACTION_DOWN) return true;
    try {
        // Horizontal offset calculation for centering
        float boardLeftOffset = (getWidth() - (squareSize * 8)) / 2f;

        int col = (int) ((event.getX() - boardLeftOffset) / squareSize);
        int row = (int) ((event.getY() - boardTopOffset) / squareSize);

        if (col < 0 || col >= 8 || row < 0 || row >= 8) return true;

        if (firstClick == null) {
            Piece[][] board = chessGame._gameRounds.getBoard();
            Piece clickedPiece = board[row][col];
            if (clickedPiece != null && clickedPiece.getColor() == chessGame.getMove()) {
                firstClick = new Point(row, col);
                invalidate();
            }
        } else {
            if (firstClick.getRow() == row && firstClick.getCol() == col) {
                firstClick = null;
            } else {
                chessGame._gameRounds.setMovePoints(firstClick, new Point(row, col));
                chessGame.game(getContext());
                firstClick = null;
                if (chessGame.getMove() != '0') {
                    startTurnTimer();
                } else {
                    stopTimer();
                }
                invalidate();
            }
        }
    } catch (Exception e) { e.printStackTrace(); }
    return true;
}
```

בסיס נתונים

מחלקת DatabaseManager

תפקיד המחלקה:



Regular Chess

מחלקה סטטית שאחראית על כל התקשורת עם Firebase Auth ו-Firebase Firestore. היא מספקת פעולות לטעינה ושמירה של סטטיסטיקות משתמש, טעינת פרופיל, עדכון שם משתמש ומחיקת חשבון

תכונות המחלקה:

שם הפעולה	הסבר
<code>deleteUserAccount(context, callback)</code>	מוחקת את המסמך של המשתמש מ-Firebase וואז מוחקת את חשבון ה-Auth שלו
<code>loadUserProfile(context, uid, callback)</code>	טוענת את שם המשתמש מ-Firebase ומעדכנת את <code>GlobalStat</code>
<code>loadStatsFromFirebase(context, callback)</code>	טוענת את כל הסטטיסטיקות של המשתמש מ-Firebase לזיכרון הגלובלי
<code>saveStatsToFirebase(context)</code>	שומרת את כל הסטטיסטיקות מהזיכרון הגלובלי חזרה ל-Firebase
<code>updateUsername(uid, newUsername)</code>	מעדכנת רק את שדה ה-username במסמך של המשתמש ב-Firebase

```
/*
The func delete the user account from the firestore thats include the stats
table
input: context -> which class he came from
output: none ( delete the account )
*/
public static void deleteUserAccount(Context context, final
DeleteAccountCallback callback) {
    com.google.firebase.auth.FirebaseAuth user =
    com.google.firebase.auth.FirebaseAuth.getInstance().getCurrentUser();

    if (user == null) {
        if (callback != null) callback.onAccountDeleted(false, "No user is
        currently signed in.");
        return;
    }

    String uid = user.getId();

    //wipe out their row/document from the Firestore collection completely
    com.google.firebase.firestore.FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .delete()
        .addOnSuccessListener(aVoid -> {
            android.util.Log.d("DB_MANAGER", "Firestore statistics
            document deleted successfully.");
        });

    // delete the identity credentials from Auth
    user.delete()
        .addOnSuccessListener(aVoidAuth -> {
            android.util.Log.d("DB_MANAGER", "User Auth
            credentials purged successfully.");
        });

    // Clear local memory space cache
    representation
    GlobalStat globalStat = (GlobalStat)
    context.getApplicationContext();
    globalStat.userStats = new UserStatistics();

    if (callback != null)
        callback.onAccountDeleted(true, null);
}
```



Regular Chess

```
        })
        .addOnFailureListener(e -> {
            android.util.Log.e("DB_MANAGER", "Auth account
                purge failed: " + e.getMessage());
            // This happens if the user logged in a long
            time ago and needs to re-authenticate
            if (callback != null)
                callback.onAccountDeleted(false, "Authentication timeout. Please log out
                    and back in to delete your account.");
        });
    })
    .addOnFailureListener(e -> {
        android.util.Log.e("DB_MANAGER", "Failed to clear Firestore
            statistics: " + e.getMessage());
        if (callback != null) callback.onAccountDeleted(false,
            e.getMessage());
    });
}
```

```
/*
The func load the username from the stats table by using the uid of the
client.
input: uid -> the user id that the firestore create for each client
output: none ( update the username in the globalstat )
*/
public static void loadUserProfile(Context context, String uid, final
    UserProfileCallback callback) {
    if (uid == null || uid.trim().isEmpty()) {
        if (callback != null) callback.onProfileLoaded(false);
        return;
    }

    GlobalStat globalStat = (GlobalStat) context.getApplicationContext();

    FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                // Extract just the username field explicitly
                String cloudUsername =
                    documentSnapshot.getString("username");
                if (cloudUsername != null &&
                    !cloudUsername.trim().isEmpty()) {
                    globalStat.userStats.username = cloudUsername;
                }
            }
            // Signal the UI thread that processing has finished safely
            if (callback != null) callback.onProfileLoaded(true);
        })
        .addOnFailureListener(e -> {
            android.util.Log.e("DB_MANAGER", "Failed to load user
                profile: " + e.getMessage());
            // Fire fallback signal anyway so login screen flow isn't
            left hanging frozen
            if (callback != null) callback.onProfileLoaded(false);
        });
}
```



Regular Chess

```
The func load all the stats from the stats table by using the uid of the client.
input: uid -> the user id that the firestore create for each client
output: none ( update the username in the globalstat )
*/
public static void loadStatsFromFirebase(Context context, final
                                         DataStatusCallback callback) {
    GlobalStat globalStat = (GlobalStat) context.getApplicationContext();

    if (globalStat.userStats == null || globalStat.userStats.Uid == null
        || globalStat.userStats.Uid.trim().isEmpty()) {
        android.util.Log.e("FIREBASE_READ", "Cannot load stats: UID is
                                                                    null.");
        return;
    }

    String uid = globalStat.userStats.Uid;

    FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .get()
        .addOnSuccessListener(snapshot -> {
            if (snapshot.exists()) {
                UserStatistics stats =
                    snapshot.toObject(UserStatistics.class);
                if (stats != null) {
                    // Keep the UID and username that we already have
                    // in memory
                    stats.Uid = globalStat.userStats.Uid;
                    stats.username = globalStat.userStats.username;

                    // Write everything back into global memory
                    globalStat.userStats = stats;
                    globalStat.statsLoadedFromFirebase = true;
                } else {
                    // New user - no document yet, that's fine
                    globalStat.statsLoadedFromFirebase = true;
                }

                if (callback != null)
                    callback.onDataLoaded(globalStat.userStats);
            }
            .addOnFailureListener(e -> {
                android.util.Log.e("FIREBASE_READ", "Read failed: " +
                                    e.getMessage());
                // Still fire callback so UI doesn't stay stuck on "..."
                if (callback != null)
                    callback.onDataLoaded(globalStat.userStats);
            });
        });
}
```

```
/*
The func saves the stats from the globalstat in the stats table in the
fireStore
input: uid -> the user id that the firestore create for each client
output: none ( update the stats in firestore )
*/
public static void saveStatsToFirebase(Context context) {
```



Regular Chess

```
GlobalStat globalStat = (GlobalStat) context.getApplicationContext();

if (globalStat.userStats == null || globalStat.userStats.Uid == null
    || globalStat.userStats.Uid.trim().isEmpty()) {
    android.util.Log.e("FIREBASE_WRITE", "Cannot save stats: UID is
        null.");
    return;
}

String uid = globalStat.userStats.Uid;

FirebaseFirestore.getInstance()
    .collection("Statistics")
    .document(uid)
    .set(globalStat.userStats)
    .addOnSuccessListener(aVoid ->
        android.util.Log.d("FIREBASE_WRITE", "Stats saved
            successfully for UID: " + uid))
    .addOnFailureListener(e ->
        android.util.Log.e("FIREBASE_WRITE", "Save failed: " +
            e.getMessage()));
}
```

```
/*
    The func update just the username in the stats table
    input: uid -> the user id that the firestore create for each client
    output: none ( update the username in firestore )
*/
public static void updateUsername(String uid, String newUsername) {
    if (uid == null || uid.trim().isEmpty()) return;

    FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .update("username", newUsername)
        .addOnSuccessListener(aVoid -> android.util.Log.d("DB_MANAGER",
            "Username synced"))
        .addOnFailureListener(e -> android.util.Log.e("DB_MANAGER",
            "Sync failed: " + e.getMessage()));
}
```

מחלקת GameOverDialog

תפקיד המחלקה:

Activity שמוצגת בסיום המשחק. היא מציגה הודעה מתאימה (נצחון, תיקו, או ניצחון על זמן) ומאפשרת לשחקן לחזור ללובי תוך שמירת הסטטיסטיקות ל-Firebase.

תכונות המחלקה:

שם התכונה	הסבר
dialogMessage (TextView)	תצוגת הטקסט שמציגה את הודעת סיום המשחק



Regular Chess

כפתור שמחזיר את השחקן ללובי	btnGoToLobby (Button)
-----------------------------	-----------------------

שם הפעולה	הסבר
onCreate(savedInstanceState)	שמאתחלת את הפרמטרים וקוראת מה Intent ומציגה את הודעת הסיום המתאימה

```

/*
The func is a constructor the find out which win or lose or stalemate
happened and show it on the activity by getIntent
*/
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.dialog_game_over);
    setFinishOnTouchOutside(false);

    TextView dialogMessage = findViewById(R.id.dialogMessage);
    Button btnGoToLobby = findViewById(R.id.btnGoToLobby);

    boolean isStalemate = getIntent().getBooleanExtra("IS_STALEMATE",
                                                    false);
    boolean whiteWon = getIntent().getBooleanExtra("WHITE_WON", true);

    // Extract the custom finish style parameter context
    String winReason = getIntent().getStringExtra("WIN_REASON");

    if (isStalemate) {
        dialogMessage.setText("Stalemate! The game ends in a Draw. ♠️");
    } else if ("TIME".equals(winReason)) {
        //Time win
        dialogMessage.setText(whiteWon
            ? "Time's up! WHITE wins on time! ⏰♠️"
            : "Time's up! BLACK wins on time! ⏰♟️");
    } else {
        // checkmate win
        dialogMessage.setText(whiteWon
            ? "Congratulations! WHITE wins by Checkmate! ♠️"
            : "Game Over! BLACK wins by Checkmate! ♟️");
    }

    btnGoToLobby.setOnClickListener(v -> {
        DatabaseManager.saveStatsToFirebase(this);

        Intent intent = new Intent(GameOverDialog.this, lobby.class);
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP |
            Intent.FLAG_ACTIVITY_NEW_TASK);
        intent.putExtra("FROM_SCREEN", "GAME");
        startActivity(intent);
        finish();
    });
}

```



Regular Chess

מחלקת lobby

תפקיד המחלקה:

המסך הראשי של האפליקציה לאחר ההתחברות. מפעיל מוזיקת רקע, מנהל התראות Alarm, מאזין לסוללה חלשה, ומנווט לכל מסכי האפליקציה (משחק, הגדרות, סטטיסטיקות, הוראות, התנתקות).

תכונות המחלקה:

שם התכונה	הסבר
ImageButton settingbtn / playbtn / instructbtn / statbtn / logoutbtn	כפתורי ניווט לכל מסכי האפליקציה
TextView usernameDisplayTv	מציג את שם המשתמש הנוכחי
String fromScreen	מזהה מאיזה מסך הגיעו ללובי (LOGIN / GAME)
BroadcastReceiver batteryLowReceiver	מאזין לאירוע סוללה חלשה ומציג אזהרה

שם הפעולה	הסבר
onCreate(savedInstanceState)	מפעילה מוזיקה, יוצרת ערוץ, מאתחלת את הלובי ומפעילה התראות
createNotificationChannel()	יוצרת ערוץ Notification נדרש לאנדרואיד +8
requestNotificationPermission()	מבקשת הרשאת הצגת התראות באנדרואיד +13
startRepeatingAlarm()	מתזמנת Alarm מדויק שיופעל כל 10 שניות
onClick(v)	מנווטת לדף המתאים בהתאם לכפתור שנלחץ
setUsernameBox()	קוראת את שם המשתמש מGlobalStat ומציגה אותו
onResume()	מרשמת את ה BroadcastReceiver לאירוע סוללה חלשה
onPause()	מסירה את ה BroadcastReceiver כדי למנוע memory leak

```
//fields
ImageButton settingbtn;
ImageButton playbtn;
ImageButton instructbtn;
ImageButton statbtn;
ImageButton logoutbtn;
String fromScreen;
TextView usernameDisplayTv;
private BroadcastReceiver batteryLowReceiver;
```

```
/*
The func construct the page start the background music start all the alarms
*/
@SuppressLint("MissingInflatedId")
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
```

Regular Chess

```

setContentView(R.layout.lobby);
fromScreen = getIntent().getStringExtra("FROM_SCREEN");

Intent musicIntent = new Intent(this, BackgroundMusicService.class);
startService(musicIntent);

batteryLowReceiver = new BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        if (Intent.ACTION_BATTERY_LOW.equals(intent.getAction())) {

            // Display a warning alert to the player
            Toast.makeText(context,
                "⚠ Battery is low! Plug in your charger before
                starting a match.",
                Toast.LENGTH_LONG).show();
        }
    }
};

ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
    insets) -> {
        Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
            systemBars.bottom);
        return insets;
    });

//start all the notification process
createNotificationChannel();
requestNotificationPermission();
startRepeatingAlarm();

//the update or load data logic
if (fromScreen != null) {
    switch (fromScreen) {
        case "LOGIN":
            Log.d("LOBBY", "Arrived from login");
            // DO NOTHING HERE - statisticsPage handles its own loading
            break;

        case "GAME":
            Log.d("LOBBY", "UPDATE IN FIREBASE");
            DatabaseManager.saveStatsToFirebase(this); // use this
            // instead of the old method
            break;

        default:
            break;
    }
}

//finds all the buttons amd text view
usernameDisplayTv = findViewById(R.id.usernameDisplayTv);
setUsernameBox(); // set the user name box

settingbtn = findViewById(R.id.settingbtn);
settingbtn.setOnClickListener(this);

playbtn = findViewById(R.id.playbtn);
playbtn.setOnClickListener(this);

```




Regular Chess

```
instructbtn = findViewById(R.id.instructbtn);
instructbtn.setOnClickListener(this);

statbtn = findViewById(R.id.statbtn);
statbtn.setOnClickListener(this);

logoutbtn = findViewById(R.id.logoutbtn);
logoutbtn.setOnClickListener(this);
```

```
//The func creates a channel for the notification process
private void createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationChannel channel = new NotificationChannel(
            "alarm_channel", // Channel ID
            "Alarm Notifications", // Channel name (shown in
                                // settings)
            NotificationManager.IMPORTANCE_HIGH
        );
        channel.setDescription("Notifications fired by the alarm");

        NotificationManager manager =
            getSystemService(NotificationManager.class);
        if (manager != null) {
            manager.createNotificationChannel(channel);
        }
    }
}
```

```
//This func request a notification permission from the phone
private void requestNotificationPermission() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) { // Android
                                                                    13+
        if (ActivityCompat.checkSelfPermission(this,
            android.Manifest.permission.POST_NOTIFICATIONS)
            != android.content.pm.PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(
                this,
                new
                String[]{android.Manifest.permission.POST_NOTIFICATIONS},
                1001
            );
        }
    }
}
```

```
//The func check if permission is granted and if so
// he start the all alarm process of alert the client to play some games
private void startRepeatingAlarm() {
    AlarmManager alarmManager = (AlarmManager)
        getSystemService(Context.ALARM_SERVICE);
    if (alarmManager == null) return;

    Intent intent = new Intent(this, AlarmReceiver.class);
    PendingIntent pendingIntent = PendingIntent.getBroadcast(
        this,
```



Regular Chess

```
0,
    intent,
    PendingIntent.FLAG_UPDATE_CURRENT |
    PendingIntent.FLAG_IMMUTABLE
);

// check permission before scheduling exact alarm
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    if (!alarmManager.canScheduleExactAlarms()) {
        // Open settings so user can grant permission
        Intent permIntent = new
Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM);
        startActivity(permIntent);
        return;
    }
}

// setExactAndAllowWhileIdle + reschedule in receiver
long triggerAtMillis = SystemClock.elapsedRealtime() + 10_000;
alarmManager.setExactAndAllowWhileIdle(
    AlarmManager.ELAPSED_REALTIME_WAKEUP,
    triggerAtMillis,
    pendingIntent
);
}
```

```
//move to every screen by the button
@Override
public void onClick(View v) {

    if(settingbtn == v)
    {
        Intent intent = new Intent(lobby.this,settingPage.class);
        startActivity(intent);
    }

    if(playbtn == v)
    {
        Intent intent = new Intent(lobby.this,playPage.class);
        startActivity(intent);
    }

    if(instructbtn == v)
    {
        Intent intent = new Intent(lobby.this,instructionPage.class);
        startActivity(intent);
    }

    if(statbtn == v)
    {
        Intent intent = new Intent(lobby.this,statisticsPage.class);
        startActivity(intent);
    }

    //log out process starting
    //stop the music than wipe the cache from the global stat and in last
    //move to the log lobby
    if(logoutbtn == v) {
        Intent musicIntent = new Intent(lobby.this,
```



Regular Chess

```
BackgroundMusicService.class);
stopService(musicIntent);

BackgroundMusicService.isMusicMuted = false;

GlobalStat globalStat = (GlobalStat) getApplicationContext();
if (globalStat.userStats != null) {
globalStat.userStats = new UserStatistics(); // Wipes current
cache
}

Intent intent = new Intent(lobby.this, log_lobby.class);
startActivity(intent);
finish();
}
}
```

```
//set the username on the textView
private void setUsernameBox()
{
GlobalStat globalStat = (GlobalStat) getApplicationContext();
usernameDisplayTv.setText(globalStat.userStats.username);
}
```

```
@Override
protected void onResume() {
super.onResume();
// listening for the low-battery broadcast when the user enters the
lobby
IntentFilter filter = new IntentFilter(Intent.ACTION_BATTERY_LOW);
registerReceiver(batteryLowReceiver, filter);
}

@Override
protected void onPause() {
super.onPause();
// Stop listening when the user navigates away from the lobby
// prevent memory leaks!
if (batteryLowReceiver != null) {
unregisterReceiver(batteryLowReceiver);
}
}
```

מחלקת loginPage

תפקיד המחלקה:



Regular Chess

מסך ההתחברות לאפליקציה. מאפשר למשתמש להזין אימייל וסיסמה, מבצע אימות מול Firebase Auth, טוען את פרטי הפרופיל מ-Firebase ומנווט ללובי בהצלחה.

תכונות המחלקה:

שם התכונה	הסבר
ImageButton backbtn	כפתור חזרה לlog_lobby
EditText emailtv / passwordtv	שדות קלט לאימייל ולסיסמה
ImageButton loginbtn	כפתור ביצוע ההתחברות
FirebaseAuth auth	אובייקט Firebase לאימות משתמשים

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את Activity ומחברת את Widgets
onClick(v)	מטפלת בלחיצות חזרה אחורה או ביצוע Login
loginClient(email, password)	מבצעת התחברות לFirebase Auth טוענת את הפרופיל ומנווטת ללובי

```
//fields
ImageButton backbtn;
EditText emailtv, passwordtv;
ImageButton loginbtn;

FirebaseAuth auth;
```

```
//constructor
@SuppressLint("MissingInflatedId")
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.login);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    //find buttons and textviews
    backbtn = findViewById(R.id.backbtn);
    backbtn.setOnClickListener(this);

    emailtv = findViewById(R.id.emailtv);
    passwordtv = findViewById(R.id.passwordtv);
    loginbtn = findViewById(R.id.loginbtn);

    auth = FirebaseAuth.getInstance();

    loginbtn.setOnClickListener(this);
```

Regular Chess

```

/*
The func see which button pressed and move to the right activity
*/
@Override
public void onClick(View v) {
    if(backbtn == v)
    {
        //letting the lobby know where does the client come from
        Intent intent = new Intent(loginPage.this, log_lobby.class);
        intent.putExtra("FROM_SCREEN", "LOGIN");
        startActivity(intent);
        finish();
    }
    if(loginbtn == v)
    {
        //try to login
        String txt_logemail = emailtv.getText().toString();
        String txt_logPw = passwordtv.getText().toString();
        loginClient(txt_logemail, txt_logPw);
    }
}

```

```

/*
The func check if the email and password are already register in the auth
table
if yes he will move to the lobby activity and let the lobby know the client
came from
the loginPage
input: txtLogemail -> the email the client gave
txtLogPw -> the password the client gave
output: none
*/
private void loginClient(String txtLogemail, String txtLogPw) {
    auth.signInWithEmailAndPassword(txtLogemail,
txtLogPw).addOnSuccessListener(new OnSuccessListener<AuthResult>() {
        @Override
        public void onSuccess(AuthResult authResult) {
            String uid = auth.getUid();
            if (uid == null) return;

            GlobalStat globalStat = (GlobalStat) getApplicationContext();

            if (globalStat.userStats == null) {
                globalStat.userStats = new UserStatistics();
            }

            // Bind the essential local variables first
            globalStat.userStats.Uid = uid;

            // Set a fallback username from the email prefix in case
            Firestore is empty
            globalStat.userStats.username = txtLogemail.substring(0,
txtLogemail.indexOf('@'));

            DatabaseManager.loadUserProfile(loginPage.this, uid, new
DatabaseManager.UserProfileCallback() {
                @Override
                public void onProfileLoaded(boolean success) {

```



Regular Chess

```
// This block executes dynamically after your network
// request drops
Toast.makeText(loginPage.this, "Login successful",
    Toast.LENGTH_SHORT).show();

Intent intent = new Intent(loginPage.this,
    lobby.class);
intent.putExtra("FROM_SCREEN", "LOGIN");
startActivity(intent);
finish();
    }
    }
    }
    }
```

מחלקת settingPage

תפקיד המחלקה:

מסך ההגדרות של האפליקציה. מאפשר למשתמש לשנות שם משתמש, להשתיק מוזיקה, לכוון עצמת שמע באמצעות Slider ולמחוק את חשבוננו לצמיתות.



Regular Chess

תכונות המחלקה:

שם התכונה	הסבר
ImageButton backbtn	כפתור חזרה ללובי תוך שמירת שם המשתמש
Switch switchmusiccontrol	מתג להפעלה / השתקה של המוזיקה
EditText editusername	שדה עריכה לשינוי שם המשתמש
AppCompatActivity btn_delete_account	כפתור למחיקת החשבון לצמיתות
SeekBar seekbarvolume	פס גלילה לכיוון עצמת המוזיקה

שם הפעולה	הסבר
onCreate(savedInstanceState)	מסנכרנת מצב, מאתחלת את כל הרכיבים מוזיקה ועצמת שמע מהזיכרון הגלובלי
onClick(v)	מטפלת בחזרה ללובי (שמירת שם משתמש) ובמחיקת חשבון

```
//fields
ImageButton backbtn;
Switch switchmusiccontrol;
EditText editusername;
AppCompatActivity btn_delete_account;
SeekBar seekbarvolume;
```

```
//this func construct and check if the seekbar is changing
// if it does he will increase or decrease the volume of the background music
@SuppressLint({"MissingInflatedId", "ClickableViewAccessibility"})
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.settingpage);

    seekbarvolume = findViewById(R.id.seekbarvolume);

    int initialProgress = (int) (BackgroundMusicService.currentVolume * 100);
    seekbarvolume.setProgress(initialProgress);

    seekbarvolume.setOnSeekBarChangeListener(new
        SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar seekBar, int progress,
                boolean fromUser) {
                // Map the integer progression scale back down to fractional
                // volume ranges
                float scalarVolume = progress / 100f;

                // Ship the updated calculation directly down to your
                // background Service pipeline
                Intent volumeIntent = new Intent(settingPage.this,
                    BackgroundMusicService.class);

                volumeIntent.setAction(BackgroundMusicService.ACTION_SET_VOLUME);

                volumeIntent.putExtra(BackgroundMusicService.EXTRA_VOLUME_VALUE,
                    scalarVolume);
                startService(volumeIntent);
```

Regular Chess

```

    }

    @Override
    public void onStartTrackingTouch(SeekBar seekBar) {}

    @Override
    public void onStopTrackingTouch(SeekBar seekBar) {}
    });

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
        insets) -> {
        Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
            systemBars.bottom);
        return insets;
    });

    // Initialize widgets
    switchmusiccontrol = findViewById(R.id.switchmusiccontrol);
    editusername = findViewById(R.id.editusername);

    // Initialize Button and Bind Listeners
    btn_delete_account = findViewById(R.id.btn_delete_account);
    btn_delete_account.setOnClickListener(this);

    // Click Compression Effect (scale: 0.92)
    btn_delete_account.setOnTouchListener(new View.OnTouchListener() {
        @Override
        public boolean onTouch(View v, MotionEvent event) {
            if (event.getAction() == MotionEvent.ACTION_DOWN) {

                v.animate().scaleX(0.92f).scaleY(0.92f).setDuration(150).start();
            } else if (event.getAction() == MotionEvent.ACTION_UP ||
                event.getAction() == MotionEvent.ACTION_CANCEL) {

                v.animate().scaleX(1.0f).scaleY(1.0f).setDuration(600).start();
                if (event.getAction() == MotionEvent.ACTION_UP) {
                    v.performClick();
                }
            }
            return false;
        }
    });

    backbtn = findViewById(R.id.backbtn);
    backbtn.setOnClickListener(this);

    // Populate local cache inputs
    GlobalStat globalStat = (GlobalStat) getApplicationContext();
    if (globalStat.userStats != null && globalStat.userStats.username !=
        null) {
        editusername.setText(globalStat.userStats.username);
    }

    // Music toggle engine system state sync
    switchmusiccontrol.setChecked(!BackgroundMusicService.isMusicMuted);
    switchmusiccontrol.setOnCheckedChangeListener(new
        CompoundButton.OnCheckedChangeListener() {
            @Override

```




Regular Chess

```
public void onCheckedChanged(CompoundButton buttonView, boolean
                                isChecked) {
    Intent musicIntent = new Intent(settingPage.this,
        BackgroundMusicService.class);
    if (isChecked) {
        BackgroundMusicService.isMusicMuted = false;
        startService(musicIntent);
    } else {
        BackgroundMusicService.isMusicMuted = true;
        stopService(musicIntent);
    }
}
});
}
```

```
//this func move and act by the logic of the button that got pressed
@Override
public void onClick(View v) {
    if (backbtn == v) {
        String newUsername = editusername.getText().toString().trim();
        if (!newUsername.isEmpty()) {
            GlobalStat globalStat = (GlobalStat) getApplicationContext();
            globalStat.userStats.username = newUsername;
            DatabaseManager.updateUsername(globalStat.userStats.Uid,
                newUsername);
        } else {
            Toast.makeText(this, "Username cannot be empty!",
                Toast.LENGTH_SHORT).show();
            return;
        }
    }

    Intent intent = new Intent(settingPage.this, lobby.class);
    startActivity(intent);
    finish();
}

if (btn_delete_account == v) {
    // Stop background audio before destroying session states
    Intent musicIntent = new Intent(settingPage.this,
        BackgroundMusicService.class);
    stopService(musicIntent);
    BackgroundMusicService.isMusicMuted = false;

    // deletes data from Firestore & Auth table inside the
    DatabaseManager
    DatabaseManager.deleteUserAccount(this, new
        DatabaseManager.DeleteAccountCallback() {
            @Override
            public void onAccountDeleted(boolean success, String
                errorMessage) {
                if (success) {
                    Toast.makeText(settingPage.this, "Account deleted
                        permanently.", Toast.LENGTH_LONG).show();

                    // Route to log_lobby initialization lane
                    Intent intent = new Intent(settingPage.this,
                        log_lobby.class);
                    // Wipe backstack completely so hardware arrows cannot
                    re-enter back into active states
                    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
```



Regular Chess

```
Intent.FLAG_ACTIVITY_CLEAR_TASK);
startActivity(intent);
finish();
} else {
Toast.makeText(settingPage.this, "Error: " +
errorMessage, Toast.LENGTH_LONG).show();
}
}
});
}
}
```

מחלקת signinPage

תפקיד המחלקה:

מסך הרישום של משתמשים חדשים. מאפשר הזנת אימייל וסיסמה, מבצע ולידציה בסיסית, ויוצר חשבון חדש ב-Firebase Authentication.

תכונות המחלקה:

שם התכונה	הסבר
ImageButton backbtn	כפתור חזרה לlog_lobby
EditText emailtv / passwordtv	שדות קלט לאימייל ולסיסמה
ImageButton signbtn	כפתור ביצוע הרישום
FirebaseAuth auth	Firebase אוברייקט לניהול הרישום
Context context	הקשר הActivity
	לשימוש בהודעות Toast

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את הActivity ומחברת את הכפתורים
onClick(v)	מטפלת בחזרה אחורה ובלחיצת רישום תוך ולידציה של הקלט
registerUser(email, password)	יוצרת חשבון חדש ב-Firebase Auth ומנווטת לlog_lobby בהצלחה

```
//fields
ImageButton backbtn;

//firebase variables
EditText emailtv, passwordtv;
ImageButton signbtn;
Context context;

FirebaseAuth auth;
```

```
//constructor
@SuppressLint("MissingInflatedId")
@Override
protected void onCreate(Bundle savedInstanceState) {
```



Regular Chess

```
super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.signin);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.sign), (v,
        insets) -> {
        Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
            systemBars.bottom);
        return insets;
    });

    //bind buttons and textviews
    backbtn = findViewById(R.id.backbtn);
    backbtn.setOnClickListener(this);

    emailtv = findViewById(R.id.emailtv);
    passwordtv = findViewById(R.id.passwordtv);
    signbtn = findViewById(R.id.signbtn);

    context = signinPage.this;
    auth = FirebaseAuth.getInstance();

    signbtn.setOnClickListener(this);
```

```
//The func move to the desire activity and act by the button that got
pressed
@Override
public void onClick(View v) {

    if(backbtn == v)
    {
        Intent intent = new Intent(signinPage.this, log_lobby.class);
        startActivity(intent);
        finish();
    }

    //make sure the strings are not empty
    if(signbtn == v)
    {
        String txt_email = emailtv.getText().toString();
        String txt_password = passwordtv.getText().toString();
        if(TextUtils.isEmpty(txt_email) || TextUtils.isEmpty(txt_password))
        {
            Toast.makeText(context, "fields must not be
                empty", Toast.LENGTH_SHORT).show();
        }
        else if(txt_password.length() < 6){
            Toast.makeText(context, "password is to
                short", Toast.LENGTH_SHORT).show();
        }
        else {
            //try to register
            registerUser(txt_email, txt_password);
        }
    }
}
```



Regular Chess

```
/*
    The func try to register in the auth table
    input: email -> client desire email
    password -> client desire password
    output: or a new row in the auth table or toast failed
*/
private void registerUser(String email, String password)
{
    auth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener(signinPage.this, new
            OnCompleteListener<AuthResult>() {
                @Override
                public void onComplete(@NonNull Task<AuthResult> task) {
                    if (task.isSuccessful())
                    {
                        Toast.makeText(context, "success",
                            Toast.LENGTH_SHORT).show();
                        startActivity(new Intent(signinPage.this,
                            log_lobby.class));
                    }
                    else {
                        Toast.makeText(context, "failed",
                            Toast.LENGTH_SHORT).show();
                    }
                }
            });
}
```

מחלקת statisticsPage

תפקיד המחלקה:

מסך הסטטיסטיקות שמציג למשתמש את נתוני הביצועים שלו: ניצחונות, הפסדים, תיקוים, משחקים שהתנגנו זמן משחק. טוען נתונים מ-Firebase ומציג אותם בזמן אמת.

תכונות המחלקה:

שם התכונה	הסבר
TextView winstv / losestv / stalematestv / timeplayedtv / gamesplayedtv	תצוגות טקסט לכל אחד מנתוני הסטטיסטיקה
ImageButton backbtn	כפתור חזרה ללובי

שם הפעולה	הסבר
onCreate(savedInstanceState)	מאתחלת את Activityn מציגה ערכי cache, אם קיימים, וטוענת נתונים עדכניים מ-Firebase
showLoadingState()	מציגה '...' בכל שדה בזמן טעינת הנתונים



Regular Chess

מעדכנת את כל תצוגות הטקסט עם הנתונים שהתקבלו	updateUI(stats)
כפתור חזרה – מנווטת ללובי	onClick(v)

```
//fields
TextView winstv, losestv, stalematestv, timeplayedtv, gamesplayedtv;
ImageButton backbtn;
```

```
//constructor
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.statistics);

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars =
            insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
            systemBars.bottom);
        return insets;
    });

    //bind all the text views and button

    winstv = findViewById(R.id.winstv);
    losestv = findViewById(R.id.losestv);
    stalematestv = findViewById(R.id.stalematestv);
    timeplayedtv = findViewById(R.id.timeplayedtv);
    gamesplayedtv = findViewById(R.id.gamesplayedtv);
    backbtn = findViewById(R.id.backbtn);
    backbtn.setOnClickListener(this);

    GlobalStat globalStat = (GlobalStat) getApplicationContext();

    // If we already loaded from Firebase before
    // show the cached values immediately – no "... flicker
    if (globalStat.statsLoadedFromFirebase && globalStat.userStats != null)
    {
        updateUI(globalStat.userStats);
    } else {
        showLoadingState();
    }

    // Always fetch fresh data from Firebase to stay up to date
    DatabaseManager.loadStatsFromFirebase(this, new
        DatabaseManager.DataStatusCallback() {
            @Override
            public void onDataLoaded(UserStatistics stats) {
                updateUI(stats);
            }
        });
}
```



Regular Chess

```
/*  
The func shows lodaing stats if it hasent got anything from the firebase  
input: none  
output: none  
*/  
private void showLoadingState() {  
    winstv.setText("...");  
    losestv.setText("...");  
    stalematestv.setText("...");  
    gamesplayedtv.setText("...");  
    timeplayedtv.setText("...");  
}
```

```
/*  
The func update all the text views by the stats from the globalstat  
input: stats -> all of the user stats  
output: none  
*/  
@SuppressWarnings("SetTextI18n")  
private void updateUI(UserStatistics stats) {  
    if (stats == null) return;  
    runOnUiThread(() -> {  
        winstv.setText(String.valueOf(stats.wins));  
        losestv.setText(String.valueOf(stats.losses));  
        stalematestv.setText(String.valueOf(stats.stalemates));  
        gamesplayedtv.setText(String.valueOf(stats.gamesPlayed));  
        timeplayedtv.setText(stats.timespent + " min");  
    });  
}
```

```
//the func goes back to the lobby page if the backbtn was pressed  
@Override  
public void onClick(View v) {  
    if (backbtn == v) {  
        Intent intent = new Intent(statisticsPage.this, lobby.class);  
        startActivity(intent);  
        finish();  
    }  
}
```



Regular Chess

טבלת סטטיסטיקות:

תפקיד הטבלה - באפליקציה ישנה דף סטטיסטיקות שמיועדת לשמור את כל עבר הניסיון של השחקן באפליקציה. הטבלה שומרת את כל הנתונים האלו כגון: שם משתמש, מספר ניצחונות, מספר הפסדים ועוד..

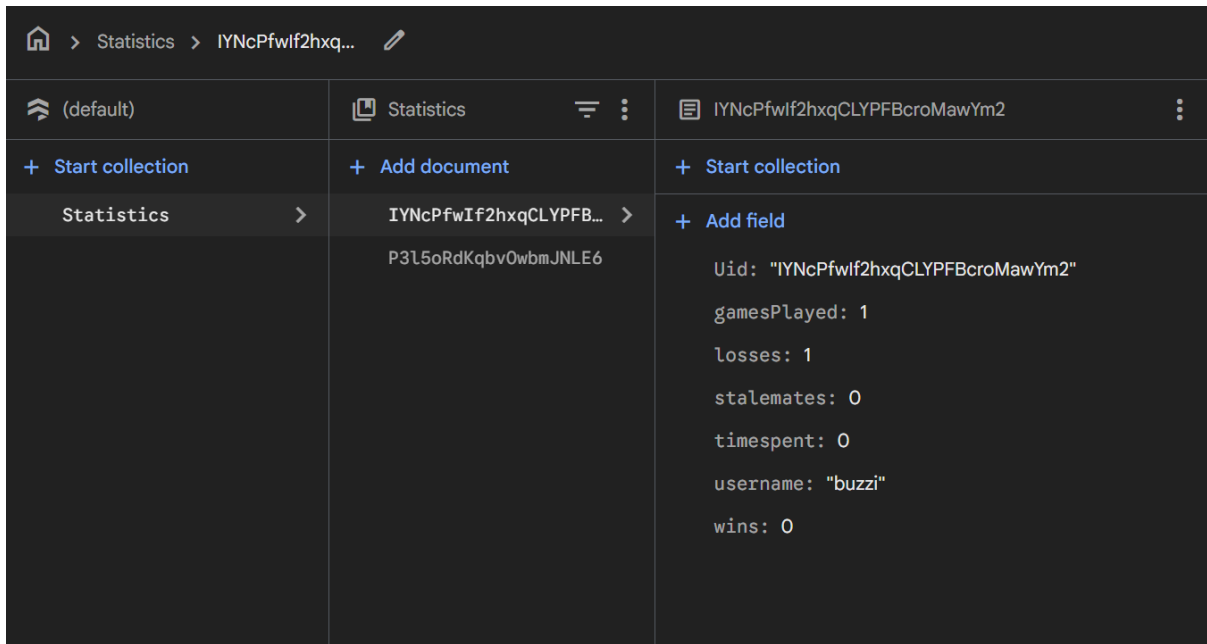
שדות הטבלה:

שם השדה	הסבר
String User Uid	מחרוזת הזהות של המשתמש שיחודית לכל משתמש ונוצרה על ידי Firestore
Long gamesPlayed	מספר כל המשחקים השלמים ששיחק הלקוח לא כולל המשחקים שהוא פרש מהם.
Long losses	מספר כל ההפסדים שהפסיד הלקוח.
Long stalemates	מספר כל מצבי התיקו שהגיע הלקוח אליהם.
Long timespent	הזמן בדקות ששיחק הלקוח אי פעם במשחק באפליקציה.
String username	מחרוזת שם הלקוח שמאותחל כהתחלה של האימייל וזמין לשינוי בכל עת השימוש באפליקציה
Long wins	מספר כל הניצחונות שניצח הלקוח



Regular Chess

תמונה מתוך Firebase:





Regular Chess

טבלת אותנטיקציה:

תפקיד הטבלה - טבלת האותנטיקציה ב-Firebase Authentication (Authentication) אחראית על ניהול תהליך ההזדהות והאימות של המשתמשים בתוך האפליקציה. היא מספקת פתרון מאובטח ונוח לניהול הרשאות גישה למשתמשים, ומספקת מספר אפשרויות להזדהות לפי הצורך של המערכת.

תפקידי טבלת האותנטיקציה הם: הרשמה, התחברות.

שדות הטבלה:

שם השדה	הסבר
String User Uid	מחרוזת הזדהות של המשתמש שיחודית לכל משתמש ונוצרה על ידי Firestore
String created	מחרוזת השומרת את התאריך שבו נוצר המשתמש.
String SignedIn	מחרוזת השומרת את תאריך ההתחברות, לאחרונה של המשתמש.
String identifier	מחרוזת השומרת – מייל של משתמש במערכת.

תמונה מתוך Firebase:

Search by email address, phone number, or user UID					Add user		
Identifier ↓	Providers	Created ↓	Signed In	User UID			
rrr@gmail.com		Jan 30, 2026	May 22, 2026	IYNcPfwf2hxqCLYPFBcroMa...			
dddd@dd.com		Dec 12, 2025	Dec 12, 2025	iDCMwIXRTaZyPuj5S22RIwnF...			
dddd@ddd.com		Dec 12, 2025	Dec 12, 2025	aKMaIQ01C2bl1ohnpzmQY3...			
user@gmail.com		Dec 12, 2025	Dec 12, 2025	wPU1XakK3ATy5jCZsjSD1CBH...			
Rows per page: 50					1 – 4 of 4	<	>



Regular Chess

באפליקציה שלי אני מייצג את המידע של הסטטיסטיקות מתוך FireBasen בעזרת מבנים שאני בנית.
להלן המבנים:

שם המבנה	הסבר
UserStatistics userstats	מבנה זה מייצג את כל הערכים שנמצאים בטבלת הסטטיסטיקות
GlobalStat globalStat	מבנה זה מכיל את משתנה הסטטיסטיקות של הלקוח ומשתנה בוליאני של האם הסטטיסטיקות של הלקוח עלו לFireBase המשתנה הזה הוא משתנה גלובלי ונותן את האפשרות לשנות אותו בכל מקום בקוד

שם הפעולה	מחלקה	הסבר	סוג הפעולה
private void registerUser(String email, String password)	signinPage	פעולה שמוסיפה משתמש חדש בטבלת אותנטיקציה בFirestore ואם כן היא שולחת את המשתמש ללובי של התחברות כדי לבצע חיבור	הוספה
private void loginClient(String txtLogemail, String txtLogPw)	loginPage	פעולה שבדוקת אם המשתמש קיים בטבלת אותנטיקציה בFirestore ואם כן הוא מעדכן את הUID ולפי זה שולף את שאר הנתונים ומעביר את הלקוח לחלון של הלובי של המשחק	קריאה
public static void deleteUserAccount(Context context, final DeleteAccountCallback callback)	DatabaseManager	פעולה שמוחקת את המשתמש המחובר לפי הUID שלו היא מוחקת גם את טבלת הסטטיסטיקות שלו וגם את טבלת האותנטיקציה שלו	עדכון
public static void loadUserProfile(Context context, String uid, final UserProfileCallback callback)	DatabaseManager	פעולה שמעלה את שם הלקוח מטבלת הסטטיסטיקות למשתנה הגלובלי לפי הUID של הלקוח	קריאה



Regular Chess

קריאה	פעולה שמעלה את את כל הנתונים של הלקוח מטבלת הסטטיסטיקות למשתנה הגלובלי לפי ה UID של הלקוח	DatabaseManager	public static void loadStatsFromFirebase(Context context, final DataStatusCallback callback)
עדכון	פעולה שמעדכנת את הנתונים בטבלת הסטטיסטיקות ב Firestore לפי המשתנה הגלובלי	DatabaseManager	public static void saveStatsToFirebase(Context context)
עדכון	פעולה שמעדכנת את השם משתמש של הלקוח בטבלת הסטטיסטיקות ב Firestore לפי המשתנה הגלובלי	DatabaseManager	public static void updateUsername(String uid, String newUsername)



Regular Chess

מדריך למשתמש

גרסאות עליהם נבדק הפרויקט:

גרסת מערכת -> baklava, android 16.0

סוג האמולטור -> Medium Phone API 36.0

שפת תכנות -> Java

תיאור אופן הפעולה של הפרויקט

איך משתמשים?

לקוח רגיל יצטרך להירשם או להתחבר לחשבון קיים באמצעות אימייל וסיסמה.

אחרי התחברות למשחק תעמוד מול הלקוח כמה אפשריות.

-לגשת אל ההגדרות ולכבות/להדליק את מוזיקת הרקע, לשלוט בווליום של המוזיקה, שינוי שם המשתמש ומחיקת החשבון לצמיתות.

-לגשת אל הסטטיסטיקות שם אפשר להבחין בכל הניסיון שהלקוח צבר באפליקציה.

-לגשת אל ההוראות שם אפשר לקרוא ולהבין את חוקי המשחק

-להתחיל לשחק במשחק מול חברים מאותו טלפון

בחלון המשחק רואים לוח עם כל כלי המשחק ושני שעונים שסופרים לאחור מחמש דקות. התור הראשון הוא של הכלים הלבנים. כדי להזיז כלי הלקוח יצטרך ללחוץ על הכלי הרצוי שהוא מעוניין להזיז, ולחיצה שנייה לכיוון המיקום שהוא רוצה להזיז את הכלי הזה.

המשחק יכול להיגמר בכמה דרכים :

-הלקוח לוחץ על כפתור היציאה ויוצא מהמשחק (לא נחשב כמשחק בסטטיסטיקות)

-אחד מהשחקנים עושה שח-מט ובכך מנצח ומעובר אל חלון הלובי שם הוא יכול להתחיל משחק חדש

-אחד השחקנים גורם לתיקו ובכך המשחק נגמר ושוב הלקוח מועבר אל חלון הלובי

-אחד מהשחקנים בזבז את כל הזמן שלו על השעון ושוב הלקוח מועבר אל חלון הלובי

יכולות להופיע שני סוגי התראות ללקוח :

התראת משחק -> התראה בנויה באפליקציה שאומרת ללקוח להתחיל משחק!

התראת אחוז נמוך -> התראה בנויה באפליקציה שמעלה הודעת אחוז נמוך.

יכולות להופיע כמה סוגי הודעות ללקוח :

-הודעת הצלחה/כישלון התחברות במסך ההתחברות

-הודעת הצלחה/כישלון הירשמות במסך ההירשמות

-הודעת "נא למלא את כל השדות" כשהלקוח פספס בהתחברות/הירשמות

-הודעת "סיסמה קצרה מדי" בהתחברות

-הודעת "המלך נמצא בשח"

סיכום אישי/ רפלקציה

התחלתי את הפרויקט עם ביטחון גבוה, אמרתי לעצמי "אני יודע איך לבנות משחק", "אני מכיר שחמט, אני יודע שיהיה לי קל". אך לאט לאט נסחפתי עם הפרויקט. שמתי לב שיש הרבה ניואנסים קטנים שלפעמים לא שמים לב אליהם כשניגשים לפרויקט כזה גדול. המשפט ששמעתי ושליווה אותי במהלך כתיבת הקוד הוא, "מה שאתה לא כותב לא קיים". אני מאמין שכל מי שיכיר את המשפט הזה כמה שיותר מוקדם זה יצמצם לו בפער את מרחב הטעות.

לדוגמה, כשניסיתי לעבוד על כל מצבי סיום המשחק של שחמט שיערתי, שהקוד שלי בתום הטיימר עושה את כל הלוגיקה שכבר כתבתי לסיומות אחרים. אז לא בדקתי, ולקראת סיום ההגשות בדקתי איזוהו פיצ'ר של הטיימר וקלטתי את הטעות, שמה שאתה לא כותב לא קיים.

למרות כל האתגרים שניצבתי מולם הייתה לי שנת פרויקט כיפית וחוויתית. מהלמידה בסביבת העבודה של Android Studio, או כתיבת קוד מאפס בjava. יצא לי ללמוד כל כך הרבה דברים חדשים שכנראה ולא הייתי ניגש אלה רק במועד מאוחר יותר. כנראה ולא הייתי מתחיל ללמוד נושאים כאלו בגלל, שתמיד עניין אותי יותר נושא הרשתות ושפות מכונה.

אני כבר שלוש שנים בתחום הסייבר/תכנות ויצא לי לעשות פרויקטים גדולים במהלך השנים מ SSH, Bootloader, Honeypot. אני מתעסק הרבה יותר בשפות low level והתעסקות בjava ובכללי לבנות מודל שחמט משלי עם לוגיקה שאני חשבתי עליה, והוספת פיצ'רים שנראו לי לנכון. כל זה היה שונה ממה שאני בדרך כלל עושה. כל השליטה בלבנות משחק משלי נתנה לי תחושת שחרור, ואהבה למה שאני עושה. כל העבודה על המשחק הביאה לי חשק לבנות game engine משלי ולהיכנס אפילו לעולם התלת מימד ולבנות physic engine. בנוסף, יצא לי בעבר לנסות לבנות משחק שחמט. בניגוד למשחק שאני בנית הייתי צריך לכתוב רק את הלוגיקה של משחק. פרויקט זה עשיתי תחת מסגרת מגשימים עם שותף. בסופו של דבר הפרויקט הזה לא היה כזה מוצלח, והיו מלא שגיאות ובעיות בלוגיקה של השח-מט וקידום פיון.

מבחינה מקצועית תהליך הלמידה שלי היה טיפה קשה. לא הייתי מגיע כל כך לשיעורים וניסיתי להשלים את החומר לבד, אבל בכנות הרבה מהפעמים התעצלתי. למרות כל החיסורים שלי כשכן הגעתי, לא הייתי מבזבז דקה והייתי משקיע ומתקדם בקצב שיא. מהסתכלות לאחור אני יודע שהייתי יכול לעשות יותר, אך מזהים אותי לראות את השליטה שלי בידע על הפרויקט ואת ההבנה שצברתי. יצא לי להתנסות גם בפעם הראשונה עם כל נושא המסכים (Activity), וכתיבת view משלי שבעצם סחבה את כל העיצוב ולוגיקת השחמט.

אני יודע שאחרי העמקה משמעותית נוספת הפרויקט יכול לקבל הרבה יותר פיצ'רים מתקדמים כמו Castle, צבירת נקודות עולמית, הוספת שניות לטיימר אחרי כל תור ומה שהכי עניין אותי זה לפתוח קהילת שחמט שלמה משלי.

אני אציין עוד אתגר שאני אישית תמיד מתקשה בו, עיצוב. עיצוב תמיד היה חלק מאתגר אצלי בעולם התכנות. יצא לי לפני לבנות אתר בתוך ומחוץ לגבולות המגמה ותמיד זה יצא זוועה. אין לי בכלל חוש לצבעים, גוונים ועוד ואני מאמין שזה די ניכר בפרויקט שלי. הייתי רוצה יותר להתעמק בעולם העיצוב והגרפיקה בגבולות התכנות. למרות שאני אישית לא הכי אהבתי את העיצוב של הפרויקט, יחסית לניסיונות קודמים יש לי יותר הבנה ובסיס בעיצוב. אני יודע שעם עוד טיפה מאמץ אני יכול להגיע לרמה טובה של frontend.

לסיכום אני מרגיש שחוויתי שנה מטלטלת. ממלחמה לעומס פתאומי להתנדבויות ועוד. האם הייתי יכול לעשות יותר? חד משמעית. האם אני מרוצה בכל זאת מהתוצר הסופי? לגמרי. מי היה מאמין שבסוף שנת ה-י"ב שלי אני אצא עם פרויקט בכיס, מוסר עבודה, איך מתחילים לחקור, איך חוקרים ועוד מלא לקחים שהפקתי מהשנה הזאת. אני לא אשקר השנה לא הייתה השנה הכי טובה שלי, אך בידיעה שיש לי פרויקט שהכנסתי אליו המון מחשבה אני יכול להגיד שאני מסופק, ומקווה להתחיל כבר את הפרויקט הבא שלי.



Regular Chess

ביבליוגרפיה

- **Android Developers.** (n.d.). *Save data in key-value pairs (SharedPreferences).* Android Developers. <https://developer.android.com/training/data-storage/shared-preferences?hl=he>
- **Android Developers.** (n.d.). *Create a notification.* Android Developers. <https://developer.android.com/develop/ui/compose/notifications/create-notification?hl=he>
- **Bartek Spitza.** (2023, October 24). *The fascinating programming of a chess engine* [Video]. YouTube. https://www.youtube.com/watch?v=w4FFX_otR-4
- **Debadatta07.** (2020, June 14). *Understanding variables in Java: Local, global, instance, reference, static, and objects.* Medium. <https://medium.com/@debadatta07/understanding-variables-in-java-local-global-instance-reference-static-and-objects-0a641c63167f>
- **Fischy Stream.** (2024, April 13). *17 Weird chess rules explained in 5 minutes* [Video]. YouTube. <https://www.youtube.com/watch?v=8qzdvJMhGtE>
- **GeeksforGeeks.** (2022, November 28). *How to create bitmap from view in Android.* GeeksforGeeks. <https://www.geeksforgeeks.org/android/how-to-create-bitmap-from-view-in-android/>
- **Gemini.** (2026). [צ'אט בינה מלאכותית] שיחת ייעוץ ופיתוח קוד: מימוש רכיב שעון עצר. Google Gemini. <https://gemini.google.com/app/bde84c52f95dac78>
- **Nutrient.** (2023, May 11). *Android: The subtle art of debugging.* Nutrient Blog. <https://www.nutrient.io/blog/android-subtle-art-of-debugging/>



Regular Chess

נספחים

מחלקת AlarmReceiver.java

```
package com.example.chessgame;

import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.Build;
import android.os.SystemClock;
import android.app.AlarmManager;
import androidx.core.app.NotificationCompat;

public class AlarmReceiver extends BroadcastReceiver {

    private static int notificationId = 0; // Increment so each notification is unique

    /*
    The func create the notification message and notify it ( display ) through the screen.
    input: context -> which screen the message came from
    output: none ( but you get notify )
    */
    @Override
    public void onReceive(Context context, Intent intent) {

        NotificationCompat.Builder builder = new NotificationCompat.Builder(context, "alarm_channel")
            .setSmallIcon(R.drawable.kingb) //the king icon
            .setContentTitle("Chess Game")
            .setContentText("Hi! Lets Play Some Chess.")
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setAutoCancel(true); // Dismisses notification when tapped

        NotificationManager manager =
            (NotificationManager) context.getSystemService(Context.NOTIFICATION_SERVICE);

        if (manager != null) {
            manager.notify(notificationId++, builder.build());
        }

        // Reschedule next alarm
        scheduleNextAlarm(context);
    }

    /*
    The func create the next alarm to start working
    if the client stays in the lobbyPage for more then 10 sec
    input: context -> which screen does the alarm happens
    output: none
    */
    private void scheduleNextAlarm(Context context) {
        AlarmManager alarmManager = (AlarmManager) context.getSystemService(Context.ALARM_SERVICE);
        if (alarmManager == null) return;

        Intent intent = new Intent(context, AlarmReceiver.class); //creating an intent for the next alarm
    }
```



Regular Chess

```
PendingIntent pendingIntent = PendingIntent.getBroadcast(
    context, 0, intent,
    PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
);

long triggerAtMillis = SystemClock.elapsedRealtime() + 10_000; // start alarm after 10 seconds

if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    if (alarmManager.canScheduleExactAlarms()) {
        alarmManager.setExactAndAllowWhileIdle(
            AlarmManager.ELAPSED_REALTIME_WAKEUP, triggerAtMillis, pendingIntent);
    }
} else {
    alarmManager.setExactAndAllowWhileIdle(
        AlarmManager.ELAPSED_REALTIME_WAKEUP, triggerAtMillis, pendingIntent);
}
}
```

מחלקת BackgroundMusicService.java

```
package com.example.chessgame;

import android.app.Service;
import android.content.Intent;
import android.media.MediaPlayer;
import android.os.IBinder;
import android.util.Log;

public class BackgroundMusicService extends Service {
    //fields
    private MediaPlayer mediaPlayer;

    public static boolean isMusicMuted = false;

    // Holds current track volume level safely (0.0f to 1.0f)
    public static float currentVolume = 0.5f;

    // Action constant key for processing slider event signals
    public static final String ACTION_SET_VOLUME = "com.example.chessgame.SET_VOLUME";
    public static final String EXTRA_VOLUME_VALUE = "volume_value";

    //bind the music to an intent
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }

    /*
    The func set up the background music
    input: none
    output: none
    */
    @Override
    public void onCreate() {
        super.onCreate();
    }
}
```


Regular Chess

```

mediaPlayer = MediaPlayer.create(this, R.raw.backgroundmusic); //create the basic music

        if (mediaPlayer != null) {
            mediaPlayer.setLooping(true);
            // Initialize volume from our shared global state flag
            mediaPlayer.setVolume(currentVolume, currentVolume);
        }
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        // Intercept incoming volume updates broadcasted from UI sliders
        if (intent != null && ACTION_SET_VOLUME.equals(intent.getAction())) {
            float incomingVol = intent.getFloatExtra(EXTRA_VOLUME_VALUE, 0.5f);
            currentVolume = incomingVol; // Cache runtime memory state

            if (mediaPlayer != null && mediaPlayer.isPlaying()) {
                // Instantly update the native audio playback frame mix wrapper properties
                mediaPlayer.setVolume(currentVolume, currentVolume);
            }
        }
        return START_STICKY; // Process action payload early and terminate control stream mapping
    }

    if (mediaPlayer != null && !mediaPlayer.isPlaying() && !isMusicMuted) {
        mediaPlayer.setVolume(currentVolume, currentVolume); // Sync before playing
        mediaPlayer.start();
        Log.d("MUSIC_SERVICE", "Background music started.");
    }
    return START_STICKY;
}

    @Override
    public void onDestroy() {
        if (mediaPlayer != null) {
            mediaPlayer.stop();
            mediaPlayer.release();
            mediaPlayer = null;
            Log.d("MUSIC_SERVICE", "Background music stopped and resources freed.");
        }
        super.onDestroy();
    }
}

```

מחלקת Bishop.java

```

package com.example.chessgame;

public class Bishop extends Piece {
    //constructor
    public Bishop(Point startIndex, char color, char type) {
        super.setStartIndex(startIndex);
        super.setEndIndex(startIndex);
        super.setColor(color);
        super.setType(type);
    }
}

```

Regular Chess

```

//destructor
@Override
public void destroy() {
    super.destroy();
}

//for bishop it just in the way
@Override
public boolean move(char color, Piece[][] board) {
    return inTheWay(color, board);
}

//for bishop it just in the way
@Override
public boolean eat(char color, Piece[][] board) {
    return inTheWay(color, board);
}

/*
The func cheks if the move is valid and if any piece in the way of the bishop
input: color of the piece, that board it self
output: true if move is valid else false
*/
@Override
public boolean inTheWay(char color, Piece[][] board) {
    // 1. Strictly enforce diagonal geometry validation
    if (Math.abs(getEndIndex().getRow() - getStartIndex().getRow()) ==
        Math.abs(getEndIndex().getCol() - getStartIndex().getCol())) {

        int rowStep = Integer.compare(getEndIndex().getRow(), getStartIndex().getRow());
        int colStep = Integer.compare(getEndIndex().getCol(), getStartIndex().getCol());

        int row = getStartIndex().getRow() + rowStep;
        int col = getStartIndex().getCol() + colStep;

        // Check intermediate squares
        while (row != getEndIndex().getRow() || col != getEndIndex().getCol()) {
            if (board[row][col] != null) {
                return false; // Blocked
            }
            row += rowStep;
            col += colStep;
        }

        // Check landing target validity
        Piece target = board[getEndIndex().getRow()][getEndIndex().getCol()];
        if (target == null) return true;
        return target.getColor() != color;
    }

    return false; // Not a diagonal move!
}

```



Regular Chess

מחלקת Chess.java

```
package com.example.chessgame;

import android.content.Context;
import android.content.Intent;
import android.widget.Toast;

public class Chess {
    //fields
    private char _move;
    public Round _gameRounds;

    //constructor
    public Chess() {
        this._move = Constants.WHITE_PIECE;
        this._gameRounds = new Round(Constants.FRONTEND_MSG);
    }

    /*
    The func is responsible on the game logic first selection of the piece to move
    and then after all of the checks it checks for pawn promotion and if there is
    a win lose or tie it open the game over dialog and finish the game
    */
    public void game(Context context) {
        this._gameRounds.setColor(_move);
        boolean moveSuccessful = this._gameRounds.moveInBoard();

        if (moveSuccessful) {
            // Check for Pawn Promotion FIRST (While it is still the moving player's turn)
            if (isPawnEligibleForPromotion()) {
                // Open the custom dialog, passing the context, current player color ('W' or 'B'), and the choice
                // callback
                PromotionDialog dialog = new PromotionDialog(context, _move, new
                PromotionDialog.PromotionCallback() {
                    @Override
                    public void onPieceSelected(char chosenPiece) {
                        // Normalize the character case depending on piece color matching your matrix framework rules
                        char absolutePiece = (_move == Constants.WHITE_PIECE)
                        ? Character.toLowerCase(chosenPiece)
                        : Character.toUpperCase(chosenPiece);

                        // Update board array layout cell with the new upgraded piece character type
                        _gameRounds.promotePawnOnBoard(absolutePiece);

                        completeTurnSequence(context);
                    }
                });
                dialog.show();
            } else {
                // No promotion happened! Run normal turn finalization workflows instantly
                completeTurnSequence(context);
            }
        } else {
            // Move was rejected — tell the player why
        }
    }
}
```

Regular Chess

```

        if (this._gameRounds.isInCheck()) {
            Toast.makeText(context,
                "Illegal move — your King is still in check!",
                Toast.LENGTH_SHORT).show();
        }
    }
}

/*The func complete all the turn event from switching turns
to check if the kings in check or checkmate or stalemate
*/

private void completeTurnSequence(Context context) {
    // Switch turn here so we can evaluate the NEW active player's state
    switchTurn();

    boolean inCheck = this._gameRounds.isPlayerInCheck();

    // Warn the new active player if they are now caught in check
    if (inCheck) {
        String colorName = (_move == Constants.WHITE_PIECE) ? "White" : "Black";
        android.util.Log.d("CHESS_GAME", colorName + " is in CHECK!");
        Toast.makeText(context,
            colorName + " is in CHECK!",
            Toast.LENGTH_SHORT).show();
    }

    // Evaluate structural end-game check patterns safely
    if (this._gameRounds.isPlayerInCheckMate()) {
        // No legal moves left AND King is in check -> Checkmate!
        finishedGame(context, false);
    } else if (!inCheck && hasNoLegalMovesLeft()) {
        // No legal moves left AND King is NOT in check -> Stalemate Draw!
        finishedGame(context, true);
    }
}

// End-Game handling pipeline supporting both Checkmates and Stalemates
public void finishedGame(Context context, boolean isStalemate) {
    GlobalStat globalState = (GlobalStat) context.getApplicationContext();
    final boolean whiteWon = (this._move == Constants.BLACK_PIECE);

    if (isStalemate) {
        // A stalemate is a draw
        globalState.userStats.stalemates++; // Increment your draws accumulator tracker
        android.util.Log.d("CHESS_GAME", "Game ended in a Stalemate (Draw)!");
    } else {
        if (whiteWon) {
            globalState.userStats.wins++;
            android.util.Log.d("CHESS_GAME", "White wins by Checkmate!");
        } else {
            globalState.userStats.losses++;
            android.util.Log.d("CHESS_GAME", "Black wins by Checkmate!");
        }
    }

    this._move = '0';
}

```

Regular Chess

```

//move to the game over dialog
Intent dialogIntent = new Intent(context, GameOverDialog.class);
dialogIntent.putExtra("IS_STALEMATE", isStalemate);
dialogIntent.putExtra("WHITE_WON", whiteWon);
dialogIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
context.startActivity(dialogIntent);
}

//this func switch to white or black or zero ( game ended )
private void switchTurn() {
    _move = (_move == Constants.WHITE_PIECE) ? Constants.BLACK_PIECE : Constants.WHITE_PIECE;
    _gameRounds.setColor(_move);
}

//set a new move
public void setMove(char move) {
    this._move = move;
}

//return the turn
public char getMove() {
    return _move;
}

// Helper checker hook looking up matching row limits inside Round class wrapper instance
private boolean isPawnEligibleForPromotion() {
    return this._gameRounds.checkLastMoveForPromotion();
}

// Helper wrapper method to determine legal fallback options
private boolean hasNoLegalMovesLeft() {
    return this._gameRounds.isPlayerInCheckMate();
}
}

```

ChessView.java מחלקת

```

package com.example.chessgame;

import android.content.Context;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.graphics.RectF;
import android.graphics.Typeface;
import android.media.MediaPlayer;
import android.os.CountDownTimer;
import android.util.AttributeSet;
import android.view.MotionEvent;
import android.view.View;

import java.util.HashMap;
import java.util.Locale;
import java.util.Map;

```

Regular Chess

```

public class ChessView extends View {
    //fields
    private final Paint paint = new Paint();
    private final Paint textPaint = new Paint();
    private float squareSize = 0f;
    private float boardTopOffset = 0f;

    private Chess chessGame;
    private Point firstClick = null;
    private final Map<String, Bitmap> pieceBitmaps = new HashMap<>();

    private long whiteTimeLeft = 300000;
    private long blackTimeLeft = 300000;
    private CountDownTimer activeTimer;
    private MediaPlayer tickPlayer;

    public interface OnTimeOutListener {
        void onTimeOut(boolean isWhiteTurn);
    }

    private OnTimeOutListener timeOutListener;

    // Setter method so playPage can listen to this event
    public void setOnTimeOutListener(OnTimeOutListener listener) {
        this.timeOutListener = listener;
    }

    //constructor that start the timers reset the tick sound and load the images to the board
    public ChessView(Context context, AttributeSet attrs) {
        super(context, attrs);
        this.chessGame = new Chess();
        paint.setAntiAlias(true);
        textPaint.setAntiAlias(true);
        textPaint.setTypeface(Typeface.create(Typeface.MONOSPACE, Typeface.BOLD));

        loadPieceBitmaps();
        try {
            tickPlayer = MediaPlayer.create(context, R.raw.tick);
        } catch (Exception e) {
            android.util.Log.e("ChessView", "Sound file missing");
        }
        startTurnTimer();
    }

    //This func load all the images if the pieces on to the board
    private void loadPieceBitmaps() {
        int[] resIds = {
            R.drawable.pawnw, R.drawable.rookw, R.drawable.knightw, R.drawable.bishopw,
            R.drawable.queenw, R.drawable.kingw,
            R.drawable.pawnb, R.drawable.rookb, R.drawable.knightb, R.drawable.bishopb, R.drawable.queenb,
            R.drawable.kingb
        };
        String[] keys = {"wp", "wr", "wn", "wb", "wq", "wk", "bp", "br", "bn", "bb", "bq", "bk"};
        for (int i = 0; i < resIds.length; i++) {
            pieceBitmaps.put(keys[i], BitmapFactory.decodeResource(getResources(), resIds[i]));
        }
    }
}

```

Regular Chess

```
//This func draw the board and clock on to the screen
@Override
protected void onDraw(Canvas canvas) {
    super.onDraw(canvas);

    // Reserve at least 160 pixels at top and bottom for clocks
    float verticalPadding = 160f;
    float availableHeightForBoard = getHeight() - (verticalPadding * 2);
    float availableWidth = getWidth();

    // The square size is the smaller of the two to ensure it's a perfect square that fits
    squareSize = Math.min(availableWidth / 8f, availableHeightForBoard / 8f);

    float totalBoardHeight = squareSize * 8;
    float totalBoardWidth = squareSize * 8;

    // Center the board vertically and horizontally
    boardTopOffset = (getHeight() - totalBoardHeight) / 2f;
    float boardLeftOffset = (getWidth() - totalBoardWidth) / 2f;

    // Draw Clocks (positioned relative to board, not screen edges)
    drawClockCard(canvas, whiteTimeLeft, boardTopOffset - 140, true);
    drawClockCard(canvas, blackTimeLeft, boardTopOffset + totalBoardHeight + 20, false);

    // Draw the Board
    canvas.save();
    canvas.translate(boardLeftOffset, boardTopOffset);
    drawBoardGrid(canvas);
    drawPiecesFromLogic(canvas);
    canvas.restore();

    drawStatusOverlay(canvas);
}

//This func draw on to the screen the clock card and when the timer gets to 15 sec
//the clock card turn from yellow to red and if the turn isnt his the card color turn to white
private void drawClockCard(Canvas canvas, long timeLeft, float yPos, boolean isWhiteClock) {
    String timeStr = String.format(Locale.getDefault(), "%02d:%02d",
        (timeLeft / 1000) / 60, (timeLeft / 1000) % 60);

    float cardWidth = getWidth() * 0.5f;
    float cardHeight = 110f;
    float xPos = (getWidth() - cardWidth) / 2;

    RectF card = new RectF(xPos, yPos, xPos + cardWidth, yPos + cardHeight);

    boolean isCurrentTurn = (isWhiteClock && chessGame.getMove() == Constants.WHITE_PIECE) ||
        (!isWhiteClock && chessGame.getMove() == Constants.BLACK_PIECE);

    if (timeLeft <= 15000) {
        // Intense warning red when 15 seconds or less remain
        paint.setColor(Color.parseColor("#EF5350"));
    } else if (isCurrentTurn) {
        // Bright yellow to denote active normal thinking turn state
        paint.setColor(Color.parseColor("#FFEB3B"));
    }
}
```

Regular Chess

```

} else {
    // Plain white for the resting opponent card lane
    paint.setColor(Color.parseColor("#FFFFFF"));
}

// Render card background element
paint.setShadowLayer(8, 0, 4, Color.parseColor("#22000000"));
canvas.drawRoundRect(card, 20, 20, paint);
paint.clearShadowLayer();

textPaint.setTextSize(25f);
textPaint.setColor(timeLeft <= 15000 ? Color.WHITE : Color.DKGRAY);
textPaint.setTextAlign(Paint.Align.CENTER);
canvas.drawText(isWhiteClock ? "WHITE" : "BLACK", xPos + (cardWidth/2), yPos + 35, textPaint);

textPaint.setTextSize(55f);
textPaint.setColor(timeLeft <= 15000 ? Color.WHITE : Color.BLACK);
canvas.drawText(timeStr, xPos + (cardWidth/2), yPos + 95, textPaint);
}

//This function stop the timer from countdown
public void stopTimer() {
    if (activeTimer != null) {
        activeTimer.cancel();
    }
}

/**
 * Starts or resumes the countdown for the player whose turn it currently is.
 * This is public so it can be called from the Activity when a dialog is closed.
 */
public void startTurnTimer() {
    // cancel the previous timer before starting a new one
    if (activeTimer != null) {
        activeTimer.cancel();
    }

    // Determine whose turn it is from the chess engine
    final boolean isWhiteTurn = (chessGame.getMove() == Constants.WHITE_PIECE);
    long currentTime = isWhiteTurn ? whiteTimeLeft : blackTimeLeft;

    // Create the new countdown
    // millisInFuture: the time remaining for the active player
    // countDownInterval: 1000ms (1 second) per tick
    activeTimer = new CountDownTimer(currentTime, 1000) {
        @Override
        public void onTick(long millisUntilFinished) {
            // Update the correct time variable
            if (isWhiteTurn) {
                whiteTimeLeft = millisUntilFinished;
            } else {
                blackTimeLeft = millisUntilFinished;
            }
        }
    };

    GlobalStat globalStat = (GlobalStat) getContext().getApplicationContext();
    globalStat.userStats.timespent += 300000 - whiteTimeLeft;
    globalStat.userStats.timespent /= 60000; //get mins
}

```


Regular Chess

```

        // Play a warning sound if 15 seconds or less remain
        if (millisUntilFinished <= 15000 && tickPlayer != null) {
            if (!tickPlayer.isPlaying()) {
                tickPlayer.start();
            }
        }

        // Redraw the view to update the clock UI
        invalidate();
    }

    @Override
    public void onFinish() {
        // Logic for when time runs out
        final boolean whiteLost = isWhiteTurn;

        if (isWhiteTurn) whiteTimeLeft = 0;
        else blackTimeLeft = 0;

        // Stop the game interaction by changing the engine state to '0'
        chessGame.setMove('0');
        invalidate();

        android.util.Log.d("ChessTimer", "Time is up for " + (whiteLost ? "White" : "Black"));

        //open the game over dialog
        if (timeOutListener != null) {
            timeOutListener.onTimeOut(whiteLost);
        }
    }

    }.start(); // Start the timer immediately
}

//this func draws the board grid on to the screen
private void drawBoardGrid(Canvas canvas) {
    for (int row = 0; row < 8; row++) {
        for (int col = 0; col < 8; col++) {
            paint.setColor((row + col) % 2 == 0 ? Color.parseColor("#EBECD0") : Color.parseColor("#779556"));
            canvas.drawRect(col * squareSize, row * squareSize, (col + 1) * squareSize, (row + 1) * squareSize,
                paint);

            if (firstClick != null && firstClick.getRow() == row && firstClick.getCol() == col) {
                paint.setColor(Color.rgb(160, 186, 202, 68));
                canvas.drawRect(col * squareSize, row * squareSize, (col + 1) * squareSize, (row + 1) * squareSize,
                    paint);
            }
        }
    }
}

//this func draw the pieces from the game board in the chess var on to the screen
//so the screen can be update every turn
private void drawPiecesFromLogic(Canvas canvas) {
    Piece[][] board = chessGame._gameRounds.getBoard();
    if (board == null) return;
    for (int row = 0; row < 8; row++) {

```

Regular Chess

```

        for (int col = 0; col < 8; col++) {
            Piece piece = board[row][col];
            if (piece != null && piece.getColor() != Constants.EMPTY_SLOT) {
                String colorPrefix = (piece.getColor() == Constants.WHITE_PIECE) ? "w" : "b";
                String typeKey = String.valueOf(piece.getType()).toLowerCase();
                String key = colorPrefix + typeKey;

                Bitmap bitmap = pieceBitmaps.get(key);
                if (bitmap != null) {
                    RectF dst = new RectF(col * squareSize, row * squareSize, (col + 1) * squareSize, (row + 1) * squareSize);
                    canvas.drawBitmap(bitmap, null, dst, paint);
                }
            }
        }

        //This func draw the status of the game
        private void drawStatusOverlay(Canvas canvas) {
            if (chessGame.getMove() == '0') {
                textPaint.setColor(Color.RED);
                textPaint.setTextSize(squareSize);
                textPaint.setTextAlign(Paint.Align.CENTER);
                canvas.drawText("CHECKMATE", getWidth() / 2f, getHeight() / 2f, textPaint);
            }
        }

        //this func handle the touch when a player moves one piece in a square into another
        @Override
        public boolean onTouchEvent(MotionEvent event) {
            if (event.getAction() != MotionEvent.ACTION_DOWN) return true;
            try {
                // Horizontal offset calculation for centering
                float boardLeftOffset = (getWidth() - (squareSize * 8)) / 2f;

                int col = (int) ((event.getX() - boardLeftOffset) / squareSize);
                int row = (int) ((event.getY() - boardTopOffset) / squareSize);

                if (col < 0 || col >= 8 || row < 0 || row >= 8) return true;

                if (firstClick == null) {
                    Piece[][] board = chessGame._gameRounds.getBoard();
                    Piece clickedPiece = board[row][col];
                    if (clickedPiece != null && clickedPiece.getColor() == chessGame.getMove()) {
                        firstClick = new Point(row, col);
                        invalidate();
                    }
                } else {
                    if (firstClick.getRow() == row && firstClick.getCol() == col) {
                        firstClick = null;
                    } else {
                        chessGame._gameRounds.setMovePoints(firstClick, new Point(row, col));
                        chessGame.game(getContext());
                        firstClick = null;
                    }
                }
                if (chessGame.getMove() != '0') {
                    startTurnTimer();
                }
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }

```

Regular Chess

```

    } else {
        stopTimer();
    }
    invalidate();
} catch (Exception e) { e.printStackTrace(); }
return true;
}

//this func help to release the timer and the tick sound when the game stops/crushing
@Override
protected void onDetachedFromWindow() {
    super.onDetachedFromWindow();
    if (activeTimer != null) activeTimer.cancel();
    if (tickPlayer != null) { tickPlayer.release(); tickPlayer = null; }
}

//this func get a game logic and set it
public void setGameLogic(Chess chessGame) {
    this.chessGame = chessGame;
    startTurnTimer();
    invalidate();
}
}

```

Constants.java מחלקת

```

package com.example.chessgame;

public class Constants {
    //consts
    public static final int BOARD_LEN = 8;
    public static final char BLACK_PIECE = 'B';
    public static final char WHITE_PIECE = 'W';
    public static final char BISHOP = 'b';
    public static final char KING = 'k';
    public static final char QUEEN = 'q';
    public static final char KNIGHT = 'n';
    public static final char PAWN = 'p';
    public static final char ROOK = 'r';
    public static final char EMPTY_SLOT = '#';
    public static final int PAWN_JUMPS = 1;
    public static final int ROW_START_BLACK = 6;
    public static final int ROW_START_WHITE = 1;
    public static final int DOUBLE_JUMP = 2;
    public static final int DIF_ROW = 1;
    public static final int DIF_COL = 1;
    public static final String FRONTEND_MSG =
        "rnbkqbnrpppppppp00000000000000000000000000000000PPPPPPPRNBKQBNR";
}

```

Regular Chess

מחלקת DatabaseManager.java

```
package com.example.chessgame;

import android.content.Context;
import com.google.firebase.firestore.FirebaseFirestore;

public class DatabaseManager {

    public interface DataStatusCallback {
        void onDataLoaded(UserStatistics stats);
    }

    public interface UserProfileCallback {
        void onProfileLoaded(boolean success);
    }

    public interface DeleteAccountCallback {
        void onAccountDeleted(boolean success, String errorMessage);
    }

    /*
    The func delete the user account from the firestore thats include the stats table
    input: context -> which class he came from
    output: none ( delete the account )
    */
    public static void deleteUserAccount(Context context, final DeleteAccountCallback callback) {
        com.google.firebase.auth.FirebaseAuth user =
            com.google.firebase.auth.FirebaseAuth.getInstance().getCurrentUser();

        if (user == null) {
            if (callback != null) callback.onAccountDeleted(false, "No user is currently signed in.");
            return;
        }

        String uid = user.getUid();

        //wipe out their row/document from the Firestore collection completely
        com.google.firebase.firestore.FirebaseFirestore.getInstance()
            .collection("Statistics")
            .document(uid)
            .delete()
            .addOnSuccessListener(aVoid -> {
                android.util.Log.d("DB_MANAGER", "Firestore statistics document deleted successfully.");

                // delete the identity credentials from Auth
                user.delete()
                    .addOnSuccessListener(aVoidAuth -> {
                        android.util.Log.d("DB_MANAGER", "User Auth credentials purged successfully.");

                        // Clear local memory space cache representation
                        GlobalStat globalStat = (GlobalStat) context.getApplicationContext();
                        globalStat.userStats = new UserStatistics();

                        if (callback != null) callback.onAccountDeleted(true, null);
                    })
                    .addOnFailureListener(e -> {
                        android.util.Log.e("DB_MANAGER", "Auth account purge failed: " + e.getMessage());
                        // This happens if the user logged in a long time ago and needs to re-authenticate
                    });
            });
    }
}
```

Regular Chess

```

        if (callback != null) callback.onAccountDeleted(false, "Authentication timeout. Please log out
                                                    and back in to delete your account.");
    });
    });
    .addOnFailureListener(e -> {
        android.util.Log.e("DB_MANAGER", "Failed to clear Firestore statistics: " + e.getMessage());
        if (callback != null) callback.onAccountDeleted(false, e.getMessage());
    });
    }

    /*
    The func load the username from the stats table by using the uid of the client.
    input: uid -> the user id that the firestore create for each client
    output: none ( update the username in the globalstat )
    */
    public static void loadUserProfile(Context context, String uid, final UserProfileCallback callback) {
        if (uid == null || uid.trim().isEmpty()) {
            if (callback != null) callback.onProfileLoaded(false);
            return;
        }

        GlobalStat globalStat = (GlobalStat) context.getApplicationContext();

        FirebaseFirestore.getInstance()
            .collection("Statistics")
            .document(uid)
            .get()
            .addOnSuccessListener(documentSnapshot -> {
                if (documentSnapshot.exists()) {
                    // Extract just the username field explicitly
                    String cloudUsername = documentSnapshot.getString("username");
                    if (cloudUsername != null && !cloudUsername.trim().isEmpty()) {
                        globalStat.userStats.username = cloudUsername;
                    }
                }
            })
            // Signal the UI thread that processing has finished safely
            if (callback != null) callback.onProfileLoaded(true);
        });
        .addOnFailureListener(e -> {
            android.util.Log.e("DB_MANAGER", "Failed to load user profile: " + e.getMessage());
            // Fire fallback signal anyway so login screen flow isn't left hanging frozen
            if (callback != null) callback.onProfileLoaded(false);
        });
    }

    /*
    The func load all the stats from the stats table by using the uid of the client.
    input: uid -> the user id that the firestore create for each client
    output: none ( update the username in the globalstat )
    */
    public static void loadStatsFromFirebase(Context context, final DataStatusCallback callback) {
        GlobalStat globalStat = (GlobalStat) context.getApplicationContext();

        if (globalStat.userStats == null || globalStat.userStats.Uid == null
            || globalStat.userStats.Uid.trim().isEmpty()) {
            android.util.Log.e("FIREBASE_READ", "Cannot load stats: UID is null.");
        }
    }

```

Regular Chess

```

return;
}

String uid = globalStat.userStats.Uid;

FirebaseFirestore.getInstance()
    .collection("Statistics")
    .document(uid)
    .get()
    .addOnSuccessListener(snapshot -> {
        if (snapshot.exists()) {
            UserStatistics stats = snapshot.toObject(UserStatistics.class);
            if (stats != null) {
                // Keep the UID and username that we already have in memory
                stats.Uid = globalStat.userStats.Uid;
                stats.username = globalStat.userStats.username;

                // Write everything back into global memory
                globalStat.userStats = stats;
                globalStat.statsLoadedFromFirebase = true;
            } else {
                // New user — no document yet, that's fine
                globalStat.statsLoadedFromFirebase = true;
            }
        }

        if (callback != null) callback.onDataLoaded(globalStat.userStats);
    })
    .addOnFailureListener(e -> {
        android.util.Log.e("FIREBASE_READ", "Read failed: " + e.getMessage());
        // Still fire callback so UI doesn't stay stuck on "..."
        if (callback != null) callback.onDataLoaded(globalStat.userStats);
    });
}

/*
The func saves the stats from the globalstat in the stats table in the firestore
input: uid -> the user id that the firestore create for each client
output: none ( update the stats in firestore )
*/

public static void saveStatsToFirebase(Context context) {
    GlobalStat globalStat = (GlobalStat) context.getApplicationContext();

    if (globalStat.userStats == null || globalStat.userStats.Uid == null
        || globalStat.userStats.Uid.trim().isEmpty()) {
        android.util.Log.e("FIREBASE_WRITE", "Cannot save stats: UID is null.");
        return;
    }

    String uid = globalStat.userStats.Uid;

    FirebaseFirestore.getInstance()
        .collection("Statistics")
        .document(uid)
        .set(globalStat.userStats)
        .addOnSuccessListener(aVoid ->
            android.util.Log.d("FIREBASE_WRITE", "Stats saved successfully for UID: " + uid))

```

Regular Chess

```

        .addOnFailureListener(e ->
android.util.Log.e("FIREBASE_WRITE", "Save failed: " + e.getMessage()));
        }

        /*
        The func update just the username in the stats table
        input: uid -> the user id that the firestore create for each client
        output: none ( update the username in firestore )
        */
        public static void updateUsername(String uid, String newUsername) {
            if (uid == null || uid.trim().isEmpty()) return;

            FirebaseFirestore.getInstance()
                .collection("Statistics")
                .document(uid)
                .update("username", newUsername)
                .addOnSuccessListener(aVoid -> android.util.Log.d("DB_MANAGER", "Username synced"))
                .addOnFailureListener(e -> android.util.Log.e("DB_MANAGER", "Sync failed: " + e.getMessage()));
        }
    }
}

```

מחלקת GameOverDialog.java

```

package com.example.chessgame;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class GameOverDialog extends AppCompatActivity {
    /*
    The func is a constructor the find out which win or lose or stalemate
    happened and show it on the activity by getIntent
    */
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.dialog_game_over);
        setFinishOnTouchOutside(false);

        TextView dialogMessage = findViewById(R.id.dialogMessage);
        Button btnGoToLobby = findViewById(R.id.btnGoToLobby);

        boolean isStalemate = getIntent().getBooleanExtra("IS_STALEMATE", false);
        boolean whiteWon = getIntent().getBooleanExtra("WHITE_WON", true);

        // Extract the custom finish style parameter context
        String winReason = getIntent().getStringExtra("WIN_REASON");

        if (isStalemate) {
            dialogMessage.setText("Stalemate! The game ends in a Draw. 🤝");
        } else if ("TIME".equals(winReason)) {
            //Time win
            dialogMessage.setText(whiteWon

```



Regular Chess

```
                ? "Time's up! WHITE wins on time! ♔ ⌚ "
                : "Time's up! BLACK wins on time! ♚ ⌚ ");
            } else {
                // checkmate win
                dialogMessage.setText(whiteWon
                ? "Congratulations! WHITE wins by Checkmate! ♔ "
                : "Game Over! BLACK wins by Checkmate! ♚ ");
            }

            btnGoToLobby.setOnClickListener(v -> {
                DatabaseManager.saveStatsToFirestore(this);

                Intent intent = new Intent(GameOverDialog.this, lobby.class);
                intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_NEW_TASK);
                intent.putExtra("FROM_SCREEN", "GAME");
                startActivity(intent);
                finish();
            });
        }
    }
}
```

GlobalStat.java מחלקת

```
package com.example.chessgame;

import android.app.Application;

public class GlobalStat extends Application {
    public UserStatistics userStats = new UserStatistics(); //the store of the userStats
    public boolean statsLoadedFromFirestore = false; //bool var to see if the stats are already loaded
}
```

instructionPage.java מחלקת

```
package com.example.chessgame;

import android.annotation.SuppressLint;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.ImageButton;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class instructionPage extends AppCompatActivity implements View.OnClickListener {

    ImageButton backbtn;

    //constructor
    @SuppressLint("MissingInflatedId")
    @Override
```




Regular Chess

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.instructionpage);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    //find the button
    backbtn = findViewById(R.id.backbtn);
    backbtn.setOnClickListener(this);
}

//The func make the back button work by set the intent to the lobby
@Override
public void onClick(View v) {
    if(backbtn == v) {
        Intent intent = new Intent(instructionPage.this, lobby.class);
        startActivity(intent);
    }
}
}
```

מחלקת King.java

```
package com.example.chessgame;

import static java.lang.Math.abs;
import android.util.Log;

public class King extends Piece {

    //constructor
    public King(Point startIndex, char color, char type) {
        super.setStartIndex(startIndex);
        super.setEndIndex(startIndex);
        super.setColor(color);
        super.setType(type);
    }

    //destructor
    @Override
    public void destroy() {
        super.destroy();
    }

    //the func checks if the king make a valid move
```

Regular Chess

```

// if yes it return true else false
@Override
public boolean move(char color, Piece[][] board) {
    if (abs(getStartIndex().getRow() - getEndIndex().getRow()) <= Constants.DIF_ROW &&
        abs(getStartIndex().getCol() - getEndIndex().getCol()) <= Constants.DIF_COL) {
        return inTheWay(color, board);
    }
    return false;
}

//the func checks if the king can eat
// if yes it return true else false
@Override
public boolean eat(char color, Piece[][] board) {
    if (abs(getStartIndex().getRow() - getEndIndex().getRow()) <= Constants.DIF_ROW &&
        abs(getStartIndex().getCol() - getEndIndex().getCol()) <= Constants.DIF_COL) {
        return inTheWay(color, board);
    }
    return false;
}

//the func checks if theres any piece in the way
// if yes it return true else false
@Override
public boolean inTheWay(char color, Piece[][] board) {
    Piece target = board[getEndIndex().getRow()][getEndIndex().getCol()];
    if (target == null) return true;
    return target.getColor() != color;
}

/*
The func checks if the king is in check
input: color of the king and the game board
output: if in check true else false
*/
public boolean isInCheck(char color, Piece[][] board) {
    int kRow = -1, kCol = -1;

    // Locates the King on the active board
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            if (board[i][j] instanceof King && board[i][j].getColor() == color) {
                kRow = i;
                kCol = j;
                break;
            }
        }
    }

    if (kRow == -1) return false;

    char opponentColor = (color == Constants.WHITE_PIECE) ? Constants.BLACK_PIECE :
        Constants.WHITE_PIECE;

    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            Piece p = board[i][j];

```

Regular Chess

```

        if (p == null || p.getColor() != opponentColor) continue;

        // Create deep copies to isolate side-effects
        Point savedStart = new Point(p.getStartIndex().getRow(), p.getStartIndex().getCol());
        Point savedEnd = new Point(p.getEndIndex().getRow(), p.getEndIndex().getCol());

        p.setStartIndex(new Point(i, j));
        p.setEndIndex(new Point(kRow, kCol));

        // Attack evaluation using enemy context color
        boolean canAttack = p.eat(opponentColor, board);

        p.setStartIndex(savedStart);
        p.setEndIndex(savedEnd);

        if (canAttack) {
            return true;
        }
        return false;
    }

    /*
     * The func checks if the king is in checkmate
     * input: color of the king and the game board
     * output: if in checkmate true else false
     */
    public boolean checkMate(char color, Piece[][] board) {
        // If the king is not even in check, it cannot be checkmate
        if (!isInCheck(color, board)) return false;

        // Loop through all squares to find current player's pieces
        for (int i = 0; i < Constants.BOARD_LEN; i++) {
            for (int j = 0; j < Constants.BOARD_LEN; j++) {
                Piece p = board[i][j];
                if (p == null || p.getColor() != color) continue;

                // Loop through all potential target destination squares
                for (int x = 0; x < Constants.BOARD_LEN; x++) {
                    for (int y = 0; y < Constants.BOARD_LEN; y++) {
                        if (i == x && j == y) continue;

                        // Check if the piece rules allow moving from (i,j) to (x,y)
                        p.setStartIndex(new Point(i, j));
                        p.setEndIndex(new Point(x, y));

                        boolean legal = p.move(color, board) || p.eat(color, board);

                        if (!legal) {
                            continue; // Not a valid chess move, skip testing it
                        }

                        // Deep copy state attributes before running simulation
                        Piece captured = board[x][y];

```

Regular Chess

```

// Perform Simulation
board[x][y] = p;
board[i][j] = null;
p.setStartIndex(new Point(x, y));
p.setEndIndex(new Point(x, y));

// Evaluate if this simulated move successfully broke the check line
boolean stillInCheck = isInCheck(color, board);

// Undo Simulation perfectly to original states
board[i][j] = p;
board[x][y] = captured;
p.setStartIndex(new Point(i, j));
p.setEndIndex(new Point(i, j));

if (!stillInCheck) {
    Log.d("CHECKMATE_DEBUG", "Escape route found! Piece " + p.getClass().getSimpleName() +
        " can move from (" + i + ", " + j + ") to (" + x + ", " + y + ")");
    return false;
}
}
}
}
}

// If we exhausted every single friendly piece's move combinations
// and every single one leaves the king in check, it is Checkmate!
return true;
}
}

```

מחלקת Knight.java

```

package com.example.chessgame;

public class Knight extends Piece {
    //constructor
    public Knight(Point startIndex, char color, char type) {
        super.setStartIndex(startIndex);
        super.setEndIndex(startIndex);
        super.setColor(color);
        super.setType(type);
    }

    //check just if the endpoint is valid
    @Override
    public boolean move(char color, Piece[][] board) {
        return isValidKnightMove(color, board);
    }

    //check just if the endpoint is valid
    @Override
    public boolean eat(char color, Piece[][] board) {
        return isValidKnightMove(color, board);
    }
}

```

Regular Chess

```

                                @Override
                                public boolean inTheWay(char color, Piece[][] board) {
                                    // Knights jump over everything, so the path is never "blocked"
                                    return true;
                                }

                                /*
                                The func if the knight move is valid by geometry
                                input: color of the piece and the board
                                output: true if valid else false
                                */

                                private boolean isValidKnightMove(char color, Piece[][] board) {
                                    int rowDiff = Math.abs(getEndIndex().getRow() - getStartIndex().getRow());
                                    int colDiff = Math.abs(getEndIndex().getCol() - getStartIndex().getCol());

                                    // Strict L-shape calculation
                                    if ((rowDiff == 2 && colDiff == 1) || (rowDiff == 1 && colDiff == 2)) {

                                        // Validate the final landing square
                                        Piece target = board[getEndIndex().getRow()][getEndIndex().getCol()];
                                        if (target == null) return true; // Empty square is valid
                                        return target.getColor() != color; // Enemy square is valid, teammate blocks it
                                    }

                                    return false; // Not a valid L-shape
                                }

```

מחלקת lobby.java

```

package com.example.chessgame;

import android.annotation.SuppressLint;
import android.app.AlarmManager;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Build;
import android.os.Bundle;
import android.os.SystemClock;
import android.provider.Settings;
import android.util.Log;
import android.view.View;
import android.widget.ImageButton;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;

```

Regular Chess

```
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class lobby extends AppCompatActivity implements View.OnClickListener {

    //fields
    ImageButton settingbtn;
    ImageButton playbtn;
    ImageButton instructbtn;
    ImageButton statbtn;
    ImageButton logoutbtn;
    String fromScreen;
    TextView usernameDisplayTv;
    private BroadcastReceiver batteryLowReceiver;

    /*
    The func construct the page start the background music start all the alarms
    */
    @SuppressWarnings({"MissingInflatedId"})
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.lobby);
        fromScreen = getIntent().getStringExtra("FROM_SCREEN");

        Intent musicIntent = new Intent(this, BackgroundMusicService.class);
        startService(musicIntent);

        batteryLowReceiver = new BroadcastReceiver() {
            @Override
            public void onReceive(Context context, Intent intent) {
                if (Intent.ACTION_BATTERY_LOW.equals(intent.getAction())) {

                    // Display a warning alert to the player
                    Toast.makeText(context,
                        "⚠ Battery is low! Plug in your charger before starting a match.",
                        Toast.LENGTH_LONG).show();
                }
            }
        };

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        //start all the notification process
        createNotificationChannel();
        requestNotificationPermission();
        startRepeatingAlarm();

        //the update or load data logic
        if (fromScreen != null) {
            switch (fromScreen) {

```

Regular Chess

```

                                case "LOGIN":
                                    Log.d("LOBBY", "Arrived from login");
                                    // DO NOTHING HERE — statisticsPage handles its own loading
                                    break;

                                case "GAME":
                                    Log.d("LOBBY", "UPDATE IN FIREBASE");
                                    DatabaseManager.saveStatsToFirestore(this); // use this instead of the old method
                                    break;

                                default:
                                    break;
                                }
                                }

                                //finds all the buttons and text view
                                usernameDisplayTv = findViewById(R.id.usernameDisplayTv);
                                setUsernameBox(); // set the user name box

                                settingbtn = findViewById(R.id.settingbtn);
                                settingbtn.setOnClickListener(this);

                                playbtn = findViewById(R.id.playbtn);
                                playbtn.setOnClickListener(this);

                                instructbtn = findViewById(R.id.instructbtn);
                                instructbtn.setOnClickListener(this);

                                statbtn = findViewById(R.id.statbtn);
                                statbtn.setOnClickListener(this);

                                logoutbtn = findViewById(R.id.logoutbtn);
                                logoutbtn.setOnClickListener(this);

                                }

                                //The func creates a channel for the notification process
                                private void createNotificationChannel() {
                                    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                                        NotificationChannel channel = new NotificationChannel(
                                            "alarm_channel", // Channel ID
                                            "Alarm Notifications", // Channel name (shown in settings)
                                            NotificationManager.IMPORTANCE_HIGH
                                        );
                                        channel.setDescription("Notifications fired by the alarm");

                                        NotificationManager manager = getSystemService(NotificationManager.class);
                                        if (manager != null) {
                                            manager.createNotificationChannel(channel);
                                        }
                                    }
                                }

                                //This func request a notification permission from the phone

```

Regular Chess

```

        private void requestNotificationPermission() {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) { // Android 13+
                if (ActivityCompat.checkSelfPermission(this, android.Manifest.permission.POST_NOTIFICATIONS)
                    != android.content.pm.PackageManager.PERMISSION_GRANTED) {
                    ActivityCompat.requestPermissions(
                        this,
                        new String[]{android.Manifest.permission.POST_NOTIFICATIONS},
                        1001
                    );
                }
            }
        }

        //The func check if permission is granted and if so
        // he start the all alarm process of alert the client to play some games
        private void startRepeatingAlarm() {
            AlarmManager alarmManager = (AlarmManager) getSystemService(Context.ALARM_SERVICE);
            if (alarmManager == null) return;

            Intent intent = new Intent(this, AlarmReceiver.class);
            PendingIntent pendingIntent = PendingIntent.getBroadcast(
                this,
                0,
                intent,
                PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE
            );

            // check permission before scheduling exact alarm
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
                if (!alarmManager.canScheduleExactAlarms()) {
                    // Open settings so user can grant permission
                    Intent permIntent = new Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM);
                    startActivity(permIntent);
                    return;
                }
            }

            // setExactAndAllowWhileIdle + reschedule in receiver
            long triggerAtMillis = SystemClock.elapsedRealtime() + 10_000;
            alarmManager.setExactAndAllowWhileIdle(
                AlarmManager.ELAPSED_REALTIME_WAKEUP,
                triggerAtMillis,
                pendingIntent
            );
        }

        //move to every screen by the button
        @Override
        public void onClick(View v) {

            if(settingbtn == v)
            {
                Intent intent = new Intent(lobby.this,settingPage.class);
                startActivity(intent);
            }

            if(playbtn == v)

```


Regular Chess

```

        {
            Intent intent = new Intent(lobby.this,playPage.class);
            startActivity(intent);
        }

        if(instructbtn == v)
        {
            Intent intent = new Intent(lobby.this,instructionPage.class);
            startActivity(intent);
        }

        if(statbtn == v)
        {
            Intent intent = new Intent(lobby.this,statisticsPage.class);
            startActivity(intent);
        }

        //log out process starting
        //stop the music than wipe the cache from the global stat and in last
        //move to the log lobby
        if(logoutbtn == v) {
            Intent musicIntent = new Intent(lobby.this, BackgroundMusicService.class);
            stopService(musicIntent);

            BackgroundMusicService.isMusicMuted = false;

            GlobalStat globalStat = (GlobalStat) getApplicationContext();
            if (globalStat.userStats != null) {
                globalStat.userStats = new UserStatistics(); // Wipes current cache
            }

            Intent intent = new Intent(lobby.this, log_lobby.class);
            startActivity(intent);
            finish();
        }

        //set the username on the textView
        private void setUsernameBox()
        {
            GlobalStat globalStat = (GlobalStat) getApplicationContext();
            usernameDisplayTv.setText(globalStat.userStats.username);
        }

        @Override
        protected void onResume() {
            super.onResume();
            // listening for the low-battery broadcast when the user enters the lobby
            IntentFilter filter = new IntentFilter(Intent.ACTION_BATTERY_LOW);
            registerReceiver(batteryLowReceiver, filter);
        }

        @Override
        protected void onPause() {
            super.onPause();
            // Stop listening when the user navigates away from the lobby

```



Regular Chess

```
// prevent memory leaks!  
if (batteryLowReceiver != null) {  
    unregisterReceiver(batteryLowReceiver);  
}  
}  
}
```

מחלקת log_lobby.java

```
package com.example.chessgame;  
  
import android.annotation.SuppressLint;  
import android.content.Intent;  
import android.os.Bundle;  
import android.view.View;  
import android.widget.ImageButton;  
  
import androidx.activity.EdgeToEdge;  
import androidx.appcompat.app.AppCompatActivity;  
import androidx.core.graphics.Insets;  
import androidx.core.view.ViewCompat;  
import androidx.core.view.WindowInsetsCompat;  
  
public class log_lobby extends AppCompatActivity implements View.OnClickListener {  
  
    //fields  
    ImageButton loginbtn;  
    ImageButton signinbtn;  
  
    //constructor  
    @SuppressWarnings({"MissingInflatedId"})  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        EdgeToEdge.enable(this);  
        setContentView(R.layout.log_lobby);  
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {  
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());  
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);  
            return insets;  
        });  
  
        //find buttons  
        loginbtn = findViewById(R.id.loginbtn);  
        loginbtn.setOnClickListener(this);  
  
        signinbtn = findViewById(R.id.signinbtn);  
        signinbtn.setOnClickListener(this);  
  
    }  
  
    //move to screen by the press of a button
```

Regular Chess

```

@Override
public void onClick(View v) {

    if(loginbtn == v)
    {
        Intent intent = new Intent(log_lobby.this,loginPage.class);
        startActivity(intent);
    }

    if(signinbtn == v)
    {
        Intent intent = new Intent(log_lobby.this,signinPage.class);
        startActivity(intent);
    }
}
}

```

מחלקת loginPage.java

```

package com.example.chessgame;

import android.annotation.SuppressLint;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.android.gms.tasks.OnSuccessListener;
import com.google.firebase.auth.AuthResult;
import com.google.firebase.auth.FirebaseAuth;

public class loginPage extends AppCompatActivity implements View.OnClickListener {

    //fields
    ImageButton backbtn;
    EditText emailtv, passwordtv;
    ImageButton loginbtn;

    FirebaseAuth auth;

    //constructor
    @SuppressLint({"MissingInflatedId"})
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.login);
    }
}

```

Regular Chess

```

ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
    Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
    v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
    return insets;
});

//find buttons and textviews
backbtn = findViewById(R.id.backbtn);
backbtn.setOnClickListener(this);

emailtv = findViewById(R.id.emailtv);
passwordtv = findViewById(R.id.passwordtv);
loginbtn = findViewById(R.id.loginbtn);

auth = FirebaseAuth.getInstance();

loginbtn.setOnClickListener(this);

}

/*
The func see which button pressed and move to the right activity
*/
@Override
public void onClick(View v) {

    if(backbtn == v)
    {
        //letting the lobby know where does the client come from
        Intent intent = new Intent(loginPage.this, log_lobby.class);
        intent.putExtra("FROM_SCREEN", "LOGIN");
        startActivity(intent);
        finish();
    }

    if(loginbtn == v)
    {
        //try to login
        String txt_logemail = emailtv.getText().toString();
        String txt_logPw = passwordtv.getText().toString();
        loginClient(txt_logemail, txt_logPw);
    }

}

/*
The func check if the email and password are already register in the auth table
if yes he will move to the lobby activity and let the lobby know the client came from
the loginPage
input: txtLogemail -> the email the client gave
txtLogPw -> the password the client gave
output: none
*/
private void loginClient(String txtLogemail, String txtLogPw) {
    auth.signInWithEmailAndPassword(txtLogemail, txtLogPw).addOnSuccessListener(new
        OnSuccessListener<AuthResult>() {
            @Override

```

Regular Chess

```

        public void onSuccess(AuthResult authResult) {
            String uid = auth.getUid();
            if (uid == null) return;

            GlobalStat globalStat = (GlobalStat) getApplicationContext();

            if (globalStat.userStats == null) {
                globalStat.userStats = new UserStatistics();
            }

            // Bind the essential local variables first
            globalStat.userStats.Uid = uid;

            // Set a fallback username from the email prefix in case Firestore is empty
            globalStat.userStats.username = txtLogemail.substring(0, txtLogemail.indexOf('@'));

            DatabaseManager.loadUserProfile(loginPage.this, uid, new DatabaseManager.UserProfileCallback() {
                @Override
                public void onProfileLoaded(boolean success) {
                    // This block executes dynamically after your network request drops
                    Toast.makeText(loginPage.this, "Login successful", Toast.LENGTH_SHORT).show();

                    Intent intent = new Intent(loginPage.this, lobby.class);
                    intent.putExtra("FROM_SCREEN", "LOGIN");
                    startActivity(intent);
                    finish();
                }
            });
        }
    }
}

```

מחלקת Pawn.java

```

package com.example.chessgame;

import static java.lang.Math.abs;

public class Pawn extends Piece {

    //field
    private int m_jumps;

    //constructor
    public Pawn(Point startIndex, char color, char type) {
        this.m_jumps = Constants.PAWN_JUMPS;
        super.setStartIndex(startIndex);
        super.setEndIndex(startIndex);
        super.setColor(color);
        super.setType(type);
    }
}

```

Regular Chess

```

//destructor
public void destroy() {
    this.m_jumps = 0;
    super.destroy();
}

//check if the pawn can be moved one or two up or down
@Override
public boolean move(char color, Piece[][] board) {
    int startRow = getStartIndex().getRow();
    int startCol = getStartIndex().getCol();
    int endRow = getEndIndex().getRow();
    int endCol = getEndIndex().getCol();

    // Straight movement must land on an empty square
    if (board[endRow][endCol] != null) {
        return false;
    }

    // Straight movement cannot change columns
    if (startCol != endCol) {
        return false;
    }

    // Check the path using the corrected inTheWay logic
    if (!inTheWay(color, board)) {
        return false; // Something is blocking the path forward
    }

    // Validate the exact step length allowed based on color direction
    if (color == Constants.WHITE_PIECE) {
        // White moves down (increasing rows): either 1 step, or 2 steps from starting row (row 1)
        if (endRow == startRow + 1) {
            return true;
        }
        if (startRow == 1 && endRow == 3) {
            return true;
        }
    } else {
        // Black moves up (decreasing rows): either 1 step, or 2 steps from starting row (row 6)
        if (endRow == startRow - 1) {
            return true;
        }
        if (startRow == 6 && endRow == 4) {
            return true;
        }
    }

    return false;
}

//the func checks if the pawn can eat diagonal
@Override
public boolean eat(char color, Piece[][] board) {
    int startRow = getStartIndex().getRow();
    int startCol = getStartIndex().getCol();
    int endRow = getEndIndex().getRow();

```

Regular Chess

```

        int endCol = getEndIndex().getCol();

        Piece target = board[endRow][endCol];

        // To capture, there must be an enemy piece sitting on the target square
        if (target == null || target.getColor() == color) {
            return false;
        }

        int rowDiff = endRow - startRow;
        int colDiff = abs(endCol - startCol);

        // Capturing must be exactly 1 square diagonally
        if (colDiff != 1) {
            return false;
        }

        if (color == Constants.WHITE_PIECE) {
            return rowDiff == 1; // White captures moving down
        } else {
            return rowDiff == -1; // Black captures moving up
        }
    }

    //the func checks if there is a player in the way of the pawn moves
    @Override
    public boolean inTheWay(char color, Piece[][] board) {
        int startRow = getStartIndex().getRow();
        int startCol = getStartIndex().getCol();
        int endRow = getEndIndex().getRow();

        // If moving 2 squares, check the intermediate square right in front of the pawn
        if (abs(endRow - startRow) == 2) {
            int direction = (color == Constants.WHITE_PIECE) ? 1 : -1;
            int intermediateRow = startRow + direction;

            // If the intermediate square is occupied, path is blocked
            if (board[intermediateRow][startCol] != null) {
                return false;
            }
        }

        // Path is completely clear
        return true;
    }

    //the func turn the piece type to a queen rook bishop or knight
    public boolean turnPiece(char type) {
        if (type == Constants.QUEEN) super.setType(Constants.QUEEN);
        else if (type == Constants.ROOK) super.setType(Constants.ROOK);
        else if (type == Constants.BISHOP) super.setType(Constants.BISHOP);
        else if (type == Constants.KNIGHT) super.setType(Constants.KNIGHT);
        else return false;
        return true;
    }
}

```

Regular Chess

מחלקת Piece.java

```
package com.example.chessgame;

public abstract class Piece {
    //fields
    protected Point m_startIndex;
    protected Point m_endIndex;
    protected char m_color;
    protected char m_type;

    //constructor reset fields
    Piece() {
        this.m_startIndex = new Point();
        this.m_endIndex = new Point();
        setColor(Constants.EMPTY_SLOT);
        setType(Constants.EMPTY_SLOT);
    }

    //destructor
    public void destroy()
    {
        this.m_startIndex.destroy();
        this.m_endIndex.destroy();
        this.m_startIndex = null;
        this.m_endIndex = null;
        this.m_color = '\0';
        this.m_type = '\0';
    }

    //abstract function for every piece
    public abstract boolean move(char color, Piece[][] board);
    public abstract boolean eat(char color, Piece[][] board);
    public abstract boolean inTheWay(char color, Piece[][] board);

    //get functions
    public Point getStartIndex()
    {
        return this.m_startIndex;
    }

    public Point getEndIndex()
    {
        return this.m_endIndex;
    }

    public char getColor()
    {
        return this.m_color;
    }

    public char getType()
    {
        return this.m_type;
    }

    //set functions
    public void setStartIndex(Point startIndex)
    {
        this.m_startIndex = startIndex;
    }
}
```




Regular Chess

```
    }  
    public void setEndIndex(Point endIndex)  
    {  
        this.m_endIndex = endIndex;  
    }  
  
    public void setColor(char color)  
    {  
        this.m_color = color;  
    }  
    public void setType(char type)  
    {  
        this.m_type = type;  
    }  
}
```

מחלקת playPage.java

```
package com.example.chessgame;  
  
import android.content.Intent;  
import android.os.Bundle;  
import android.view.View;  
import android.widget.ImageButton;  
  
import androidx.appcompat.app.AlertDialog;  
import androidx.appcompat.app.AppCompatActivity;  
  
public class playPage extends AppCompatActivity implements View.OnClickListener {  
  
    //fields  
    private ImageButton backbtn;  
    private ChessView chessView;  
    private Chess chessGame;  
  
    /*  
    The func construct and start the game logic  
    */  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.playpage);  
  
        // Initialize Chess Logic first  
        chessGame = new Chess();  
  
        //updating the games amount  
        GlobalStat globalStat = (GlobalStat)getApplicationContext();  
        globalStat.userStats.gamesPlayed++;  
  
        // Initialize Views  
        chessView = findViewById(R.id.chessBoard);  
        backbtn = findViewById(R.id.backbtn);  
    }  
}
```

Regular Chess

```

//Safety Check
if (chessView == null) {
    android.util.Log.e("playPage", "chessView not found! Check your XML ID.");
}

// Link logic to view
if (chessView != null) {
    chessView.setGameLogic(chessGame);

    chessView.setOnTimeoutListener(new ChessView.OnTimeoutListener() {
        //if the game stoped on time this handle it to the GameOverDialog
        @Override
        public void onTimeOut(boolean isWhiteTurn) {
            // If White ran out of time, Black wins (whiteWon = false)
            // If Black ran out of time, White wins (whiteWon = true)
            boolean whiteWon = !isWhiteTurn;

            // Route straight to the GameOverDialog package lane
            Intent intent = new Intent(playPage.this, GameOverDialog.class);
            intent.putExtra("IS_STALEMATE", false);
            intent.putExtra("WHITE_WON", whiteWon);

            // Pass the dynamic layout polish key we set up
            intent.putExtra("WIN_REASON", "TIME");

            // Wipe backstack
            intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_NEW_TASK);
            startActivity(intent);
            finish();
        }
    });

    if (backbtn != null) {
        backbtn.setOnClickListener(this);
    }

    //handles the exit button
    @Override
    public void onClick(View v) {
        if (v.getId() == R.id.backbtn) {
            showExitDialog();
        }
    }

    /*
    The func open a dialog for the player and handle if the player wants to stop the game
    and it handle if the player wants to continue the game
    input: none
    output: none (continue timer/move to lobby)
    */
    private void showExitDialog() {
        // Stop the clock
        if (chessView != null) {

```

Regular Chess

```

        chessView.stopTimer();
    }

    new AlertDialog.Builder(this)
        .setTitle("Exit Game")
        .setMessage("Are you sure you want to quit?")
        .setCancelable(false)
        .setPositiveButton("Yes, Exit", (dialog, which) -> {
            Intent intent = new Intent(playPage.this, lobby.class);
            intent.putExtra("FROM_SCREEN", "GAME");
            finish();
        })
        .setNegativeButton("No, Resume", (dialog, which) -> {
            // Resume the timer
            if (chessView != null) {
                chessView.startTurnTimer();
            }
        })
        .show();
    }
}

```

מחלקת Point.java

```

package com.example.chessgame;

public class Point {

    //fields
    private int m_row;
    private int m_col;

    //constructor with row and col
    public Point(int row, int col)
    {
        this.m_row = row;
        this.m_col = col;
    }

    //constructor without row and col
    public Point()
    {
        this.m_row = 0;
        this.m_col = 0;
    }

    //destructor
    public void destroy()
    {
        this.m_row = 0;
        this.m_col = 0;
    }

    //The func return the row of the point
    public int getRow()
    {

```

Regular Chess

```

return this.m_row;
}

//The func return the col of the point
public int getCol()
{
return this.m_col;
}

//the func get a new row and replace with the old row
public void setRow(int row)
{
this.m_row = row;
}

//the func get a new col and replace with the old col
public void setCol(int col)
{
this.m_col = col;
}
}

```

מחלקת PromotionDialog.java

```

package com.example.chessgame;

import android.app.Dialog;
import android.content.Context;
import android.os.Bundle;
import android.view.Window;
import android.widget.ImageButton;

public class PromotionDialog extends Dialog {

    public interface PromotionCallback {
        void onPieceSelected(char chosenPiece);
    }

    private final PromotionCallback callback;
    private final char playerColor;

    // Constructor that pass the color of the current turn player to load the correct assets
    public PromotionDialog(Context context, char playerColor, PromotionCallback callback) {
        super(context);
        this.playerColor = playerColor;
        this.callback = callback;
    }

    /*
    The func bind the image buttons and set their images by the turn color
    and open the dialog so the player have to choose which piece he wants to promote
    */

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
    }
}

```

Regular Chess

```
requestWindowFeature(Window.FEATURE_NO_TITLE);
setContentView(R.layout.dialog_promotion);
setCancelable(false); // Make sure they can't dismiss without choosing

ImageButton btnQueen = findViewById(R.id.img_btn_queen);
ImageButton btnRook = findViewById(R.id.img_btn_rook);
ImageButton btnKnight = findViewById(R.id.img_btn_knight);
ImageButton btnBishop = findViewById(R.id.img_btn_bishop);

// Set the images programmatically depending on whose turn it is
if (playerColor == Constants.WHITE_PIECE) {
    btnQueen.setImageResource(R.drawable.queenw);
    btnRook.setImageResource(R.drawable.rookw);
    btnKnight.setImageResource(R.drawable.knightw);
    btnBishop.setImageResource(R.drawable.bishopw);
} else {
    btnQueen.setImageResource(R.drawable.queenb);
    btnRook.setImageResource(R.drawable.rookb);
    btnKnight.setImageResource(R.drawable.knightb);
    btnBishop.setImageResource(R.drawable.bishopb);
}

// Set up the click listeners to trigger selection
btnQueen.setOnClickListener(v -> select('Q'));
btnRook.setOnClickListener(v -> select('R'));
btnKnight.setOnClickListener(v -> select('N'));
btnBishop.setOnClickListener(v -> select('B'));
}

//The func send the desire piece to the callback and close the dialog
private void select(char piece) {
    callback.onPieceSelected(piece);
    dismiss();
}
}
```

מחלקת Queen.java

```
package com.example.chessgame;

public class Queen extends Piece {

    //fields
    private final Rook m_rook;
    private final Bishop m_bishop;

    //constructor
    public Queen(Point startIndex, char color, char type) {
        this.m_rook = new Rook(new Point(startIndex.getRow(), startIndex.getCol()), color, Constants.ROOK);
        this.m_bishop = new Bishop(new Point(startIndex.getRow(), startIndex.getCol()), color,
            Constants.BISHOP);
        super.setStartIndex(startIndex);
        super.setEndIndex(startIndex);
        super.setColor(color);
        super.setType(type);
    }
}
```

Regular Chess

```

//This func sync the pieces points
private void syncSubPieces() {
    // Pass brand new Point objects so mutations don't alter shared data structures
    Point currentStart = new Point(getStartIndex().getRow(), getStartIndex().getCol());
    Point currentEnd = new Point(getEndIndex().getRow(), getEndIndex().getCol());

    m_rook.setStartIndex(currentStart);
    m_rook.setEndIndex(currentEnd);

    m_bishop.setStartIndex(currentStart);
    m_bishop.setEndIndex(currentEnd);
}

//this func checks if the queen can move diagonal or in straight line
@Override
public boolean move(char color, Piece[][] board) {
    syncSubPieces();

    // Diagonal check
    if (Math.abs(getEndIndex().getRow() - getStartIndex().getRow()) == Math.abs(getEndIndex().getCol() -
        getStartIndex().getCol())) {
        return m_bishop.move(color, board);
    }

    // Straight line check
    else if (getStartIndex().getRow() == getEndIndex().getRow() || getStartIndex().getCol() ==
        getEndIndex().getCol()) {
        return m_rook.move(color, board);
    }

    return false;
}

//this func checks if the queen can eat diagonal or in straight line
@Override
public boolean eat(char color, Piece[][] board) {
    syncSubPieces();

    if (Math.abs(getEndIndex().getRow() - getStartIndex().getRow()) == Math.abs(getEndIndex().getCol() -
        getStartIndex().getCol())) {
        return m_bishop.eat(color, board);
    } else if (getStartIndex().getRow() == getEndIndex().getRow() || getStartIndex().getCol() ==
        getEndIndex().getCol()) {
        return m_rook.eat(color, board);
    }

    return false;
}

//this func checks if the queen have any pieces int the way diagonal or in straight line
@Override
public boolean inTheWay(char color, Piece[][] board) {
    syncSubPieces();

    if (Math.abs(getEndIndex().getRow() - getStartIndex().getRow()) == Math.abs(getEndIndex().getCol() -
        getStartIndex().getCol())) {
        return m_bishop.inTheWay(color, board);
    } else if (getStartIndex().getRow() == getEndIndex().getRow() || getStartIndex().getCol() ==
        getEndIndex().getCol()) {

```



Regular Chess

```
return m_rook.inTheWay(color, board);
    }
    return false;
    }
    }
```

מחלקת Rook.java

```
package com.example.chessgame;

public class Rook extends Piece {
    //constructor
    public Rook(Point startIndex, char color, char type) {
        super.setStartIndex(startIndex);
        super.setEndIndex(startIndex);
        super.setColor(color);
        super.setType(type);
    }

    //destructor
    @Override
    public void destroy() {
        super.destroy();
    }

    //for rook just return the intheway
    @Override
    public boolean move(char color, Piece[][] board) {
        // Must be a valid straight line and have a clear path
        return inTheWay(color, board);
    }

    //for rook just return the intheway
    @Override
    public boolean eat(char color, Piece[][] board) {
        // To attack/eat, it MUST follow the exact same structural path rules
        return inTheWay(color, board);
    }

    /*
    The func cheks if the move is valid and if any piece in the way of the rook
    input: color of the piece, that board it self
    output: true if move is valid else false
    */
    @Override
    public boolean inTheWay(char color, Piece[][] board) {
        // Strictly enforce positional validation (Same row OR same column)
        if (getStartIndex().getRow() == getEndIndex().getRow() || getStartIndex().getCol() ==
            getEndIndex().getCol()) {

            // Vertical movement logic
            if (getStartIndex().getCol() == getEndIndex().getCol()) {
                int upOrDown = (getStartIndex().getRow() > getEndIndex().getRow()) ? -1 : 1;
                for (int row = getStartIndex().getRow() + upOrDown; row != getEndIndex().getRow(); row +=
                    upOrDown) {
```

Regular Chess

```

        if (board[row][getStartIndex().getCol()] != null) {
            return false; // Blocked
        }
    }
}

// Horizontal movement logic
else if (getStartIndex().getRow() == getEndIndex().getRow()) {
    int leftOrRight = (getStartIndex().getCol() > getEndIndex().getCol()) ? -1 : 1;

    for (int col = getStartIndex().getCol() + leftOrRight; col != getEndIndex().getCol(); col += leftOrRight) {
        if (board[getStartIndex().getRow()][col] != null) {
            return false; // Blocked
        }
    }
}

// 2. Check the landing square targeting rule
Piece target = board[getEndIndex().getRow()][getEndIndex().getCol()];
if (target == null) return true; // Empty square is fine
return target.getColor() != color; // Enemy square is fine, teammate is blocked
}

return false; // Not a straight line!
}
}

```

מחלקת Round.java

```

package com.example.chessgame;

import android.util.Log;

public class Round {
    //fields
    private final Piece[][] _board;
    private char _color;
    private Point _p, _p1;
    String _inputBoard;
    private boolean _isInCheck = false;

    //constructor
    public Round(String inputBoard) {
        if (inputBoard == null || inputBoard.length() < 64) {
            Log.e("Round", "Invalid board string length!");
            this._inputBoard = "0".repeat(64);
        } else {
            this._inputBoard = inputBoard;
        }

        this._board = new Piece[Constants.BOARD_LEN][Constants.BOARD_LEN];
        this._p = new Point();
        this._p1 = new Point();
        this._color = Constants.WHITE_PIECE;

        turnToBoard();
    }
}

```


Regular Chess

```

/*
The func get to points and reset the p and p1 points
*/
public void setMovePoints(Point start, Point end) {
    this._p = start;
    this._p1 = end;
}

//the function returns a bool var of the isInCheck
public boolean isInCheck() {
    return _isInCheck;
}

/*
The func move the piece that locates in p to p1 but
first it simulate to see if that cause any check
*/
public boolean moveInBoard() {
    if (_p == null || _p1 == null) return false;

    Piece pieceToMove = _board[_p.getRow()][_p.getCol()];

    if (pieceToMove == null) {
        Log.d("CHESS_DEBUG", "No piece at " + _p.getRow() + "," + _p.getCol());
        return false;
    }

    if (pieceToMove.getColor() != this._color) {
        Log.d("CHESS_DEBUG", "Wrong color. Piece=" + pieceToMove.getColor() + " Turn=" + this._color);
        return false;
    }

    pieceToMove.setStartIndex(new Point(_p.getRow(), _p.getCol()));
    pieceToMove.setEndIndex(new Point(_p1.getRow(), _p1.getCol()));

    boolean canMove = pieceToMove.move(this._color, _board);
    boolean canEat = pieceToMove.eat(this._color, _board);

    Log.d("CHESS_DEBUG", "Move from (" + _p.getRow() + "," + _p.getCol() + ") to (" +
        _p1.getRow() + "," + _p1.getCol() + ")");
    Log.d("CHESS_DEBUG", "canMove=" + canMove + " canEat=" + canEat);

    if (!canMove && !canEat) {
        Log.d("CHESS_DEBUG", "Piece says move is illegal by its own rules");
        return false;
    }

    // Simulate
    Piece captured = _board[_p1.getRow()][_p1.getCol()];
    _board[_p1.getRow()][_p1.getCol()] = pieceToMove;
    _board[_p.getRow()][_p.getCol()] = null;

    pieceToMove.setStartIndex(new Point(_p1.getRow(), _p1.getCol()));
    pieceToMove.setEndIndex(new Point(_p1.getRow(), _p1.getCol()));

    King king = findKing(this._color);

```

Regular Chess

```

Log.d("CHESS_DEBUG", "King found: " + (king != null));
        if (king != null) {
Log.d("CHESS_DEBUG", "King position: " + king.getStartIndex().getRow() + "," +
        king.getStartIndex().getCol());
        }

boolean stillInCheck = (king != null) && king.isInCheck(this._color, _board);
Log.d("CHESS_DEBUG", "stillInCheck after simulation=" + stillInCheck);

// Undo
_board[_p.getRow()][_p.getCol()] = pieceToMove;
_board[_p1.getRow()][_p1.getCol()] = captured;
pieceToMove.setStartIndex(new Point(_p.getRow(), _p.getCol()));
pieceToMove.setEndIndex(new Point(_p1.getRow(), _p1.getCol()));

        if (stillInCheck) {
            _isInCheck = true;
Log.d("CHESS_DEBUG", "REJECTED: king still in check");
            return false;
        }

// Apply
_board[_p1.getRow()][_p1.getCol()] = pieceToMove;
_board[_p.getRow()][_p.getCol()] = null;
pieceToMove.setStartIndex(new Point(_p1.getRow(), _p1.getCol()));
pieceToMove.setEndIndex(new Point(_p1.getRow(), _p1.getCol()));

        _isInCheck = false;
        return true;
    }

//The func try to find the king by the color its given
    public King findKing(char color) {
        for (int i = 0; i < 8; i++)
            for (int j = 0; j < 8; j++)
                if (_board[i][j] instanceof King && _board[i][j].getColor() == color)
                    return (King) _board[i][j];
        return null;
    }

    /*
    The func return true if the player king is in checkmate else false
    */
    public boolean isPlayerInCheckMate() {
        King activeKing = findKing(this._color);
        if (activeKing != null)
            return activeKing.checkMate(this._color, this._board);
        return false;
    }

    /*
    The func return true if the player king is in check else false
    */
    public boolean isPlayerInCheck() {
        King activeKing = findKing(this._color);
        if (activeKing != null)
            return activeKing.isInCheck(this._color, this._board);
    }

```

Regular Chess

```

return false;
}

/*
The func takes a list of chars and turn it to a full board of pieces
*/

public void turnToBoard() {
if (this._inputBoard == null || this._inputBoard.length() < 64) {
Log.e("CHESS_ERROR", "Board string is too short! Length: " +
(this._inputBoard == null ? 0 : this._inputBoard.length()));
return;
}

//the turn board logic
int txtIndex = 0;
for (int row = 0; row < 8; row++) {
for (int col = 0; col < 8; col++) {
char type = this._inputBoard.charAt(txtIndex);
Point p = new Point(row, col);

if (type == ' ' || type == '0') {
this._board[row][col] = null;
} else {
char color = Character.isLowerCase(type) ? Constants.WHITE_PIECE : Constants.BLACK_PIECE;
char lowerType = Character.toLowerCase(type);

if (lowerType == Constants.KING) this._board[row][col] = new King(p, color, type);
else if (lowerType == Constants.KNIGHT) this._board[row][col] = new Knight(p, color, type);
else if (lowerType == Constants.PAWN) this._board[row][col] = new Pawn(p, color, type);
else if (lowerType == Constants.QUEEN) this._board[row][col] = new Queen(p, color, type);
else if (lowerType == Constants.ROOK) this._board[row][col] = new Rook(p, color, type);
else if (lowerType == Constants.BISHOP) this._board[row][col] = new Bishop(p, color, type);
}
txtIndex++;
}
}
}

// checks if the piece that just successfully moved is a pawn landing on its promotion row
public boolean checkLastMoveForPromotion() {
if (_p1 == null) return false;

int targetRow = _p1.getRow();
Piece lastMovedPiece = _board[targetRow][_p1.getCol()];

if (lastMovedPiece instanceof Pawn) {

// White Pawns move DOWN to row index 7
if (targetRow == 7 && lastMovedPiece.getColor() == Constants.WHITE_PIECE) {
Log.d("CHESS_PROMOTION", "White pawn reached row 7! Opening Dialog...");
return true;
}

// Black Pawns move UP to row index 0
if (targetRow == 0 && lastMovedPiece.getColor() == Constants.BLACK_PIECE) {
Log.d("CHESS_PROMOTION", "Black pawn reached row 0! Opening Dialog...");
return true;
}
}
}

```

Regular Chess

```

    }
    }
    return false;
    }

    /*
    The func promote the pawn on the board by
    turning the pawn to queen, bishop, knight or a rook
    */
    public void promotePawnOnBoard(char absolutePiece) {
        if (_p1 == null) return; // Exit early safely if there's no point context

        int row = _p1.getRow();
        int col = _p1.getCol();
        Point p = new Point(row, col);

        // Determine color context by checking the case format style of the character parameter
        char pieceColor = Character.isLowerCase(absolutePiece) ? Constants.WHITE_PIECE :
            Constants.BLACK_PIECE;
        char lowerType = Character.toLowerCase(absolutePiece);

        if (lowerType == Constants.QUEEN) this._board[row][col] = new Queen(p, pieceColor, absolutePiece);
        else if (lowerType == Constants.ROOK) this._board[row][col] = new Rook(p, pieceColor, absolutePiece);
        else if (lowerType == Constants.KNIGHT) this._board[row][col] = new Knight(p, pieceColor, absolutePiece);
        else if (lowerType == Constants.BISHOP) this._board[row][col] = new Bishop(p, pieceColor, absolutePiece);

        Log.d("CHESS_PROMOTION", "Pawn at (" + row + ", " + col + ") promoted to char target: " + absolutePiece);
    }

    //set the color
    public void setColor(char c) { this._color = c; }
    //return the turn by the color
    public char getColor() { return _color; }
    //return the game board
    public Piece[][] getBoard() { return _board; }
    }

```

settingPage.java מחלקת

```

package com.example.chessgame;

import android.annotation.SuppressLint;
import android.content.Intent;
import android.os.Bundle;
import android.view.MotionEvent;
import android.view.View;
import android.widget.CompoundButton;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.SeekBar;
import android.widget.Switch;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.AppCompatButton;

```

Regular Chess

```

import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class settingPage extends AppCompatActivity implements View.OnClickListener {

    //fields
    ImageButton backbtn;
    Switch switchmusiccontrol;
    EditText editusername;
    AppCompatActivity btn_delete_account;
    SeekBar seekbarvolume;

    //this func construct and check if the seekbar is changing
    // if it does he will increase or decrease the volume of the background music
    @SuppressWarnings({"MissingInflatedId", "ClickableViewAccessibility"})
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.settingpage);

        seekbarvolume = findViewById(R.id.seekbarvolume);

        int initialProgress = (int) (BackgroundMusicService.currentVolume * 100);
        seekbarvolume.setProgress(initialProgress);

        seekbarvolume.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
                // Map the integer progression scale back down to fractional volume ranges
                float scalarVolume = progress / 100f;

                // Ship the updated calculation directly down to your background Service pipeline
                Intent volumeIntent = new Intent(settingPage.this, BackgroundMusicService.class);
                volumeIntent.setAction(BackgroundMusicService.ACTION_SET_VOLUME);
                volumeIntent.putExtra(BackgroundMusicService.EXTRA_VOLUME_VALUE, scalarVolume);
                startService(volumeIntent);
            }

            @Override
            public void onStartTrackingTouch(SeekBar seekBar) {}

            @Override
            public void onStopTrackingTouch(SeekBar seekBar) {}
        });

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        // Initialize widgets
        switchmusiccontrol = findViewById(R.id.switchmusiccontrol);
        editusername = findViewById(R.id.editusername);
    }

```

Regular Chess

```

// Initialize Button and Bind Listeners
btn_delete_account = findViewById(R.id.btn_delete_account);
btn_delete_account.setOnClickListener(this);

// Click Compression Effect (scale: 0.92)
btn_delete_account.setOnTouchListener(new View.OnTouchListener() {
    @Override
    public boolean onTouch(View v, MotionEvent event) {
        if (event.getAction() == MotionEvent.ACTION_DOWN) {
            v.animate().scaleX(0.92f).scaleY(0.92f).setDuration(150).start();
        } else if (event.getAction() == MotionEvent.ACTION_UP || event.getAction() ==
            MotionEvent.ACTION_CANCEL) {
            v.animate().scaleX(1.0f).scaleY(1.0f).setDuration(600).start();
            if (event.getAction() == MotionEvent.ACTION_UP) {
                v.performClick();
            }
        }
        return false;
    }
});

backbtn = findViewById(R.id.backbtn);
backbtn.setOnClickListener(this);

// Populate local cache inputs
GlobalStat globalStat = (GlobalStat) getApplicationContext();
if (globalStat.userStats != null && globalStat.userStats.username != null) {
    editusername.setText(globalStat.userStats.username);
}

// Music toggle engine system state sync
switchmusiccontrol.setChecked(!BackgroundMusicService.isMusicMuted);
switchmusiccontrol.setOnCheckedChangeListener(new CompoundButton.OnCheckedChangeListener() {
    @Override
    public void onCheckedChanged(CompoundButton buttonView, boolean isChecked) {
        Intent musicIntent = new Intent(settingPage.this, BackgroundMusicService.class);
        if (isChecked) {
            BackgroundMusicService.isMusicMuted = false;
            startService(musicIntent);
        } else {
            BackgroundMusicService.isMusicMuted = true;
            stopService(musicIntent);
        }
    }
});

//this func move and act by the logic of the button that got pressed
@Override
public void onClick(View v) {
    if (backbtn == v) {
        String newUsername = editusername.getText().toString().trim();
        if (!newUsername.isEmpty()) {
            GlobalStat globalStat = (GlobalStat) getApplicationContext();
            globalStat.userStats.username = newUsername;
            DatabaseManager.updateUsername(globalStat.userStats.Uid, newUsername);
        }
    }
}

```


Regular Chess

```

import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.auth.AuthResult;
import com.google.firebase.auth.FirebaseAuth;

public class signinPage extends AppCompatActivity implements View.OnClickListener {

    //fields
    ImageButton backbtn;

    //firebase variables
    EditText emailtv, passwordtv;
    ImageButton signbtn;
    Context context;

    FirebaseAuth auth;

    //constructor
    @SuppressWarnings("MissingInflatedId")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.signin);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.sign), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        //bind buttons and textviews
        backbtn = findViewById(R.id.backbtn);
        backbtn.setOnClickListener(this);

        emailtv = findViewById(R.id.emailtv);
        passwordtv = findViewById(R.id.passwordtv);
        signbtn = findViewById(R.id.signbtn);

        context = signinPage.this;
        auth = FirebaseAuth.getInstance();

        signbtn.setOnClickListener(this);
    }

    //The func move to the desire activity and act by the button that got pressed
    @Override
    public void onClick(View v) {

        if(backbtn == v)
        {
            Intent intent = new Intent(signinPage.this, log_lobby.class);
            startActivity(intent);
            finish();
        }
    }
}

```


Regular Chess

```

//make sure the strings are not empty
        if(signbtn == v)
        {
            String txt_email = emailtv.getText().toString();
            String txt_password = passwordtv.getText().toString();
            if(TextUtils.isEmpty(txt_email) || TextUtils.isEmpty(txt_password))
            {
                Toast.makeText(context, "fields must not be empty", Toast.LENGTH_SHORT).show();
            }
            else if(txt_password.length() < 6){
                Toast.makeText(context, "password is too short", Toast.LENGTH_SHORT).show();
            }
            else {
                //try to register
                registerUser(txt_email, txt_password);
            }
        }

        /*
        The func try to register in the auth table
        input: email -> client desire email
        password -> client desire password
        output: or a new row in the auth table or toast failed
        */
        private void registerUser(String email, String password)
        {
            auth.createUserWithEmailAndPassword(email, password)
            .addOnCompleteListener(signinPage.this, new OnCompleteListener<AuthResult>() {
                @Override
                public void onComplete(@NonNull Task<AuthResult> task) {
                    if(task.isSuccessful())
                    {
                        Toast.makeText(context, "success", Toast.LENGTH_SHORT).show();
                        startActivity(new Intent(signinPage.this, log_lobby.class));
                    }
                    else {
                        Toast.makeText(context, "failed", Toast.LENGTH_SHORT).show();
                    }
                }
            });
        }
    }

```



Regular Chess

מחלקת Splash.java

```
package com.example.chessgame;

import android.content.Intent;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class Splash extends AppCompatActivity {

    // Define the delay time as a constant for clarity
    private static final long SPLASH_DELAY_MS = 3000;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.splash);

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets
                systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        //run the splash intent for three sec
        new Handler(Looper.getMainLooper()).postDelayed(new Runnable() {
            @Override
            public void run() {

                Intent intent = new Intent(Splash.this, log_lobby.class);
                startActivity(intent);

                finish();
            }
        }, SPLASH_DELAY_MS);
    }
}
```

Regular Chess

מחלקת statisticsPage.java

```
package com.example.chessgame;

import android.annotation.SuppressLint;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.ImageButton;
import android.widget.TextView;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class statisticsPage extends AppCompatActivity implements View.OnClickListener {
    //fields
    TextView winstv, losestv, stalematestv, timeplayedtv, gamesplayedtv;
    ImageButton backbtn;

    //constructor
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.statistics);

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        //bind all the text views and button
        winstv = findViewById(R.id.winstv);
        losestv = findViewById(R.id.losestv);
        stalematestv = findViewById(R.id.stalematestv);
        timeplayedtv = findViewById(R.id.timeplayedtv);
        gamesplayedtv = findViewById(R.id.gamesplayedtv);
        backbtn = findViewById(R.id.backbtn);
        backbtn.setOnClickListener(this);

        GlobalStat globalStat = (GlobalStat) getApplicationContext();

        // If we already loaded from Firebase before
        // show the cached values immediately — no "... flicker
        if (globalStat.statsLoadedFromFirebase && globalStat.userStats != null) {
            updateUI(globalStat.userStats);
        } else {
            showLoadingState();
        }

        // Always fetch fresh data from Firebase to stay up to date
        DatabaseManager.loadStatsFromFirebase(this, new DatabaseManager.DataStatusCallback() {
```

Regular Chess

```

@Override
public void onDataLoaded(UserStatistics stats) {
    updateUI(stats);
}
});
}

/*
The func shows lodaing stats if it hasent got anything from the firebase
input: none
output: none
*/

private void showLoadingState() {
    winstv.setText("...");
    losestv.setText("...");
    stalematestv.setText("...");
    gamesplayedtv.setText("...");
    timeplayedtv.setText("...");
}

/*
The func update all the text views by the stats from the globalstat
input: stats -> all of the user stats
output: none
*/

@SuppressWarnings("SetTextI18n")
private void updateUI(UserStatistics stats) {
    if (stats == null) return;
    runOnUiThread() -> {
        winstv.setText(String.valueOf(stats.wins));
        losestv.setText(String.valueOf(stats.losses));
        stalematestv.setText(String.valueOf(stats.stalemates));
        gamesplayedtv.setText(String.valueOf(stats.gamesPlayed));
        timeplayedtv.setText(stats.timespent + " min");
    });
}

//the func goes back to the lobby page if the backbtn was pressed
@Override
public void onClick(View v) {
    if (backbtn == v) {
        Intent intent = new Intent(statisticsPage.this, lobby.class);
        startActivity(intent);
        finish();
    }
}
}

```



Regular Chess

מחלקת UserStatistics.java

```
package com.example.chessgame;

public class UserStatistics {
    //stats fields
    public String Uid = "";
    public String username = "";
    public long gamesPlayed = 0;
    public long wins = 0;
    public long losses = 0;
    public long stalemates = 0;
    public long timespent = 0;

    //constructor
    public UserStatistics() {}
}
```

מחלקת dialog_game_over.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="320dp"
    android:layout_height="wrap_content"
    android:background="@android:color/white"
    android:orientation="vertical"
    android:padding="24dp"
    android:gravity="center"
    android:layout_gravity="center">

    <!-- Trophy or Victory Icon -->
    <ImageView
        android:id="@+id/dialogIcon"
        android:layout_width="80dp"
        android:layout_height="80dp"
        android:src="@android:drawable/btn_star_big_on"
        android:layout_marginBottom="16dp" />

    <!-- Status Title -->
    <TextView
        android:id="@+id/dialogTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="CHECKMATE!"
        android:textColor="#212121"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_marginBottom="8dp" />

    <!-- Winner Message -->
    <TextView
```

Regular Chess

```

        android:id="@+id/dialogMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="White wins the match!"
        android:textColor="#666666"
        android:textSize="16sp"
        android:gravity="center"
        android:layout_marginBottom="24dp" />

        <!-- Action Button -->
        <Button
        android:id="@+id/btnGoToLobby"
        android:layout_width="match_parent"
        android:layout_height="50dp"
        android:text="Return to Lobby"
        android:textColor="FFFFFF"
        android:backgroundTint="#779556"
        android:textSize="16sp"
        android:textStyle="bold" />
    </LinearLayout>

```

dialog_promotion.xml מחלקת

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical"
    android:padding="24dp"
    android:background="@color/eggshell_bg"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Pawn Promotion"
        android:textSize="20sp"
        android:textStyle="bold"
        android:textColor="#0E0E1A"
        android:layout_marginBottom="24dp" />

    <GridLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:columnCount="2"
        android:rowCount="2"
        android:alignmentMode="alignBounds">

        <ImageButton
            android:id="@+id/img_btn_queen"
            android:layout_width="80dp"
            android:layout_height="80dp"
            android:layout_margin="12dp"
            android:padding="8dp"
            android:background="?android:attr/selectableItemBackground"

```

Regular Chess

```

        android:scaleType="fitCenter"
        android:contentDescription="Queen"
    />

    <ImageButton
        android:id="@+id/img_btn_rook"
        android:layout_width="80dp"
        android:layout_height="80dp"
        android:layout_margin="12dp"
        android:padding="8dp"
        android:background="?android:attr/selectableItemBackground"
        android:scaleType="fitCenter"
        android:contentDescription="Rook"
    />

    <ImageButton
        android:id="@+id/img_btn_knight"
        android:layout_width="80dp"
        android:layout_height="80dp"
        android:layout_margin="12dp"
        android:padding="8dp"
        android:background="?android:attr/selectableItemBackground"
        android:scaleType="fitCenter"
        android:contentDescription="Knight"
    />

    <ImageButton
        android:id="@+id/img_btn_bishop"
        android:layout_width="80dp"
        android:layout_height="80dp"
        android:layout_margin="12dp"
        android:padding="8dp"
        android:background="?android:attr/selectableItemBackground"
        android:scaleType="fitCenter"
        android:contentDescription="Bishop"
    />

</GridLayout>
</LinearLayout>

```

instructionpage.xml מחלקת

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/main"
    android:background="@color/black">

    <ImageView
        android:id="@+id/imageView8"

```

Regular Chess

```

        android:layout_width="413dp"
        android:layout_height="737dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:srcCompat="@drawable/instructionpage"
    />

    <ImageButton
        android:id="@+id/backbtn"
        android:layout_width="96dp"
        android:layout_height="81dp"
        android:background="@null"
        android:scaleType="fitXY"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.972"
        app:srcCompat="@drawable/backbtn" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

lobby.xml מחלקת

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:id="@+id/main"
    android:layout_height="match_parent"
    android:background="@drawable/lobby">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="411dp"
        android:layout_height="739dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        android:visibility="invisible"
        app:srcCompat="@drawable/lobby" />

    <ImageButton
        android:id="@+id/logoutbtn"
        android:layout_width="321dp"
        android:layout_height="150dp"
        android:background="@null"
        android:scaleType="fitXY"
        app:layout_constraintBottom_toBottomOf="parent"

```


Regular Chess

```

        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.488"
        app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="@+id/imageView"
        app:layout_constraintVertical_bias="0.864"
        app:srcCompat="@drawable/logoutbtn" />

        <ImageButton
            android:id="@+id/playbtn"
            android:layout_width="306dp"
            android:layout_height="133dp"
            android:background="@null"
            android:scaleType="fitXY"
        app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintHorizontal_bias="0.495"
            app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="@+id/imageView"
            app:layout_constraintVertical_bias="0.146"
            app:srcCompat="@drawable/playbtn" />

        <ImageButton
            android:id="@+id/instructbtn"
            android:layout_width="319dp"
            android:layout_height="141dp"
            android:background="@null"
            android:scaleType="fitXY"
        app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintHorizontal_bias="0.423"
            app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="@+id/imageView"
            app:layout_constraintVertical_bias="0.374"
            app:srcCompat="@drawable/instructionbtn" />

        <ImageButton
            android:id="@+id/statbtn"
            android:layout_width="319dp"
            android:layout_height="141dp"
            android:background="@null"
            android:scaleType="fitXY"
        app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintHorizontal_bias="0.478"
            app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="@+id/imageView"
            app:layout_constraintVertical_bias="0.613"
            app:srcCompat="@drawable/statisticsbtn" />

        <ImageButton
            android:id="@+id/settingbtn"
            android:layout_width="152dp"
            android:layout_height="98dp"
            android:background="@null"
            android:scaleType="fitXY"
        app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"

```

Regular Chess

```

        app:layout_constraintHorizontal_bias="1.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.006"
        app:srcCompat="@drawable/settingbtn" />
        <androidx.cardview.widget.CardView
            android:id="@+id/usernameCard"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_margin="16dp"

            app:cardElevation="8dp"
            app:cardCornerRadius="12dp"

            app:cardBackgroundColor="#F0EAD6"

        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">

            <LinearLayout
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:paddingStart="16dp"
                android:paddingEnd="16dp"
                android:paddingTop="10dp"
                android:paddingBottom="10dp"
                android:gravity="center_vertical">

                    <ImageView
                        android:layout_width="18dp"
                        android:layout_height="18dp"
                        android:layout_marginEnd="8dp"
                        android:src="@android:drawable/ic_menu_myplaces"
                        app:tint="#555555" /> <TextView
                        android:id="@+id/usernameDisplayTv"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Guest"
                        android:textColor="#222222" android:textSize="16sp"
                        android:textStyle="bold" />

                </LinearLayout>

            </androidx.cardview.widget.CardView>
        </androidx.constraintlayout.widget.ConstraintLayout>

```

מחלקת log_lobby.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
        android:id="@+id/main"
        android:layout_width="match_parent"

```

Regular Chess

```

        android:layout_height="match_parent"
        android:background="@drawable/lobby">

                <ImageButton
                    android:id="@+id/loginbtn"
                    android:layout_width="256dp"
                    android:layout_height="120dp"
                    android:background="@null"
                    android:scaleType="fitXY"
                    app:layout_constraintBottom_toBottomOf="parent"
                    app:layout_constraintEnd_toEndOf="parent"
                    app:layout_constraintHorizontal_bias="0.496"
                    app:layout_constraintStart_toStartOf="parent"
                    app:layout_constraintTop_toTopOf="parent"
                    app:layout_constraintVertical_bias="0.536"
                    app:srcCompat="@drawable/loginbutton" />

                <ImageButton
                    android:id="@+id/signbtn"
                    android:layout_width="258dp"
                    android:layout_height="123dp"
                    android:background="@null"
                    android:scaleType="fitXY"
                    app:layout_constraintBottom_toBottomOf="parent"
                    app:layout_constraintEnd_toEndOf="parent"
                    app:layout_constraintHorizontal_bias="0.496"
                    app:layout_constraintStart_toStartOf="parent"
                    app:layout_constraintTop_toTopOf="parent"
                    app:layout_constraintVertical_bias="0.792"
                    app:srcCompat="@drawable/signinbutton" />

                <ImageView
                    android:id="@+id/imageView3"
                    android:layout_width="465dp"
                    android:layout_height="417dp"
                    app:layout_constraintBottom_toBottomOf="parent"
                    app:layout_constraintEnd_toEndOf="parent"
                    app:layout_constraintStart_toStartOf="parent"
                    app:layout_constraintTop_toTopOf="parent"
                    app:layout_constraintVertical_bias="0.0"
                    app:srcCompat="@drawable/welcome" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

מחלקת login.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:id="@+id/main"
    android:layout_height="match_parent"
    android:background="@drawable/lobby">

```

Regular Chess

```

<ImageView
    android:id="@+id/imageView6"
    android:layout_width="594dp"
    android:layout_height="648dp"
    android:scaleType="fitXY"
app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.502"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.349"
    app:srcCompat="@drawable/logpage" />

<ImageButton
    android:id="@+id/imageButton"
    android:layout_width="235dp"
    android:layout_height="203dp"
    android:background="@null"
    android:scaleType="fitXY"
app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.409"
    app:srcCompat="@drawable/loginbtn1" />

<ImageButton
    android:id="@+id/imageButton2"
    android:layout_width="239dp"
    android:layout_height="205dp"
    android:background="@null"
    android:scaleType="fitXY"
app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.604"
    app:srcCompat="@drawable/loginbtn2" />

<ImageButton
    android:id="@+id/loginbtn"
    android:layout_width="239dp"
    android:layout_height="205dp"
    android:background="@null"
    android:scaleType="fitXY"
app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.692"
    app:srcCompat="@drawable/coolerlogbtn" />

<EditText
    android:id="@+id/emailtv"
    android:layout_width="129dp"

```

Regular Chess

```

        android:layout_height="48dp"
        android:textSize="16sp"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.531"
    app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.421" />

        <EditText
        android:id="@+id/passwordtv"
        android:layout_width="133dp"
        android:layout_height="38dp"
        android:textSize="16sp"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.541"
    app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.581" />

        <TextView
        android:id="@+id/textView5"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:fontFamily="sans-serif-black"
        android:text="Enter Email"
        android:textSize="20sp"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
    app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.382" />

        <TextView
        android:id="@+id/textView6"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:fontFamily="sans-serif-black"
        android:text="Enter Password"
        android:textSize="20sp"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.529" />

        <ImageButton
        android:id="@+id/backbtn"
        android:layout_width="96dp"
        android:layout_height="81dp"
        android:background="@null"
        android:scaleType="fitXY"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.0"

```

Regular Chess

```

        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.972"
        app:srcCompat="@drawable/backbtn" />
    </androidx.constraintlayout.widget.ConstraintLayout>

```

playpage.xml מחלקת

```

        <?xml version="1.0" encoding="utf-8"?>
        <androidx.constraintlayout.widget.ConstraintLayout
            xmlns:android="http://schemas.android.com/apk/res/android"
            xmlns:app="http://schemas.android.com/apk/res-auto"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:background="#222222">

            <com.example.chessgame.ChessView
                android:id="@+id/chessBoard"
                android:layout_width="0dp"
                android:layout_height="0dp"
                app:layout_constraintBottom_toBottomOf="parent"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toTopOf="parent"
                app:layout_constraintDimensionRatio="1:1" />

            <ImageButton
                android:id="@+id/backbtn"
                android:layout_width="103dp"
                android:layout_height="98dp"
                android:background="@null"
                android:scaleType="fitXY"
                app:layout_constraintBottom_toBottomOf="parent"
                app:layout_constraintEnd_toEndOf="parent"
                app:layout_constraintHorizontal_bias="0.022"
                app:layout_constraintStart_toStartOf="parent"
                app:layout_constraintTop_toTopOf="parent"
                app:layout_constraintVertical_bias="0.909"
                app:srcCompat="@drawable/backbtn" />

        </androidx.constraintlayout.widget.ConstraintLayout>

```

settingpage.xml מחלקת

```

        <?xml version="1.0" encoding="utf-8"?>
        <androidx.constraintlayout.widget.ConstraintLayout
            xmlns:android="http://schemas.android.com/apk/res/android"
            xmlns:app="http://schemas.android.com/apk/res-auto"
            xmlns:tools="http://schemas.android.com/tools"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:id="@+id/main"
            android:background="@color/eggshell_bg"
            android:padding="24dp">

            <androidx.appcompat.widget.AppCompatButton
                android:id="@+id/btn_delete_account"

```

Regular Chess

```

        android:layout_width="315dp"
        android:layout_height="68dp"

        android:clickable="true"
        android:focusable="true"

        android:background="@drawable/btn_delete_bg"
        app:backgroundTint="@null"

        android:elevation="12dp"
        android:paddingStart="40dp"
        android:paddingTop="16dp"
        android:paddingEnd="40dp"
        android:paddingBottom="16dp"
        android:text="DELETE ACCOUNT"

        android:textColor="@color/btn_delete_color"
        android:textSize="16sp"
        android:textStyle="bold"

        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.793" />

        <TextView
            android:id="@+id/textttitle"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Settings"
            android:textColor="@color/black_ink"
            android:textSize="32sp"
            android:textStyle="bold"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            android:layout_marginBottom="32dp" />

        <TextView
            android:id="@+id/headerpreferences"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Preferences"
            android:textColor="@color/black_ink"
            android:textSize="18sp"
            android:textStyle="bold"
            android:layout_marginTop="32dp"
            app:layout_constraintTop_toBottomOf="@id/textttitle"
            app:layout_constraintStart_toStartOf="parent" />

        <TextView
            android:id="@+id/labelmusiccontrol"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Music Control"
            android:textColor="@color/black_ink"

```

Regular Chess

```

                                android:textSize="16sp"
                                android:layout_marginTop="16dp"
app:layout_constraintTop_toBottomOf="@id/headerpreferences"
                                app:layout_constraintStart_toStartOf="parent" />

                                <Switch
                                android:id="@+id/switchmusiccontrol"
                                android:layout_width="wrap_content"
                                android:layout_height="wrap_content"
                                android:checked="false"
                                android:minWidth="60dp"
                                android:minHeight="40dp"
                                android:thumbTint="@color/black_ink"
                                android:trackTint="#CCCCCC"
                                app:layout_constraintEnd_toEndOf="parent"
                                app:layout_constraintTop_toTopOf="@id/labelmusiccontrol"
                                app:layout_constraintBottom_toBottomOf="@id/labelmusiccontrol" />

                                <TextView
                                android:id="@+id/labelvolumecontrol"
                                android:layout_width="wrap_content"
                                android:layout_height="wrap_content"
                                android:text="Music Volume"
                                android:textColor="@color/black_ink"
                                android:textSize="16sp"
                                android:layout_marginTop="24dp"
                                app:layout_constraintTop_toBottomOf="@id/labelmusiccontrol"
                                app:layout_constraintStart_toStartOf="parent" />

                                <SeekBar
                                android:id="@+id/seekbarvolume"
                                android:layout_width="0dp"
                                android:layout_height="wrap_content"
                                android:max="100"
                                android:progress="50"
                                android:thumbTint="@color/black_ink"
                                android:progressTint="@color/black_ink"
                                android:progressBackgroundTint="#CCCCCC"
                                android:layout_marginTop="12dp"
                                app:layout_constraintTop_toBottomOf="@id/labelvolumecontrol"
                                app:layout_constraintStart_toStartOf="parent"
                                app:layout_constraintEnd_toEndOf="parent" />

                                <TextView
                                android:id="@+id/headeraccount"
                                android:layout_width="wrap_content"
                                android:layout_height="wrap_content"
                                android:text="Account"
                                android:textColor="@color/black_ink"
                                android:textSize="18sp"
                                android:textStyle="bold"
                                android:layout_marginTop="32dp"
                                app:layout_constraintTop_toBottomOf="@id/seekbarvolume"
                                app:layout_constraintStart_toStartOf="parent" />

```


Regular Chess

```

<TextView
    android:id="@+id/labeleditusername"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Edit Username"
    android:textColor="@color/black_ink"
    android:textSize="16sp"
    android:layout_marginTop="16dp"
    app:layout_constraintTop_toBottomOf="@id/headeraccount"
    app:layout_constraintStart_toStartOf="parent" />

<EditText
    android:id="@+id/editusername"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:inputType="textPersonName"
    android:hint="Enter new username..."
    android:text=""
    android:textColor="@color/black_ink"
    android:textColorHint="@color/black_ink"
    android:background="@drawable/edittext_border_black"
    android:padding="12dp"
    android:layout_marginTop="8dp"
    app:layout_constraintTop_toBottomOf="@id/labeleditusername"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

<TextView
    android:id="@+id/headerdangerzone"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Danger Zone"
    android:textColor="@color/black_ink"
    android:textSize="18sp"
    android:textStyle="bold"
    android:layout_marginTop="32dp"
    app:layout_constraintTop_toBottomOf="@id/editusername"
    app:layout_constraintStart_toStartOf="parent" />

<TextView
    android:id="@+id/textdeletewarning"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="Once you delete your account, there is no going back. Please be certain."
    android:textColor="@color/black_ink"
    android:textSize="14sp"
    android:layout_marginTop="8dp"
    app:layout_constraintTop_toBottomOf="@id/headerdangerzone"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

<ImageButton
    android:id="@+id/backbtn"
    android:layout_width="96dp"
    android:layout_height="81dp"

```

Regular Chess

```

        android:background="@null"
        android:scaleType="fitXY"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.0"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.978"
    app:srcCompat="@drawable/backbtn" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

מחלקת signin.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:id="@+id/sign"
    android:layout_height="match_parent"
    android:background="@drawable/lobby">

    <ImageView
        android:id="@+id/imageView6"
        android:layout_width="594dp"
        android:layout_height="648dp"
        android:scaleType="fitXY"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.502"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.349"
    app:srcCompat="@drawable/signinpage" />

    <ImageView
        android:id="@+id/imageButton"
        android:layout_width="235dp"
        android:layout_height="203dp"
        android:background="@null"
        android:scaleType="fitXY"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.409"
    app:srcCompat="@drawable/loginbtn1" />

    <ImageView
        android:id="@+id/imageButton2"
        android:layout_width="239dp"
        android:layout_height="205dp"

```

Regular Chess

```

        android:background="@null"
        android:scaleType="fitXY"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.604"
        app:srcCompat="@drawable/loginbtn2" />

    <ImageButton
        android:id="@+id/signbtn"
        android:layout_width="345dp"
        android:layout_height="213dp"
        android:background="@null"
        android:scaleType="fitXY"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.7"
    app:srcCompat="@drawable/coolersignbtn" />

    <EditText
        android:id="@+id/emailtv"
        android:layout_width="135dp"
        android:layout_height="37dp"
        android:textSize="16sp"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.536"
    app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.43" />

    <EditText
        android:id="@+id/passwordtv"
        android:layout_width="131dp"
        android:layout_height="41dp"
        android:textSize="16sp"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.528"
    app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.581" />

    <TextView
        android:id="@+id/textView5"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:fontFamily="sans-serif-black"
        android:text="Enter Email"
        android:textSize="20sp"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.498"
    app:layout_constraintStart_toStartOf="parent"

```

Regular Chess

```

        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.386" />

        <TextView
            android:id="@+id/textView6"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="sans-serif-black"
            android:text="Enter Password"
            android:textSize="20sp"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintHorizontal_bias="0.507"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintVertical_bias="0.531" />

        <ImageButton
            android:id="@+id/backbtn"
            android:layout_width="96dp"
            android:layout_height="81dp"
            android:background="@null"
            android:scaleType="fitXY"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintHorizontal_bias="0.0"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintVertical_bias="0.972"
            app:srcCompat="@drawable/backbtn" />
    </androidx.constraintlayout.widget.ConstraintLayout>

```

מחלקת splash.xml

```

    <?xml version="1.0" encoding="utf-8"?>
    <androidx.constraintlayout.widget.ConstraintLayout
        xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:app="http://schemas.android.com/apk/res-auto"
        xmlns:tools="http://schemas.android.com/tools"
        android:id="@+id/main"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:background="@drawable/splash"
        tools:context=".Splash">

    </androidx.constraintlayout.widget.ConstraintLayout>

```

מחלקת statistics.xml

```

    <?xml version="1.0" encoding="utf-8"?>
    <androidx.constraintlayout.widget.ConstraintLayout
        xmlns:android="http://schemas.android.com/apk/res/android"

```

Regular Chess

```

xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/main"
    android:background="#F0EAD6">

        <ImageView
            android:id="@+id/imageView7"
            android:layout_width="424dp"
            android:layout_height="420dp"
            android:scaleType="fitXY"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintHorizontal_bias="1.0"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintVertical_bias="0.498"
            app:srcCompat="@drawable/statisticspage" />

        <TextView
            android:id="@+id/winstv"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="TextView"
            android:textColor="#000000"
            android:textSize="18sp"
            android:textStyle="bold"
            app:layout_constraintBottom_toBottomOf="@id/imageView7"
            app:layout_constraintEnd_toEndOf="@id/imageView7"
            app:layout_constraintHorizontal_bias="0.231"
            app:layout_constraintStart_toStartOf="@id/imageView7"
            app:layout_constraintTop_toTopOf="@id/imageView7"
            app:layout_constraintVertical_bias="0.477" />

        <TextView
            android:id="@+id/losestv"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="TextView"
            android:textColor="#000000"
            android:textSize="18sp"
            android:textStyle="bold"
            app:layout_constraintBottom_toBottomOf="@id/imageView7"
            app:layout_constraintEnd_toEndOf="@id/imageView7"
            app:layout_constraintHorizontal_bias="0.514"
            app:layout_constraintStart_toStartOf="@id/imageView7"
            app:layout_constraintTop_toTopOf="@id/imageView7"
            app:layout_constraintVertical_bias="0.477" />

        <TextView
            android:id="@+id/stalematestv"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="TextView"
            android:textColor="#000000"
            android:textSize="18sp"

```

Regular Chess

```

                                android:textStyle="bold"
app:layout_constraintBottom_toBottomOf="@id/imageView7"
    app:layout_constraintEnd_toEndOf="@id/imageView7"
        app:layout_constraintHorizontal_bias="0.8"
app:layout_constraintStart_toStartOf="@id/imageView7"
    app:layout_constraintTop_toTopOf="@id/imageView7"
        app:layout_constraintVertical_bias="0.477" />

                                <TextView
                                android:id="@+id/timeplayedtv"
                                android:layout_width="wrap_content"
                                android:layout_height="wrap_content"
                                android:text="TextView"
                                android:textColor="#000000"
                                android:textSize="18sp"
                                android:textStyle="bold"
app:layout_constraintBottom_toBottomOf="@id/imageView7"
    app:layout_constraintEnd_toEndOf="@id/imageView7"
        app:layout_constraintHorizontal_bias="0.228"
app:layout_constraintStart_toStartOf="@id/imageView7"
    app:layout_constraintTop_toTopOf="@id/imageView7"
        app:layout_constraintVertical_bias="0.785" />

                                <TextView
                                android:id="@+id/gamesplayedtv"
                                android:layout_width="wrap_content"
                                android:layout_height="wrap_content"
                                android:text="TextView"
                                android:textColor="#000000"
                                android:textSize="18sp"
                                android:textStyle="bold"
app:layout_constraintBottom_toBottomOf="@id/imageView7"
    app:layout_constraintEnd_toEndOf="@id/imageView7"
        app:layout_constraintHorizontal_bias="0.514"
app:layout_constraintStart_toStartOf="@id/imageView7"
    app:layout_constraintTop_toTopOf="@id/imageView7"
        app:layout_constraintVertical_bias="0.785" />

                                <ImageButton
                                android:id="@+id/backbtn"
                                android:layout_width="96dp"
                                android:layout_height="81dp"
                                android:background="@null"
                                android:scaleType="fitXY"
                                app:layout_constraintStart_toStartOf="parent"
                                app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintTop_toBottomOf="@id/imageView7"
    app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintVertical_bias="0.85"
        app:srcCompat="@drawable/backbtn" />

                                </androidx.constraintlayout.widget.ConstraintLayout>

```